



REFEREX

Computer Science NEA Project

Matthew Emerson
Colchester Royal Grammar School

Table of Contents

Planning & Analysis.....	5
Project Proposal:.....	5
User Interviews:	6
Identifying a User:	6
Interviews:	6
Interview Summaries & Conclusions:	10
Modelling:	12
Current Process for Referees:	12
New Proposed Process:.....	12
Research:.....	15
Database Management:	15
Web Scraping:.....	20
Encryption & Hashing:	20
Data Structures:.....	22
Password Security:.....	23
Existing Solutions:.....	24
FA Full-Time:.....	24
Centre Circle:	28
Aims & Objectives:	30
Login System:	30
Web Scraper:	30
End-User Application:	31
Prototype:	33
Source Code:	33
Output:.....	38
User Interview for Prototype:.....	39
Interview Summary:.....	39
Critical Path:	40
Analysis & Planning Tasks:.....	40
Design Tasks:.....	40
Technical Solution Tasks:	41
Testing Tasks:	41
Evaluation Tasks:	41

Design	42
Overall Design Discussion:	42
Structure Diagram:	45
Class Diagram:	46
Human-Computer Interaction:.....	47
User Interaction:	47
Page Layouts:	48
Data Structures, Algorithms & Object-Oriented Programming:.....	53
Algorithms:	53
Object-Oriented Programming:.....	54
Data Structures:.....	69
Database Structure:	74
Database Structure:	74
Technical Solution	78
Main File:	78
Band A Technical Skills:.....	78
Code:	78
Utility Classes:	81
Band A Technical Skills:.....	81
Code:	81
Database Manager:	88
Band A Technical Skills:.....	88
Code:	88
Custom Data Structures:	94
Band A Technical Skills:.....	94
Code:	94
Web Scraper:	102
Band A Technical Skills:.....	102
Code:	102
Custom Objects:	106
Band A Technical Skills:.....	106
Code:	106
Full-Time Web Scraper:	114
Band A Technical Skills:.....	114
Code:	114

Login System:	123
Band A Technical Skills:.....	123
Code:	123
Page Classes:	127
Band A Technical Skills:.....	127
Code:	127
Testing	140
Testing Plan:	140
Tests:	140
Login System:	140
Web-Scraper:.....	142
End-User Application:	146
Testing Video & Results:	150
Testing Video:	150
Results:	150
Evaluation	151
Aims & Objectives:	151
Completeness:	151
How it was Achieved:.....	151
Exit Interview:	152
Improving the Solution:	155
Client & User Feedback:	155
My Opinion:	155
References	156
Glossary:	156
Bibliography:	157

Planning & Analysis

Project Proposal:

I have been a qualified football referee for the past four years (beginning in 2021), splitting my time between the Blackwater & Dengie Youth Football League, Essex & Suffolk Border League, Essex County Women's League, and the Essex Veterans League.

Whilst these leagues differ in format, each being a youth, open-age men's, open-age women's, and veterans' league, respectively, they are all similar in that they use the FA's Full-Time website for administration. This includes generating fixture lists, recording league tables and player statistics, and, most importantly for me, assigning match officials.

This is where the problem arises. The Full-Time website is outdated and difficult to navigate, with information split across many pages and inconsistent formatting and links between them. This makes information gathering difficult and time-consuming.

As a referee, I prefer to go into a game knowing as much about the two teams as possible in advance so I can plan for any potential flashpoints or problematic players. This is done through gathering information such as league table positions, top goal scorers, and frequently booked individuals.

However, I often don't do this because of the time it would take me to gather the information. It is split across as many as five pages, each difficult to filter, poorly laid out, and filled with irrelevant information.

As a result, I would like to develop a simple application explicitly designed for referees that will collate a referee's fixture list and present them with the information they require to referee the game in an easy-to-understand format so that they can go into the fixture with maximum preparation.

User Interviews:

Identifying a User:

My project is aimed at referees officiating in leagues that use the Full-Time website for administrative tasks and referee assignments. As a result, I have decided to gather a range of clients. These are Leo Weller (a local referee, the end-user), REFSIX (a refereeing app, a corporate client), and, more generally, the group chats for the leagues I officiate (whom I surveyed).

Interviews:

End-User Interview:

Introduction:

This project aims to referee officiating in leagues similar to mine and with varying experience levels. I plan to use my experience refereeing and combine it with the alternative view of a newer referee. For this reason, I have chosen Leo Weller as my primary client. He is a level 7 referee who officiates on the Blackwater & Dengie Youth Football League and the Essex & Suffolk Border League.

As a part of the preparation for this project, I interviewed him to find out what problems he encountered with the Full-Time website and what features they wanted my project to contain.

Interview:

Q1 – How long have you been refereeing?

“I have been refereeing for about a year and a half now.”

Q2 – Which leagues do you officiate on?

“I officiate on the Blackwater League and, more recently, the Border League.”

Q3 – What do you primarily use the Full-Time website for?

“I use the Full-Time website to get the minimum information about my assigned weekly fixtures.”

Q4 – You mentioned the ‘bare minimum’ information. What does that include?

“Normally, I just write down the two team names and the ground postcode.”

Q5 – Do you feel like taking that information makes you fully prepared for your games?

“If I’m being honest, I could probably be more prepared for games if I had more information about the games, but I find the FT website to be difficult to navigate, and so I don’t feel like it’s worth it to spend the time gathering the information.”

Q7 – So, you say the Full-Time website is hard to navigate. Can you elaborate?

“The information is spread across at least three pages, and the filtering options don’t work consistently. I have to go onto the referee pages for both leagues to get each set of fixtures, so it would be too slow to gather this information for each fixture.”

Q8 – What do you find challenging about the filtering options available on the Full-Time website?

“The filters reset themselves if used in the wrong order, and options such as filtering by the team are missing, making it harder to gather the information I want.”

Q9 – Would you be open to using an alternative app to the Full-Time website?

“Yes, I probably would. But I would need assurance that the app was secure.”

Q10 – Would you consider the app secure if it had a login system and encrypted important information?

“Yes, I would.”

Q11 – What features would you want the app to have to replace the Full-Time website?

“Well, I would expect the app to have a fixture list with games from all my leagues on there – sorted by date.”

Q12 – What information about teams would you want to be stored beside the info on the Full-Time website?

“Ideally, I would at least want kit colours and information about the home venue, like whether there are toilets and if they serve food and drink at the ground.”

End-User Interview:

Introduction:

I have also contacted REFSIX, which produces a refereeing app that most referees use. I did this whilst searching for previous solutions. When I reached out, I was put into contact with Luke Gardiner.¹

Interview:

Q13 – Have you found any way to work with the FA to upload fixtures to your app automatically?

“We have been trying to get official API access from the FA for years, but they aren't interested.”

Q14 – Have you attempted other methods to gather the information?

“A couple of companies are already pulling data from Full Time using a "screen scraping" technique. We were thinking of going for this approach ourselves soon.”

Q15 – What information does your app require for referees to add games?

“To add a game, a referee must add the competition name, date and time, venue address, and the home and away team names”

Q16 – And is all this information available on the FT site?

“No. Most of the information is, but you cannot get team colours off the site, so this would have to be added separately.”

¹ The CTO & Co-Founder of REFSIX ([REFSIX, 2022](#)).

Q17 – How would you like the extra information entered when creating a fixture?

“Well, ideally, it would all be done automatically by the system – I suppose that would mean that it would need to be stored separately so it can then be added to each request.”

Q18 – You mentioned that this extra information would be stored separately; how would this information be collected, and what would happen if some information wasn’t available?

“Well, I suppose each user could have the choice of entering the information if it hasn’t been used before, and then that could be stored for future use, or there could also be some default values that come with the program.”

Referee Survey:

Introduction:

I created a survey to send to the referee group chats I participate in to gain a broader range of opinions. I took similar questions to those asked in the client interviews but simplified them for the survey to make it faster to complete.

Results:

Q21 – Which leagues do you officiate on?

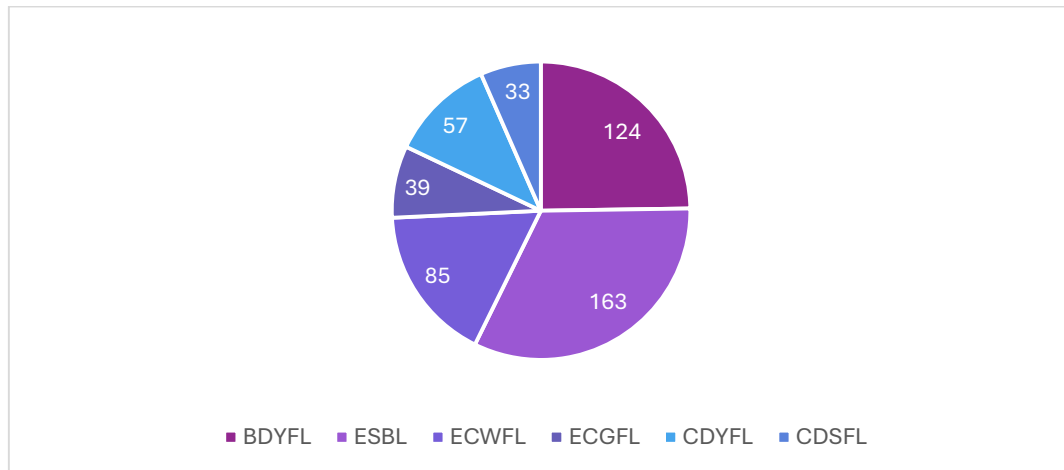


Figure 1

Q22 – Do you use REFSIX?

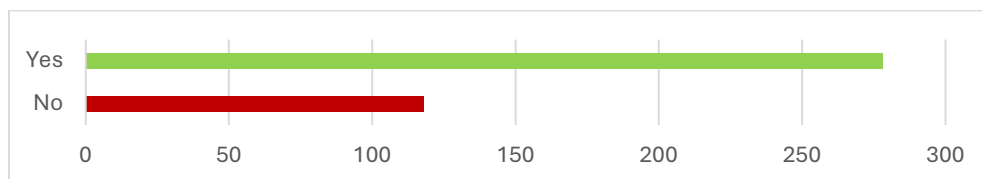


Figure 2

Q23 – Do you believe that the Full-Time website is easy to use?

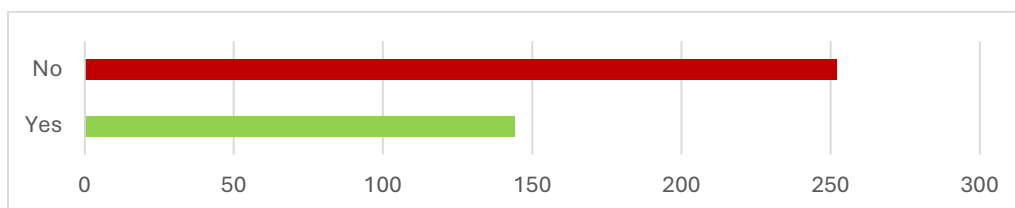


Figure 3

Q24 – Would you use an alternative app if it was available?

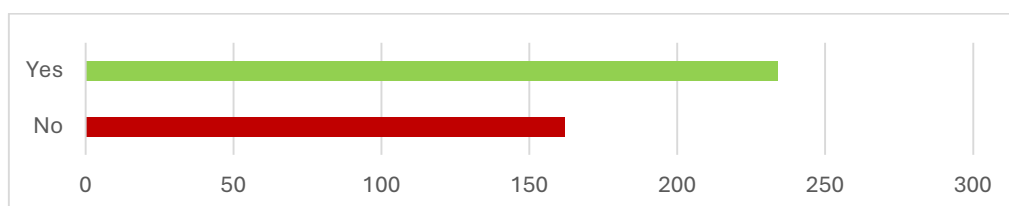


Figure 4

Interview Summaries & Conclusions:

End-User (Leo Weller):

Leo uses the Full-Time website to get team names and venue postcodes for each assigned fixture.

Leo finds the Full-Time website challenging to navigate. The information is spread across many pages, and he feels the filtering options offered don't work consistently and often reset if multiple filters are used at once. He would especially like a team filter on the stats page.

Leo would be open to using a 3rd party app to get his fixture list but would want it to be secure, so he would need the system to be encrypted and include a login system. He would also like the app to display the fixture information all in one place, with the potential for additional information not stored on the Full-Time website, such as kit colours and venue information, to be added from a database.

Conclusion:

Information to be collected from the Full-Time website:

- Team names.
- Venue address.

Additional information to be added:

- A database could add some information that isn't present on the Full-Time website, such as team colours and venue information.

App requirements:

- The app must be simple to navigate.
- Filtering options must be extensive and must work in conjunction with each other.
- The app must contain an encrypted login system to protect users' accounts.
- All the information relating to a fixture must be present on one screen.

Corporate Client (Luke Gardiner):

REFSIX have previously attempted to work with the FA to upload Full-Time information to their app but hasn't been given access. They are currently investigating "screen-scraping" techniques to pull information from the website and may implement this approach in the future.

When adding a game to their app, users must enter the competition name, date and time of the fixture, venue address, and the home and away team names.

The majority of this information is present on the Full-Time website. However the team colours aren't included, so Luke would want this information added to a database. Where data is missing, Luke suggests that the user would be prompted to either add it themselves or leave it blank.

Retrieving information from the Full-Time website:

- “Screen-scraping” techniques could be used to pull the information from the website.
- The system would need to collect:
 - o Competition name.
 - o Fixture date and time.
 - o Venue address.
 - o Team names.

Additional information to be added:

- Team colours.

Referee Survey:

The sample of referees officiates in various leagues that use the Full-Time website, so the app should have an option to add and remove leagues, with support for all leagues on the Full-Time website.

Many referees find the Full-Time website challenging to use and would be open to an easier-to-use replacement.

App requirements:

- The app must allow for adding and removing leagues.
- The app must be simple to use.

Modelling:

Current Process for Referees:

At the beginning of each week (generally on a Tuesday), each referee goes onto the Full-Time website and is prompted to enter a league name. Most referees will officiate in many leagues (I referee for four different leagues), so I must complete the following process multiple times.

Once on the page for a league, the user goes onto the 'Refs' sub-page and uses the 'Referee' filter to select themselves before attempting to use the 'Date' filter to choose the upcoming weekend. This filter frequently breaks, so the user must search through all their assigned fixtures for the season to find their next ones.

Then, the user must click on each fixture individually to visit their sub-page, where they can collect team names, the competition name, the venue address, and the kick-off date and time. If the user also wants table position and goal difference, they must traverse to the 'Table' sub-page and filter that for the competition provided on the fixture page. Finally, if the user wants statistic leaders for the team, they must visit the 'Stats' page and select the division. There is no filter for the team, so the user must search through the list themselves for each statistic they want information on.

This time-consuming process must then be repeated for each fixture in each league.

New Proposed Process:

Logging In:

Instead of visiting the Full-Time website at the beginning of each week, the referee can load the Referex app. Here, they will be prompted to log in or create an account.

If they choose to log in, they will be prompted to enter a username and password. An encryption algorithm will be applied to the entered details before entering into the parameters of an SQL SELECT query to search the database for the user ID relating to the given username and password. The home page will be loaded if a user ID is returned. Otherwise, the user will be prompted to try again with an error message. The user will be given three attempts to log in before being temporarily logged out of the system. Each failed login attempt will be logged with a time stamp and hashed username so the relevant user can be informed when they next log in. These failed attempts can also be viewed on the admin account.

If they choose to create an account, they will be prompted to enter a username and a password and to confirm their chosen password. The password will be validated against secure password standards to ensure system security. If it fails, an error message will be displayed, and the user will be prompted to enter a new set of details. Once the password is validated, details will be encrypted, and an SQL SELECT query will be run to compare the username to existing ones in the system. If the username already exists, an error message will be displayed, and the user will be prompted to enter a new set of details. If it is a new username, an SQL INSERT command will be run to add the details to the database, and a new user ID will be assigned.

Full-Time Scraper:

Once the app is logged in, it will periodically attempt to sync with the Full-Time system to keep the fixture information up-to-date. To work out which leagues to query, the system will run a cross-table parameterised SQL SELECT query to take information to gather a list of leagues that the user has previously submitted as leagues they wish to be loaded by the system.

For each league, the system will take the league ID from the database and the user's code if it has been previously collected and stored. If it has not been done previously, the system will search the 'select' tag relating to the 'Referee' drop-down box on the 'Refs' page for the league and take the code assigned to the user's name. It will then use an SQL UPDATE command to change the record for future use.

The system will then loop through each month of the season (August to June), gathering the fixture ID of each fixture assigned to the user. If a month is missing, the system will skip it to prevent crashes. The system will load each fixture's page and pull the division, team names, venue address, other match officials' names, and kick-off date and time.

The system will also query the database for team colours and additional venue information. If it finds the data, it will return it and store it with the fixture information. If not, the user will be presented with incomplete information and prompted to insert the missing information or leave it blank.

Page Layout & Navigation:

Once the user has successfully logged into the system, they will be presented with the home screen. Each page on the app consists of a navigation bar at the top, information in the middle, and an input bar at the bottom.

To navigate between pages, the user will enter the name or initial of the page they want to visit, and the current page will be added to the undo button's stack. If the user chooses to "press" the undo button, the current page will be added to the redo button's stack and the top page from the undo button's stack will be loaded. The inverse is also true for the redo button. Nothing will happen if the stack the user attempts to load from is empty. The text on the navigation bar for these buttons should be greyed out when the stacks are empty to indicate to the user that they won't do anything. Similarly, the current page's text will be coloured to indicate it is currently loaded, so attempting to navigate that page won't do anything.

The Home Page:

When on the home page, the user will be presented with status updates and statistics to monitor the web scraper and ensure it works as they expect it to. This will include the number of leagues currently loaded, calculated using a cross-table parameterised SQL COUNT query, and a count of the number of fixtures found and successfully loaded. It will also include the time stamp for the last web scrape attempt and whether it was successful. Finally, a notification counter will inform the user if they have any new messages, such as information about failed login attempts. This will also be implemented using a cross-table parameterised SQL COUNT query.

The user will also be allowed to log out of the system. If they choose to do so, they will be returned to the login page for the following user to use.

The Settings Page:

When on the settings page, the user can view and update the list of currently loaded leagues, view notifications, change their password, or delete their account.

If they choose to update their list of leagues, a cross-table parameterised SQL SELECT query will retrieve the list of leagues they have decided to load. They will then be outputted to the screen, and the user will be prompted to add or remove a league or to return to the settings page. If they choose to remove a league, the system will run an SQL DELETE command to remove the combination between the user account and the league. If the user decides to add a league, they will be prompted to enter the league's name. The system will compare the league name to its list of known leagues, and if it is already in the system, it will create the combination using an SQL INSERT command. If the league doesn't exist in the system, the user will be directed to go onto the Full-Time website and retrieve the league ID for the page from the URL of the league page. The system will then validate the ID that was entered. If it is correct, it will be added to the league page, and the combination will be created with SQL INSERT commands; otherwise, an error message will be displayed, and the user will be returned to the list of leagues.

If the user decides to view their notifications, a cross-table parameterised SQL SELECT will run to pull the list of notifications from the database and display them to the user. Each notification record will then show as viewed so they aren't counted in the new notification counter on the home page.

If the user chooses to update their password or delete their account, they will be prompted to reenter their current one. If the password is correct, they will either be prompted to enter a new password and to confirm it, which will be validated as before and then passed into an SQL UPDATE command to modify their current account details. Alternatively, if the user wants to delete their account, they will be prompted to confirm that they do want to delete their account before multiple SQL DELETE commands are run to remove all records relating to them from the system.

The Fixture List Page:

When users load the fixture list page, they are prompted to select filtering and/or sorting by either fixture date or competition. This will allow the user to choose the group of fixtures they wish to see displayed in their desired order. They would then be shown each fixture with the team names, venue address, competition name, and kick-off date and time.

Research:

My project will be divided into four significant areas: the supporting database and its management, web scraping from the Full-Time website, encrypting login details, and the data structures I will use throughout. In addition, I will research any current solutions to the problem and analyse their advantages and disadvantages.

Database Management:

C# Libraries:

I need a library to run SQL and design my database for my project. Ideally, the library shouldn't require any other libraries to run and should work quickly so it doesn't slow the app's running.

SQLite:

As a part of my research, I came across SQLite. On the webpage ([SQLite, 2024](#)), the developers claim that:

“SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine.”

As a result, I decided to investigate the library further. They back up their claims ([SQLite, 2021](#)) of the library being small where they state:

“The size of SQLite library is generally less than 1 megabyte. The size varies by compiler, operating system, CPU architecture, compile-time options, and other factors.”

[SQLite \(2021\)](#) also explains that their library is self-contained, which can be used without relying on other libraries besides the native C# methods.

According to the research ([Davies, 2022](#)), the library is faster than other alternatives. This makes it practical for the project, as data can be stored and retrieved faster than if I used default file-handling methods.

“SQLite – once warmed up – is roughly twice as fast as the file system.”

These factors made this library ideal for my project. The app should be able to run on any computer, and the speed minimises the time referees need to wait before they get their fixture list returned.

Due to how well the library has met my criteria and the extensive documentation on its website, I have decided to use this library within my project.

Types of Databases:

I have researched the different types of databases to decide how to structure my database to work optimally for the project. Ideally, the database will allow many connections between tables and be simple to implement through SQLite. [siddhant_baroth \(2020\)](#) lays out four different database types with advantages and disadvantages, so I have used them for my research.

Hierarchical Databases:

“This database follows the progression of data being categorised in ranks or levels, wherein data is categorised based on a common point of linkage.”

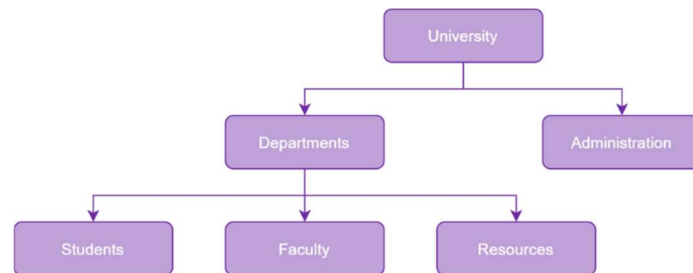


Figure 5

Advantages:

- Allows two objects that are unlike each other to have the same parent.
- Means each fixture can have a set of objects related to it.
- Each parent is allowed multiple child nodes.
- Lays out the structure of relationships between nodes.

Disadvantages:

- Child nodes can only have one parent.
- Child nodes cannot relate to each other.
- The addition of new data elements requires a lengthy traversal.

Network Databases:

“In Layman’s terms, a network database is hierarchical but with a major tweak. The child records are free to associate with multiple parent records.”

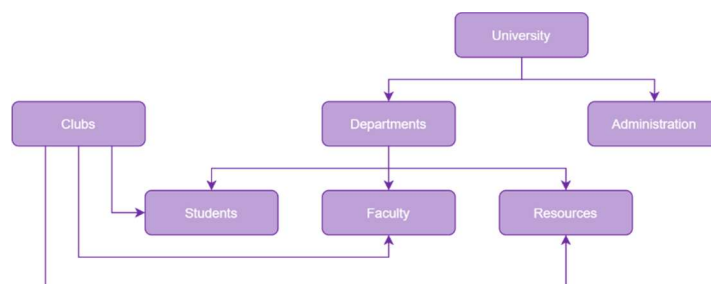


Figure 6

Advantages:

- Allows two objects that are unlike each other to have the same parent.
- Means each fixture can have a set of objects related to it.
- Each parent is allowed multiple child nodes.
- Lays out the structure of relationships between nodes.
- Can represent two-directional relationships.
- Nodes have the freedom to form relationships with multiple nodes.

Disadvantages:

- Cannot alter the structure of the database due to the complexity of the structure & the high structural dependency.

Object-Oriented Databases:

“Information stored in a database is capable of being represented as an object which responds as an instance of the database model.”

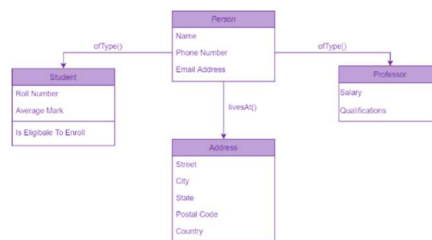


Figure 7

Advantages:

- Objects can be easily translated between the database and object-oriented code.
- Objects can be referenced and called without any difficulty.
- Workload on the database is significantly reduced.

Disadvantages:

- It is not easily translated into the SQLite database model due to how the library was written.

Relational Databases:

“In this database, every piece of information has a relationship with every other piece of information. This is because every data value in the database has a unique identity as a record.”

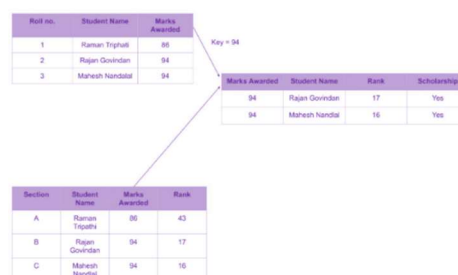


Figure 8

Advantages:

- Every row of data is linked to all others with its primary key.
- The SQL used to interact with it is simple to understand.
- The scaling and traversing are more straightforward than the hierarchical database model.

Disadvantages:

- Parent-child relationships cannot be explicitly created using this model.

Decision:

After considering these four options and their advantages and disadvantages, I have created a relational database, which will be the simplest to implement with SQLite. The format also allows me to efficiently manage the relationships between user accounts, leagues, and clubs.

Normalisation:

First Normal Form (1NF):

To be in 1NF, the database must:

- Only contain single-valued expressions (not multiple values in a single cell).
- The attribute domain does not change.
- There is a unique name for each attribute/column.
- The order in which data is stored does not matter.

Second Normal Form (2NF):

The database must be in 1NF to be in 2NF, and no relations must depend on partial dependencies (not part of a subject's unique key).

Third Normal Form (3NF):

To be in 3NF, the database must be in 2NF and have no transitive dependencies (if $A \rightarrow B$, $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency).

SQL:

My project will require various SQL commands and queries, so I have researched² Each one and its format.

INSERT:

The insert command is used to add a new record into a table.

Single Insert:

```
INSERT INTO table_name (column1, column2, column3...)
VALUES (value1, value2, value3...);
```

The column bracket isn't needed if values are inserted into every table column.

Multiple Insert:

```
INSERT INTO table_name (column1, column2, column3...)
VALUES (value1, value2, value3...), (value4, value5, value6...)...;
```

² The syntax and explanations have come from [W3Schools \(2019\)](#).

SELECT:

The select query is used to select data from a table.

```
SELECT column1, column2, column3...  
FROM table_name  
WHERE condition;
```

The 'WHERE' is optional, but without it, it will return every record in the table.

If you replace the list of columns with *, every column will be returned.

DELETE:

The update command is used to remove existing records from a table.

```
DELETE table_name  
WHERE condition;
```

The 'WHERE' is optional, but without it, it will delete every record in the table.

UPDATE:

The update command is used to modify existing records in a table.

```
UPDATE table_name  
SET column1 = value1, column2 = value2...  
WHERE condition;
```

The 'WHERE' is optional, but without it, it will update every record in the table.

COUNT:

The count query returns the number of rows in a table.

```
SELECT COUNT (column1)  
FROM table_name  
WHERE condition;
```

The 'WHERE' is optional, but without it, it will count every record in the table.

If you replace the list of columns with *, every column will be returned.

By adding 'DISTINCT' before the column name, it will only count the first occurrence of each value.

MAX:

JOINS:

Web Scraping:

My project will require information to be taken from the Full-Time website. During interviews, Luke Gardiner suggested “screen scraping” – more commonly known as web scraping.

According to [GeeksForGeeks \(2020\)](#), web scraping is an automatic process of gathering data from websites. The data is often returned in HTML and converted to a more usable format.

C# Libraries:

To use web scraping in my project, I will need a website scraping library compatible with C#. The library must have HTML parsing tools and good documentation explaining how to use it best.

HtmlAgilityPack:

While researching, I came across HTMLAgilityPack³. A C-language library that implements a web scraping interaction engine.

This library allows for parsing and manipulating HTML documents, such as searching for specific tags within the site, allowing for data collection. It also has good documentation of its various methods for quickly collecting and manipulating a website's HTML to gather the information requested by the program. As a result, it will be an adequate library for my project.

Encryption & Hashing:

I will have a user login system to store users’ saved leagues and preferences for this program. As a result, I will implement encryption algorithms when storing this data. I have found some encryption algorithms that can be implemented into C# from [codezone \(2023\)](#).

Hashing vs. Encryption:

Encryption algorithms are typically two-way processes used to protect data confidentiality. They are often used to preserve user preferences, whereas hash functions work one way to protect data integrity, such as by storing passwords.

AES (Advanced Encryption Standard):

AES is a symmetric encryption algorithm widely used for data encryption. It supports key lengths of 128, 192, or 256 bits.

Advantages:

- Most secure encryption protocol.
- Encryption processes are easy to learn.
- Uses longer keys, making it more secure.
- Fast encryption and decryption times.
- Consumes less memory and system resources than other protocols.

Disadvantages:

- If the AES key is ineffective, cryptoanalysis attacks could be possible.
- Each block is encrypted with the same algorithm.

³ See the page and documentation at [ZZZ Projects \(n.d.\)](#).

RSA (Rivest–Shamir–Adleman):

RSA is a widely used asymmetric encryption algorithm for secure data transmission and digital signatures.

Advantages:

- Keys aren't shared – increasing security.
- Faster than other encryption algorithms.
- Preserves data integrity during transfer.

Disadvantages:

- Slow data transfer rate.
- High processing is required for decryption.
- RSA can't be used for public data encryption.

DES (Data Encryption Standard):

DES is a symmetric encryption algorithm but is considered less secure than AES.

Advantages:

- The algorithm has been thoroughly analysed and adapted to cover up weaknesses.
- Relatively fast and efficient.

Disadvantages:

- Vulnerable to brute force attacks due to 56-bit key.
- A block size of 64 bits makes it vulnerable to block cypher attacks.

Triple DES (3DES):

Triple DES is a symmetric encryption algorithm that applies the DES algorithm three times to each data block.

Advantages:

- It's much more secure than DES.
- Widely used and backwards compatible with DES.
- Customisable key sizes make the algorithm less predictable and more secure.

Disadvantages:

- Slow speed due to the triple-layered encryption process.
- The maximum key size is one-hundred-and-ninety-two bits, limiting the protocol's security.

Secure Hash Algorithm (SHA256):

SHA256 is a cryptographic hash function which uses a one-way compression function with 256-bit digests.

Advantages:

- Reasonably secure compared to other similar algorithms.
- Very simple to implement into C# through 'System.Security.Cryptography'.
- SHA algorithms come in various digest sizes, making them less predictable.

Disadvantages:

- Published in 2001, while the algorithm is missing improvements in some areas, it is still considered cryptographically secure.

Decision:

After investigating the advantages and disadvantages of each of the encryption algorithms detailed above, I have chosen to use SHA256 due to its security and simplicity of implementation.

Data Structures:

Stacks:

A stack is a last-in-first-out data structure. The last item added (pushed) to the stack is the first to be removed (popped) off. I will use this throughout my project in the page navigation system for my end-user application.

I will do this by assigning stacks to the undo and redo buttons. This will allow me to push pages the user leaves onto the corresponding stack and pop the required page from the opposite stack.

Stack Operations:

- PUSH: The item is added to the top of the stack, and the top pointer is incremented.
- POP: The top item is returned from the stack, and the top pointer is decremented.
- PEEK: The value of the top item is returned from the stack.
- PRINT STACK: The value of all the items in the stack are outputted.

Linked Lists:

A linked list is a data structure consisting of nodes. Each node contains data and a reference (link) to the next node in the sequence. My project will use this instead of the native list data structure.

Linked List operations:

- GET INDEX: Returns the index of an inputted data element.
- GET NODE: Returns the node at an inputted index.
- INSERT AT BEGINNING: Inserts a new node with an inputted data value at the start of the list, moving the head node.
- INSERT AFTER: Inserts a new node with an inputted data value after the index.
- INSERT AT END: Inserts a new node onto the end of the list with the inputted data value.
- DELETE START: Removes the node at the start of the list.
- DELETE AT: Removes the node at an inputted index.
- DELETE END: Removes the node from the end of the list.
- MOVE: Takes the node at an inputted index and moves it to a different index.
- REPLACE ITEM: Replaces the data value at an inputted index.
- DISPLAY: Outputs the value of every item in the list.

Queues:

A queue is a first-in-first-out data structure. The first item added (enqueued) to the queue is the first to be removed (dequeued). I will use this throughout my project when processing fixtures and teams.

Queue Operations:

- ENQUEUE: Adds an element to the end of the queue and increments the rear pointer.
- DEQUEUE: Returns the first element from the queue and increments the front pointer.
- FRONT: Returns the first element from the queue without updating the pointers.
- REAR: Returns the element at the rear of the queue without updating the pointers.
- IS FULL: Checks whether the queue is full.
- IS EMPTY: Checks whether the queue is empty.
- SIZE: Returns the number of elements in the queue.

Password Security:

As requested in the user interviews, I will validate user passwords following standard password guidelines to ensure the security of user accounts. In the article by [Gracy, M. \(2024\)](#), the writer explains the National Institute of Standards & Technology's guidance for password security.

What To Do:

Password Length:

Passwords should be at least eight characters long and preferably fifteen characters. This length should be flexible, with only a minimum enforced to create more possible passwords.

Character Options:

Passwords should be able to accept spaces and all printable ASCII characters.

What To Stop:

Arbitrary Complexity Requirements:

Eliminate rigid guidelines such as using both upper- and lowercase characters, numbers, and symbols, as these requirements don't add much security.

Periodic Password Resets:

Don't reset passwords regularly, as constant resets lead to weaker passwords over time.

Password Hints & Security Questions:

Giving users password hints and security questions increases the chance of unauthorised users breaking into accounts through social engineering.

Truncating Passwords:

Always verify the full password rather than part of it. This ensures that the security obtained by the password's length isn't undermined.

Existing Solutions:

FA Full-Time:

FA Full-Time is the website from which I am web scraping information for my project. As seen above in the interviews⁴, the system is outdated and poorly formatted. The process for referees is time-consuming and repetitive and needs streamlining.⁵

The Referee Page:

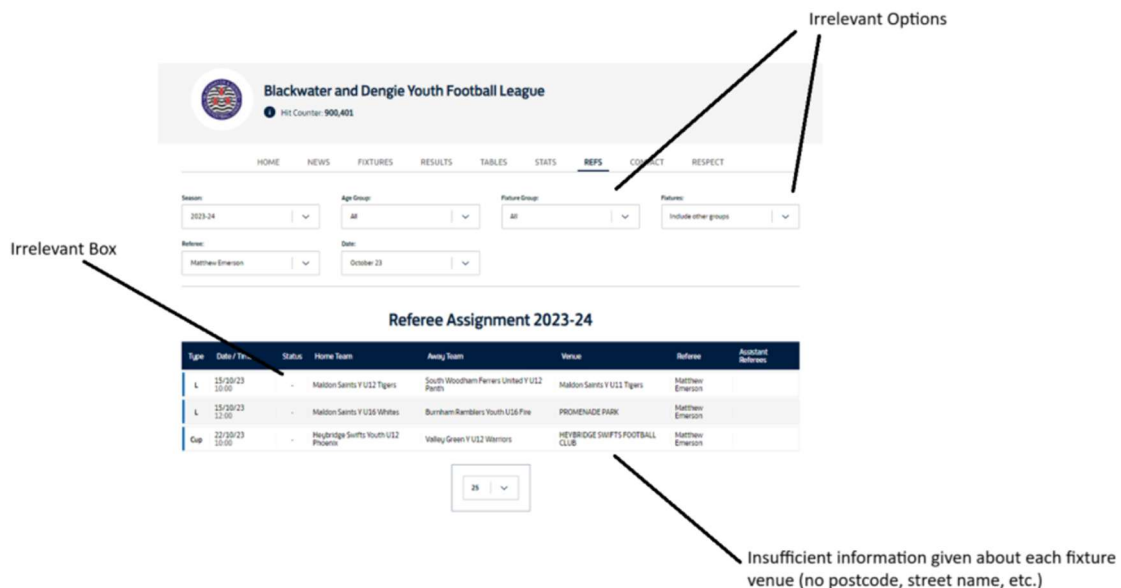


Figure 9

This page is poorly formatted, containing irrelevant dropdown boxes that don't remove fixtures. The fixture list area also has many empty sections missing essential information like complete addresses. This means referees must open each fixture individually to gather the minimum information needed for each game.

⁴ See Question 23

⁵ See Current Process

Fixtures Page:

Don't need both options

Season: 2023-24 Age Group: All Fixture Group: All

Date: May 24 Club: Denbury & Bicknacre Y Team: All

Fixtures: Include other groups and County Cups Fixture State Status: All Fixture Status: All

Irrelevant Options

Again, need more information

FIXTURES LIST FIXTURES GRID

Type	Date / Time	Home Team	Away Team	Venue	Competition	Status / Notes
L	05/05/24 10:00	Denbury & Bicknacre Y U9 Blues	VS	Broomfield Youth U9 Warriors	PRIORITY FIELDS	U9 Dangle Postponed
L	19/05/24 12:00	Denbury & Bicknacre Y U9 Royals	VS	Sporting 77 Y U9 Lions	PRIORITY FIELDS	U9 Crouch Postponed
L	26/05/24 10:00	Denbury & Bicknacre Y U9 Royals	VS	Maldon & Tiptree Youth U9 Tigers	PRIORITY FIELDS	U9 Crouch Postponed

25

Figure 10

Again, this page contains unnecessary dropdown options that don't add much to the filtering options while taking up valuable screen space that could be used for crucial missing information, such as the venue address. However, the fixture grid is a good resource as it allows users to find results between teams without needing to scroll through the list.

Results Page:

Expand tool good

Colour differentiation

Irrelevant Option

Full Results Table

Type	Date / Time	Home Team	Away Team	Competition
L	26/11/23 12:30	Broomfield Youth U14 Jaguars	3 - 1	Dunmow United Y U14 Vikings
CC	26/11/23 11:45	Walthamstow Youth U14 Eux	12 - 0	Talsley Youth U14 B&O
CC	19/11/23 14:00	Heybridge Swifts Youth U14 Phoenix	2 - 1	Roughedge Junior Y U14 Blues
CC	19/11/23 13:30	Pro Soccer Youth Development U14	9 - 0	Uplands Rangers Y U14 Blues
CC	19/11/23 10:00	Elsternham Youth U14 Ajax	4 - 6	Stamway Progress Youth U14 Blues
L	12/11/23 14:00	Writtle Minors Y U14 Greens	7 - 0	Dunmow United Y U14 Vikings

25

Figure 11

The results page is laid out more effectively than the fixtures page. It displays basic information about games while giving the option to show more through a button press. Additionally, the colour differentiation on the left tab makes finding specific competitions far quicker. However, the unnecessary dropdowns are still present at the top of the page.

Statistics Page:

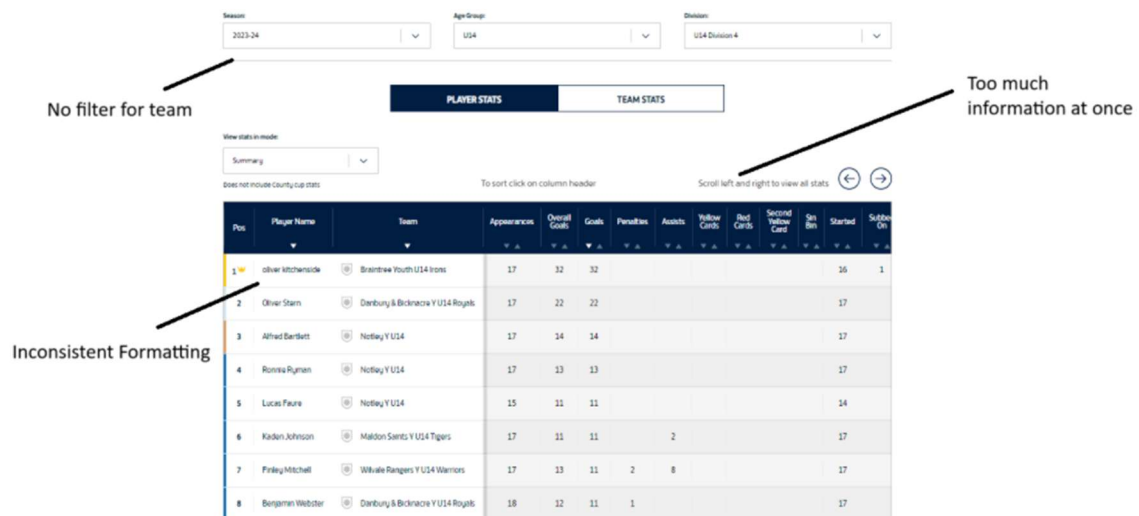


Figure 12

In contrast with the rest of the site, this page fails to provide enough filtering options – it does not allow you to filter by team. The player names are capitalised inconsistently, and the table takes up most of the screen and fails to demonstrate statistic leaders for multiple categories.

Table Page:

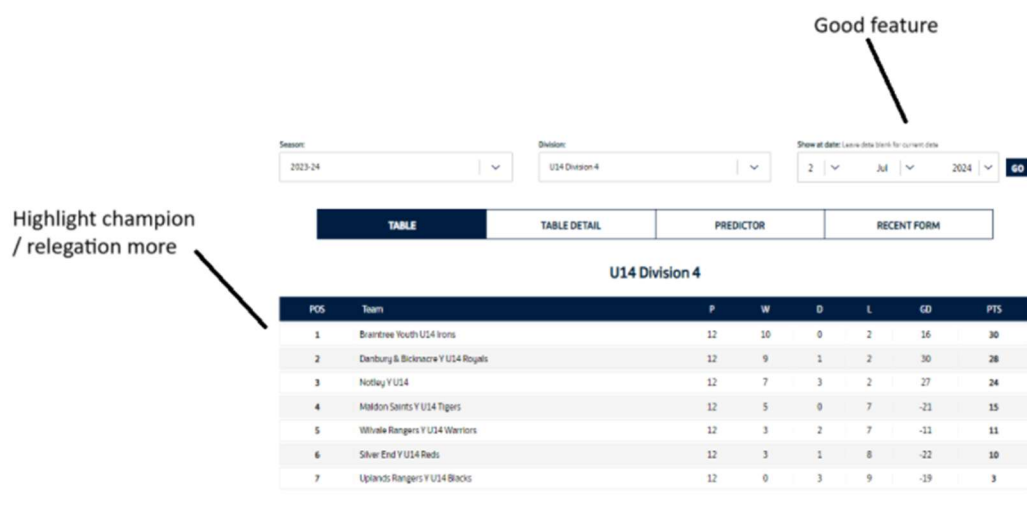


Figure 13

This page is the best-formatted page on the website, as the unnecessary filtering options have been removed, leaving only the minimum options and giving more space for the data to be displayed. However, a good feature could have been highlighting each division's promotion and relegation spots.

Individual Fixture Page:

Poor formatting

Missing club badges / colours

Uplands Rangers Y U14 Blacks

0 6

Notley Y U14

Could have more details here

Substitutes

Lineup

Lineup

Substitutes

Good design

Additional Stats

Repeated names, takes up too much space

Could be shown earlier in the page (is a higher priority)

Venue: MELBOURNE PARK

VIEW ON MAP

Match Officials

Doesn't actually work

Team	Player	Pos
Notley Y U14	Alfred Barker	Appearance
Notley Y U14	Alfred Barker	Overall Goals
Notley Y U14	Notley Thomas	Appearance
Notley Y U14	Notley Thomas	Overall Goals
Notley Y U14	Lucas Proux	Appearance
Notley Y U14	Notley Thomas	Appearance

Figure 14

This page continues with poor text formatting, the competition name containing underscores, and inconsistent capitalisation on player names. Also, placeholder badges are used in most leagues instead of the correct club ones. The page doesn't seem to prioritise the proper information, and venue address and match officials appear at the bottom. A list of repeated player names is placed near the top despite being unnecessary when coupled with the team lineups.

Centre Circle:

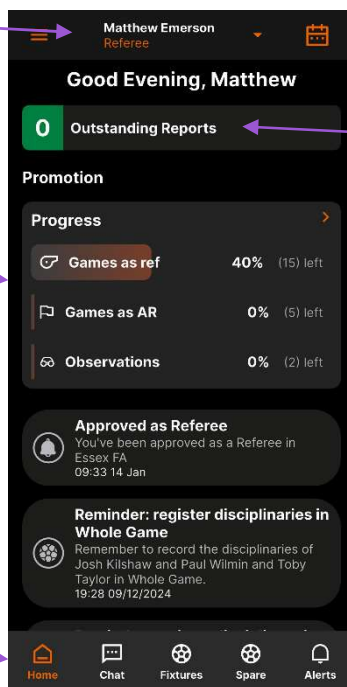
The Centre Circle app allows referees to manage their fixtures and submit match reports. It was designed more recently, meaning its present features are much slicker and easier to use. The app syncs with the Full-Time website, meaning referees can see their assigned games on the app. However, the app is not currently implemented in all of my leagues, with only the Blackwater & Dengie Youth Football League and the Essex Veterans League presently using it.

Home Page:

Displays who is logged in.

The promotion information is wrong as it doesn't include all my leagues.

Simple navigation bar.



Button to easily access the match report screen.

Notifications from the app displayed here.

Figure 15

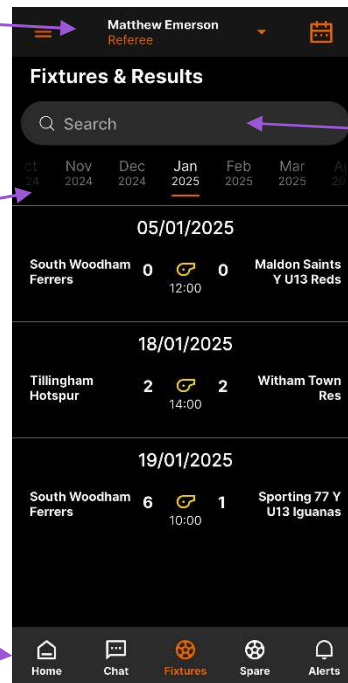
This page initially appears very well laid out with buttons and text clearly explaining what each feature does. However, the page has become cluttered as the developers have tried showing off too many features on the one page. Additionally, some information is completely wrong as the promotion box doesn't account for the two leagues where I actually accumulate progress towards my promotion.

Fixture List Page:

Displays who is logged in.

Easy filtering by month.

Simple navigation bar.



Search bar to look though the fixture list.

Fixture information is still missing the venue address.

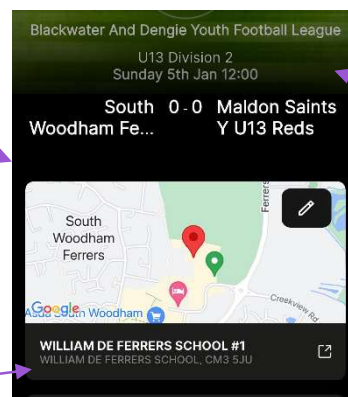
Figure 16

This page is very well laid out. It allows referees to view their fixtures and is filtered by month, with the option to search the entire list. However, the fixture information present is insufficient for a referee to prepare themselves for matchday as the venue addresses are missing from the page.

Fixture Page:

The team name is cut off as too long for the given box.

Address clearly written out in text with a link for directions.



Fixture information such as division and kick-off date and time are displayed.

Pin demonstrating exactly where the pitch is.

Figure 17

This page makes up for the previous lack of a venue address with three ways to view the venue – a text address, a link to Google Maps, and a pin on a map cutout. Additionally, the page displays the competition, division, and kick-off date and time. However, the team names can be cut off if they are longer than the very short character limit.

The app also has pages for picking up friendly games and methods for giving availability. However, the leagues I am currently on don't use this, so I have not been able to test these features.

Aims & Objectives:

Login System:

I aim to implement a login system to ensure user account security.

The system should hash any login details stored in the database to ensure they cannot be read directly from the database or decrypted and reentered into the system.

To prevent duplication, the system should validate new usernames against existing users during account creation.

Passwords should be checked to comply with NIST guidelines for increased account security.

Users should only be given a limited number of attempts to log in to the system before they are locked out, with the lock-out time increasing for each failed attempt to prevent brute-force attacks.

Failed login attempts should be logged in a table, and the related user should be informed of the failed logins when they next log in.

Web Scraper:

I aim to create a web scraper capable of retrieving requested information from the Full-Time website.

I aim to create a set of functions to format URLs to access any page within the Full-Time website.

The web scraper should have functions to format URLs for each of the site's sub-pages, considering inputted parameters such as the referee's name, date ranges, division, or team names.

The URL formatting functions should be able to deal with the differing URL formats of youth and open-age football leagues.

Any URLs created using the formatting functions should be checked to ensure the page loads correctly before it's passed over to the web scraper to search through.

I aim to create a set of functions to retrieve information from the HTML returned when the page URLs are loaded.

The system should have a function that takes an HTML tag and attribute as parameters and returns the relevant element from the HTML page for further processing.

A function should take a 'select' tag and translate it into a dictionary so that the drop-down boxes can filter the website pages based on text parameters.

A function should take a 'table' tag, a list of strings, and a column index and return the table rows where the value at the column index is in the list of strings.

I aim to use the list of leagues loaded by a user to collect a list of fixtures.

The system should run a cross-table parameterised SQL SELECT query on the user and league combination table to gather a list of leagues for the web scraper to search. If the league has been loaded previously for the user, this will return the league ID and the related referee ID.

If the referee ID isn't present, the drop-down translator should be used with the drop-down box of referees on the 'Refs' page to get it. The ID can then be placed in an SQL UPDATE command for future use.

Once the referee ID has been identified, the system should collect the fixture IDs for the league and add them to a queue for processing.

I aim to process each fixture to collect the information on the Full-Time website.

Each fixture should be taken from the queue, and its league ID should be used to generate the URL for its sub-page.

This sub-page should be searched to retrieve the fixture's division/cup name, kick-off date and time, venue address, team names, and match officials' names.

I aim to supplement the information on the Full-Time website with additional information stored in database tables.

A database table should contain a list of clubs and their associated kit colours.

The system should take the information from these tables (using SELECT SQL queries) and apply it in coordination with the fixture information from Full-Time to create a more comprehensive set of fixture information.

End-User Application:

I aim to create an end-user application that allows referees to view their upcoming fixtures for multiple leagues simultaneously faster.

I aim to implement a simple navigation system so the user can move between pages quickly and easily traverse back to previously visited pages.

The navigation bar should display all pages that can be visited and the information that must be entered to get there.

The navigation bar should highlight the current page and grey out the undo and redo buttons when they cannot be used.

The input bar should accept the page's name, initial, or page number while rejecting invalid inputs.

Once a valid input has been entered, the relevant page should be loaded, and the current page should be added to the undo button's stack.

When the undo button is pressed, the current page should be added to the redo button's stack, and a page should be popped from the undo button's stack and loaded. The opposite should also be valid for the redo button.

I aim to create a home page that the user can use to monitor the status of the Full-Time web scraper.

The home page should display the number of leagues the user has chosen to load. This should be implemented using an SQL COUNT query on the database.

The page should display the current number of fixtures located and processed.

The page should display the time stamp for the last time the web-scraper attempted to run and whether it succeeded.

The home page should have an additional option for logging out of the system. The user should be returned to the login screen when this is used.

I aim to create a settings page for users to edit their list of leagues, change their passwords, and delete their accounts.

The settings page should display the complete list of loaded leagues and offer to edit the list.

If the user chooses to edit the list, they should be able to choose between removing or adding a league.

If the user chooses to remove a league, they should be prompted to enter the league's name from their list. If a valid name is entered, an SQL DELETE command should remove the combination from the database table. The user should be returned to the list editing screen if the entered name is invalid.

If the user chooses to add a league, they should be prompted to enter the league's name. An SQL SELECT query should be used to check for the league ID in the database. If the query finds one, the combination can be inserted into the table. Otherwise, the user will be directed to the Full-Time website to get the league ID. The ID entered will then be tested, and if it's valid, the league will be added to the list.

If the user chooses to edit their password or delete their account, they will be prompted to enter their current password. If the login system validates it, then they will be allowed to proceed. Else, they will be returned to the settings page.

If the user is trying to change their password, they will be prompted to enter a new password and to confirm it. If the password is valid, an SQL UPDATE command will update the login details on the system.

If the user wants to delete their account, multiple SQL DELETE commands will remove their login details and any other records with their user ID present.

I aim to create a fixture list page where users can view the list of assigned fixtures based on their leagues and filtering preferences.

When users enter the fixture list page, they should be presented with the option of filtering or sorting their fixture list by date and/or competition.

The fixture list should be sorted using a recursive merge sort algorithm.

Each fixture should display the team names, competition and division names, the venue address, and the kick-off date and time.

Prototype:

For the prototype, I wanted to create a basic version of the Full-Time web scraper to collect the fixture list for a league of the user's choosing. It would be sorted by month and would output home and away teams, as well as the fixture ID.

Source Code:

```
1 using HtmlAgilityPack;
2
3 namespace Referex_Prototype
4 {
5     internal class Program
6     {
7         static void Main(string[] args)
8         {
9             (string leagueId, string leagueName) = GetLeagueId();
10
11             Console.Clear();
12             Console.WriteLine($"League Loaded - {leagueName}\n");
13
14             (bool valid, HtmlDocument leagueHomePage) =
15 TestPage($"https://fulltime.thefa.com/index.html?league={leagueId}");
16
17             Dictionary<string, string> seasonIds =
18 GenerateDropdownDictionary(leagueHomePage, "form1_selectedSeason");
19
20             string seasonId = seasonIds["2024-25"];
21
22             (string refereeId, string refereeName) = GetRefereeId(leagueId,
23 seasonId);
24
25             Console.Clear();
26             Console.WriteLine($"League Loaded - {leagueName}\n");
27             Console.WriteLine($"Getting Fixture List for {refereeName}...\n");
28
29             GetFixtureIds(seasonId, refereeId);
30         }
31
32
33         static Tuple<string, string> GetLeagueId()
34         {
35             bool validId = false;
36
37             string leagueId = "";
38             string leagueName = "";
39             HtmlDocument leagueHomePage = new HtmlDocument();
40
41             while (validId == false)
42             {
43                 Console.Write("Enter the league ID: ");
44                 leagueId = Console.ReadLine();
45
46                 (validId, leagueHomePage) =
47 TestPage($"https://fulltime.thefa.com/index.html?league={leagueId}");
48
49                 if (leagueHomePage == null)
50                 {
51                     continue;
52                 }
53             }
54 }
```

```

55         (bool found, HtmlNode leagueNameDiv) =
56         TryGetNodeFromAttribute(leagueHomePage, "div", "class", "league-name");
57
58         leagueName = FormatString(leagueNameDiv.InnerText);
59
60         return Tuple.Create(leagueId, leagueName);
61     }
62
63
64     static Tuple<bool, HtmlNode> TryGetNodeFromAttribute(HtmlDocument page,
65     string nodeTag, string attributeName, string attributeValue)
66     {
67         HtmlNodeCollection nodes =
68         page.DocumentNode.SelectNodes($"//{nodeTag}");
69
70         if (nodes == null)
71         {
72             return Tuple.Create(false, page.DocumentNode);
73         }
74
75         foreach (HtmlNode potentialNode in nodes)
76         {
77             if (potentialNode.GetAttributeValue(attributeName, "Default
78 Return Value") == attributeValue)
79             {
80                 return Tuple.Create(true, potentialNode);
81             }
82         }
83
84         return Tuple.Create(false, page.DocumentNode);
85     }
86
87
88     static List<HtmlNode> TryGetNodesFromTag(HtmlDocument page, string
89     nodeTag)
90     {
91         HtmlNodeCollection nodes =
92         page.DocumentNode.SelectNodes($"//{nodeTag}");
93         List<HtmlNode> collectedNodes = new List<HtmlNode>();
94
95         if (nodes == null)
96         {
97             return collectedNodes;
98         }
99
100        foreach (HtmlNode potentialNode in nodes)
101        {
102            collectedNodes.Add(potentialNode);
103        }
104
105        return collectedNodes;
106    }
107
108
109    static Tuple<bool, HtmlDocument> TestPage(string url)
110    {
111        HtmlWeb web = new HtmlWeb();
112        HtmlDocument page = web.Load(url);
113
114        (bool invalidPage, HtmlNode node) = TryGetNodeFromAttribute(page,
115        "section", "id", "error-page");
116
117        if (invalidPage == true)

```

```

118         {
119             return Tuple.Create(false, new HtmlDocument());
120         }
121
122         return Tuple.Create(true, page);
123     }
124
125
126     static string FormatString(string inputString)
127     {
128         string outputString = "";
129
130         int whitespaceCount = 0;
131         foreach (char c in inputString.Trim())
132         {
133             if (char.IsWhiteSpace(c))
134             {
135                 whitespaceCount++;
136             }
137             else
138             {
139                 whitespaceCount = 0;
140             }
141
142
143             if (whitespaceCount == 2)
144             {
145                 break;
146             }
147             outputString += c;
148         }
149
150         return outputString.Replace("&", "&");
151     }
152
153
154     public static Dictionary<string, string>
155     GenerateDropdownDictionary(HtmlDocument page, string dropdownId)
156     {
157         Dictionary<string, string> dropdownOptions = new Dictionary<string,
158 string>();
159
160         (bool foundNode, HtmlNode dropdown) = TryGetNodeFromAttribute(page,
161 "select", "id", dropdownId);
162
163         if (foundNode == false)
164         {
165             Console.WriteLine("Dropdown box not found.\n");
166         }
167
168         foreach (HtmlNode option in dropdown.ChildNodes)
169         {
170             string optionValue = option.GetAttributeValue("value", "-");
171             string optionId = option.InnerText;
172
173             if (optionValue == "-") // If the dropdown value isn't found.
174             {
175                 continue;
176             }
177
178             if (dropdownOptions.ContainsKey(optionId) == false)
179             {
180                 dropdownOptions.Add(optionId, optionValue);

```

```

181     }
182     }
183
184     return dropdownOptions;
185 }
186
187
188     static Tuple<string, string> GetRefereeId(string leagueId, string
189 seasonId)
190     {
191         bool validId = false;
192
193         string refereeId = "";
194         string refereeName = "";
195
196         (bool foundPage, HtmlDocument leagueRefereePage) =
197 TestPage($"https://fulltime.thefa.com/referees.html?league={leagueId}&selectedSea
198 son={seasonId}");
199         Dictionary<string, string> refereeIds =
200 GenerateDropdownDictionary(leagueRefereePage, "form1_selectedReferee");
201
202         while (validId == false)
203         {
204             Console.WriteLine($"{refereeIds.Count} referees found\n");
205             Console.Write("Enter the referee's name: ");
206             refereeName = Console.ReadLine();
207
208             if (refereeIds.ContainsKey(refereeName))
209             {
210                 refereeId = refereeIds[refereeName];
211                 validId = true;
212             }
213         }
214
215         return Tuple.Create(refereeId, refereeName);
216     }
217
218
219     static List<string> GetFixtureIds(string seasonId, string refereeId)
220     {
221         (bool foundPage, HtmlDocument refereePage) =
222 TestPage($"https://fulltime.thefa.com/referees.html?selectedSeason={seasonId}&sel
223 ectedFixtureGroupAgeGroup=0&selectedFixtureGroupKey=1_595992329&selectedRelatedFi
224 xtureOption=2&selectedReferee={refereeId}&selectedDateCode=all&previousSelectedFi
225 xtureGroupAgeGroup=&previousSelectedFixtureGroupKey=1_595992329&previousSelectedR
226 eferree=&previousSelectedDateCode=all");
227         Dictionary<string, string> possibleDateCodes =
228 GenerateDropdownDictionary(refereePage, "form1_selectedDateCode");
229
230         List<string> fixtureList = new List<string>();
231
232         foreach (KeyValuePair<string, string> dateCodes in possibleDateCodes)
233         {
234             if (dateCodes.Value.Length == 16)
235             {
236                 (foundPage, HtmlDocument monthPage) =
237 TestPage($"https://fulltime.thefa.com/referees.html?&selectedSeason={seasonId}&se
238 lectedFixtureGroupAgeGroup=0&selectedFixtureGroupKey=&selectedRelatedFixtureOptio
239 n=2&selectedReferee={refereeId}&selectedDateCode={dateCodes.Value}&previousSelect
240 edFixtureGroupAgeGroup=0&previousSelectedFixtureGroupKey=&previousSelectedReferee
241 ={refereeId}&previousSelectedDateCode=all");
242
243                 Console.WriteLine($"{\n{dateCodes.Key}\n");

```

```

244         List<HtmlNode> linkNodes = TryGetNodesFromTag(monthPage,
245 "a");
246
247         foreach (HtmlNode linkNode in linkNodes)
248         {
249             string link = linkNode.GetAttributeValue("href", "not
250 found");
251
252             if (link.Length < 25)
253             {
254                 continue;
255             }
256
257             if (link.Substring(0,24) == "/displayFixture.html?id=")
258             {
259                 string fixtureId = link.Substring(24);
260
261                 if (fixtureList.Contains(fixtureId) == false)
262                 {
263
264 Console.Write($"{FormatString(linkNode.InnerText)} vs. ");
265                 fixtureList.Add(fixtureId);
266                 }
267                 else
268                 {
269
270 Console.WriteLine($"{FormatString(linkNode.InnerText)}, {fixtureId}");
271                 }
272             }
273         }
274     }
275 }
276 return fixtureList;
277 }
278 }
279 }

```

Output:

```
Enter the league ID: fh
Enter the league ID: 625214|
```

Figure 18

```
League Loaded - Blackwater and Dengie Youth Football League
138 referees found
Enter the referee's name: Leo Weller|
```

Figure 19

```
League Loaded - Blackwater and Dengie Youth Football League
Getting Fixture List for Leo Weller...

September 24
Maldon Saints Y U13 Greens vs. Writtle Minors Y U13 Yellows, 27218401
Maldon Saints Y U14 vs. Writtle Minors Y U14 Blues, 27207674
Galleywood Youth U12 Kestrels vs. Tiptree Jobserve Y U12 White, 26585413

October 24
Maldon & Tiptree Youth U13 Panthers vs. Tiptree Jobserve Y U13 White, 27214219

November 24
Maldon Saints Y U11 Lions vs. Uplands Rangers Y U11, 27464221
Maldon & Tiptree Youth U13 Panthers vs. Maldon Saints Y U13 Reds, 27214220
Maldon Saints Y U12 vs. Beacon Hill Rovers Y U12, 27186481
Galleywood Youth U12 Kestrels vs. Heybridge Swifts Youth U12 Falcons, 27186485

December 24
Tollesbury Juniors Y U12 vs. Great Leighs Youth U12 Panthers, 27578582
Tollesbury Juniors Y U11 vs. Witham Town Youth U11 Yellow, 27470111
Baddow Spartak U14 Pumas vs. Valley Green Y U14 Vipers, 27210047

January 25
Heybridge Swifts Youth U14 Panthers vs. Mayland Village Youth U14, 27210183
Heybridge Swifts Youth U11 Tigers vs. Torquespeed Y U11 Leopards, 27470877
Heybridge Swifts Youth U12 Tigers vs. Uplands Rangers Y U12, 27186341
Maldon Saints Y U12 vs. Bradwell Bay United Youth U12, 27636776
```

Figure 20

User Interview for Prototype:

I presented the prototype to Leo for his feedback on the current design.

Q25 – Is this the correct list of fixtures?

“Yes. These are my fixtures for the Blackwater league.”

Q26 – Do you find this easier to use than the Full-Time website?

“It is quicker than the Full-Time website. However, a lot of fixture information is still missing.”

Q27 – Do you like any features from this version that you would like carried through to the final version?

“I like that the fixtures are separated by month and that messages saying the league has been loaded are helpful.”

Q28 – Are there any bugs you have found while testing the program?

“When I tried to load my fixtures for the Border league, the program stated that the URL was invalid.”

Q29 – Okay, that is from the differing URLs between youth and open-age leagues. Having seen this fixture layout, do you want to change anything?

“I would like the team names to be more consistently spaced and coloured based on club colours.”

Q30 – Do you have any questions?

“What are the numbers at the end of the fixtures?”

They are the fixture IDs for finding the fixture pages – primarily for debugging.

Interview Summary:

When asked, Leo confirmed that the fixtures scraped from the Full-Time website were accurate and that the prototype ran faster than using the Full-Time website. However, he did mention that some crucial fixture information was missing and that there was a bug loading certain leagues due to a difference in URL formatting.

Leo particularly liked that the fixtures were separated by month and that status updates were provided throughout the process, so this will be incorporated into the final design.

Features to change:

- Add the fixture information (date and time, venue).
- Deal with the differing league URL formats.

Features to keep:

- Separating fixtures clearly by month (and perhaps also by league).
- Status updates are provided to the user throughout the process.

Critical Path:

To ensure that my project is completed on schedule, I have created a task list with estimated completion times and identified dependent tasks to make a critical path for the project.

Analysis & Planning Tasks:

Task	Predecessor	Task Description	Estimated Length?	Critical?	Done?
1		Create a project proposal outlining the problem that I wish to solve.	1 day	Yes	Yes
2	1	Find someone who shares the problem and select them to be your user.	3 days	Yes	Yes
3	2	Interview the user to find out what solution they want/need.	1 day	Yes	Yes
4	2	Create a model outlining how the current system works, explaining why that is a problem.	2 days	No	Yes
5	3	Create a model explaining how your solution will work and how it will solve the problem.	4 days	Yes	Yes
6	5	Split the project into sections and research how you will implement these sections most effectively.	10 days	Yes	Yes
7	3	Create the project's aims and objectives, ensuring they cover all the users' wants and needs.	2 days	Yes	Yes
8	3	Research previous attempts at solving the problem and analyse them for features to imitate and issues to avoid.	4 days	No	Yes
9	6, 7	Create a prototype solution attempting to solve part of the problem.	5 days	Yes	Yes
10	9	Present your prototype to the user and ask them for feedback on it.	1 day	Yes	Yes

Total Length: 33 days

Design Tasks:

Task	Predecessor	Task Description	Estimated Length?	Critical?	Done?
11	10	Write an overall design discussion explaining how the project will flow and link the major sections together.	3 days	Yes	Yes
12	11	Create a structure diagram illustrating how the project flows between sections.	1 day	Yes	Yes
13	11	Design the custom objects needed throughout the program with variables and subroutines.	5 days	Yes	Yes
14	11	Layout how essential functions will work with parameters, processes, and outputs.	4 days	Yes	Yes
15	11	Explain how complex algorithms such as the merge sort are going to work.	2 days	Yes	Yes
16	13, 14, 15	Create a class diagram demonstrating how the various objects, classes, and functions link together and their components.	3 days	Yes	Yes
17	5	Plan the layout of each screen and how the user will interact with the program.	3 days	No	Yes
18	11	Plan the database layout with tables, fields, and example records.	2 days	Yes	Yes
19	18	Design the queries that will be carried out on the database with fields, tables, and parameters to be entered.	3 days	Yes	Yes

Total Length: 26 days

Technical Solution Tasks:

Task	Predecessor	Task Description	Estimated Length?	Critical?	Done?
20	16, 18	Create the database according to the database layout.	1 day	Yes	Yes
21	15	Develop the utility functions to be used by the other programs.	2 days	Yes	Yes
22	15, 16, 19, 21	Develop the database manager according to the algorithms and class diagram.	2 days	Yes	Yes
23	16, 21	Develop the generic web scraper program.	2 days	Yes	Yes
24	16, 21	Create the custom objects needed for the web scraper and the app.	2 days	Yes	Yes
25	23, 24	Develop the Full-Time web scraper utilising the functions created in the generic web scraper.	3 days	Yes	Yes
26	16, 21	Develop the page navigation system and set up basic pages.	2 days	Yes	Yes
27	17, 22, 25	Implement the fixture list page using the Full-Time web scraper and database manager.	3 days	Yes	Yes
28	17, 22	Implement the home page using the database manager.	2 days	Yes	Yes
29	17, 22, 25	Implement the settings page using the database manager and Full-Time web scraper.	2 days	Yes	Yes
30	22	Develop the login system.	2 days	Yes	Yes
31	17, 30	Implement the login screen.	1 day	Yes	Yes

Total Length: 24 days

Testing Tasks:

Task	Predecessor	Task Description	Estimated Length?	Critical?	Done?
32	7	Create a test plan outlining how I intend to test each objective and how to format the results.	1 day	Yes	Yes
33	32	Develop tests to check that each aim and objective has been met using normal, boundary, and erroneous data.	4 days	Yes	Yes
34	33	Record a video where each test is carried out, and the result is compared to the expected one.	2 days	Yes	Yes
35	34	Edit the testing video.	5 days	Yes	Yes

Total Length: 11 days

Evaluation Tasks:

Task	Predecessor	Task Description	Estimated Length?	Critical?	Done?
36	10, 19, 31, 35	Compare the completed tests to the aims and objectives and ensure they are all suitably completed.	1 day	Yes	Yes
37	36	Discuss potential improvements for the solution with the client.	3 days	Yes	Yes
38	37	Create a list of potential improvements for the solution.	1 day	No	Yes

Total Length: 5 days

Design

Overall Design Discussion:

When the user loads the app, a database manager object with the file path for the Referex database will be initiated. The constructor function for this object sets the file path variable. It runs the 'TryConnectToDatabase' function to test the file path and establish a working connection, returning its success or failure as a boolean. The login page object will be initiated with the database connection and assigned to the current page variable. At the same time, the undo and redo button stacks are both set to be empty. In the constructor function for the login page, a login system object will be initiated, taking the database manager as a parameter, which will be used to interact with the user login details table in the database.

The login page will then run its display function and allow the user to create an account or log in to an existing one. If the user chooses to create a new account, they will be prompted to enter a username and password and confirm their password one at a time. To validate these details, the password and the confirmation will be compared, and if they are different, the details will be rejected, and the user will have to begin the process again. The password will then be validated to see if it's longer than eight characters to meet security requirements, and if not, then the user will have to begin again. The username and password will then have the SHA-256 algorithm applied to them and stored as hashed details for further processing. The database manager will then use a parameterised SQL SELECT query to check whether the username has already been taken (and is already in the user details table). If it is, the user will have to begin the process again. If all these validation tests pass, the user will be prompted to enter their name as it appears on Full-Time. The database manager will then run an SQL MAX query to get the highest existing user ID and increment it by one to give the user ID for the new account before running an SQL INSERT command to place the user ID, username, password, and account name into the user details table. Once this process is complete, the user will be presented with the home page, and the login system will return the user ID and account name to be used later in the program.

Conversely, if the user chooses to log in to an existing account, they will be prompted to enter their username and password. The details will then be passed on through the SHA-256 algorithm to hash them before a parameterised SQL SELECT query to retrieve the user ID and account name related to the entered username and password if they exist. If a user ID and name are found, they will be returned to the program, and the user will be presented with the home page. However, if the user ID and account name cannot be found, the user will be told that the details are invalid and must try again. This will increment a fails counter, which, upon reaching three, locks the user out for thirty seconds (with the lockout time increasing by thirty seconds for each subsequent failed attempt). Simultaneously, a will be recorded, and the database manager will use an SQL INSERT command to add the username, password, and into the notifications table as a failed login attempt to be reviewed.

Once the user has either logged in or created an account, the login page will be pushed onto the undo page stack, and the home page object will be initiated and assigned to the current page variable. The page will then run the display function. This will present the user with the app navigation bar with input options for the home page, fixture list page, settings page, undo, and redo, with the home page highlighted in red when the user enters one of the options. In that case, the new page will be initiated and assigned to the current page variable with the home page pushed onto the undo button stack, or the undo and redo button functions will run.

If the undo button is pressed, the system will attempt to pop a page off the undo button stack. If there's one to pop, it will take the current page and push it onto the redo button stack before assigning the popped page to the current page variable. Conversely, if the redo button is pressed, the reverse operation will occur, with a page being popped from the redo button stack and the current page being pushed onto the undo button stack. If either stack is empty, the button will set an 'active' boolean to false, blocking any attempts to press the button and colouring its text grey.

In the background, the database manager will run a cross-table parameterised SQL SELECT to collect the league names and Full-Time IDs from the league table based on the user ID combinations in the account-to-league combination table. Additionally, the referee ID for each combination will be returned if known, and all the results will be passed to the Full-Time web scraper object. If a league does not have a referee ID assigned, the web scraper will load the referee page for the league and search the dropdown of referees for the account name associated with the user ID. If the account name is found, the corresponding referee code will be taken and, using an SQL UPDATE command will be added to the combination entry. Once each league has an associated referee ID, the web scraper will load the referee's fixtures for each month across each league one by one, pulling the fixture IDs of any games found and adding them to the queue of fixtures for further processing. Once complete, each fixture in the queue is taken, and its Full-Time page is loaded. The home team, away team, division, kick-off time and date, venue information, and match officials are then scraped and used to initialise a fixture object.

The database manager will also run two SQL SELECT queries, one to find clubs and information such as team colours and the other to see the venue address from the name provided. If the venue name is not in the venue table, the program will run an additional SQL SELECT query to find the venue information using an alternate name. If either the club or the venue are not present in the database, the user will be prompted to enter the details themselves, which will be passed into an SQL INSERT command.

Initially, the user is presented with the app's home page. As an additional navigation option, the user can log out of the app. The page will display the length of the loaded leagues list alongside the number of fixtures processed and the number still in the queue. It will also display the of the last attempt to run the above operation and whether it succeeded. The user will also be able to see if they have any unread notifications, and if so, will be directed to the settings page to read them.

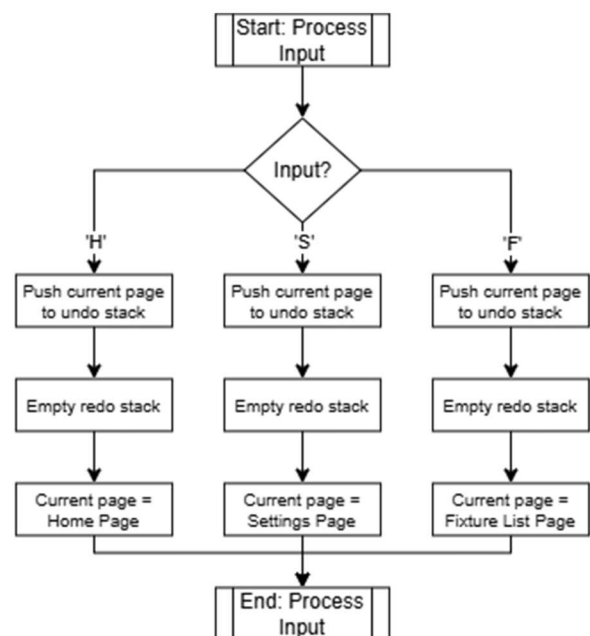
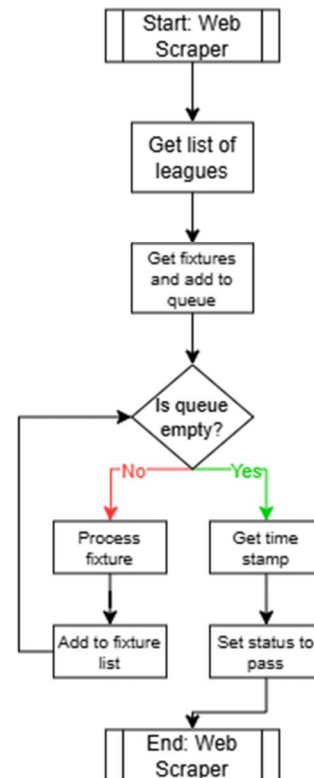
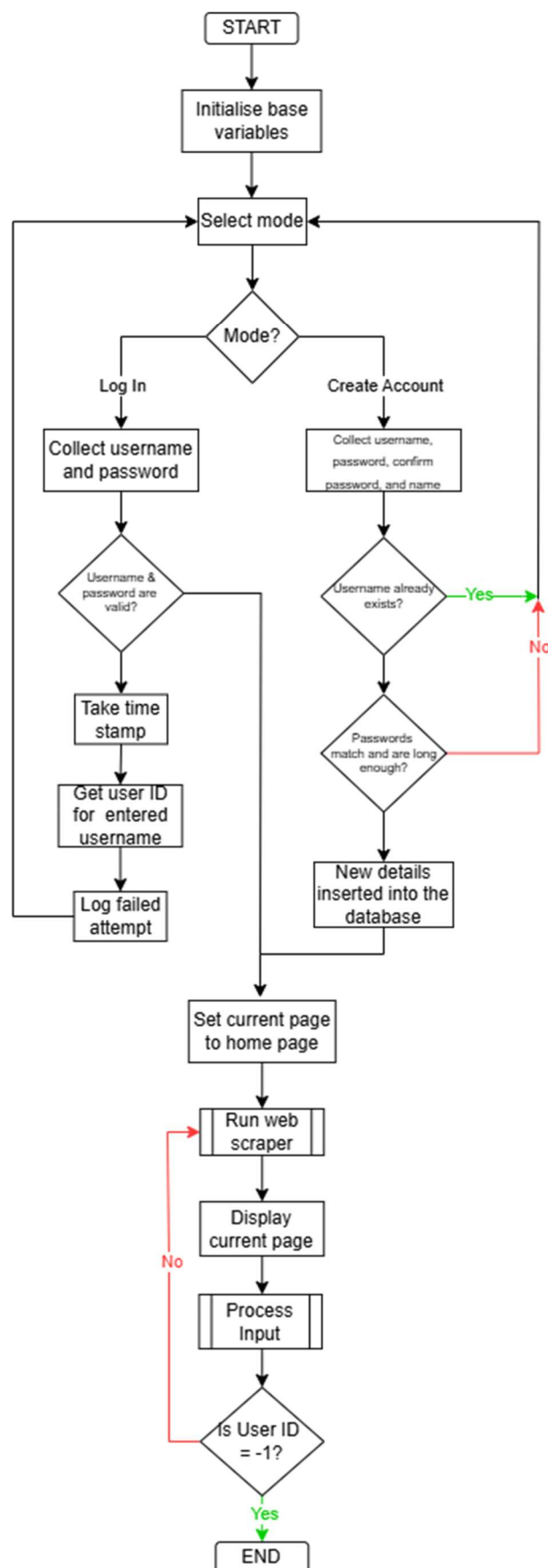
If the user chooses to navigate to the fixture list page, they will be asked what date range they want to view and how they want their fixtures sorted (date or competition). If the user chooses to enter a date range, a linear search will move through the fixture list and remove any fixtures outside of the specified range. Then, a merge sort will sort the fixtures by date or competition. The competition name and division, home and away team names, kick-off date and time, and venue address will be displayed for each fixture. If there are more fixtures than space on the page, there will be an option to scroll up and down pages to view different fixtures.

If the user chooses to load the settings page, they will be presented with the complete list of leagues they have decided to load and the option to edit it. They will also be allowed to change their password or delete their account. If the user chooses to edit the list of leagues, they can either add or remove leagues. If they decide to remove a league, an SQL DELETE command will remove the combination connecting the user and the league but leave the league information in case another user also has that league. If the user chooses to add a league, they will be prompted to enter a league name, and an SQL SELECT query will search the league's table for the relevant one and, if it

exists, return the league ID. If not, the user will be prompted to go onto Full-Time and get the Full-Time ID, and an SQL INSERT command will add it to the league's table with the league ID assigned using an SQL MAX query on the league ID field.

If the user chooses to change their password or to delete their account, they will be prompted to enter their password. An SQL COUNT query with the user ID and hashed password will check that it's valid before proceeding; otherwise, the user will be returned to the settings page. If the user wants to change their password, they will be prompted to enter and confirm a password. It will then be validated as when the user created their account. Once validated, an SQL UPDATE command will update the old password to the new one. If the user chooses to delete their account, their user details record will be deleted using an SQL DELETE command alongside their league combinations. The user will then be returned to the login screen.

Structure Diagram:



Class Diagram:

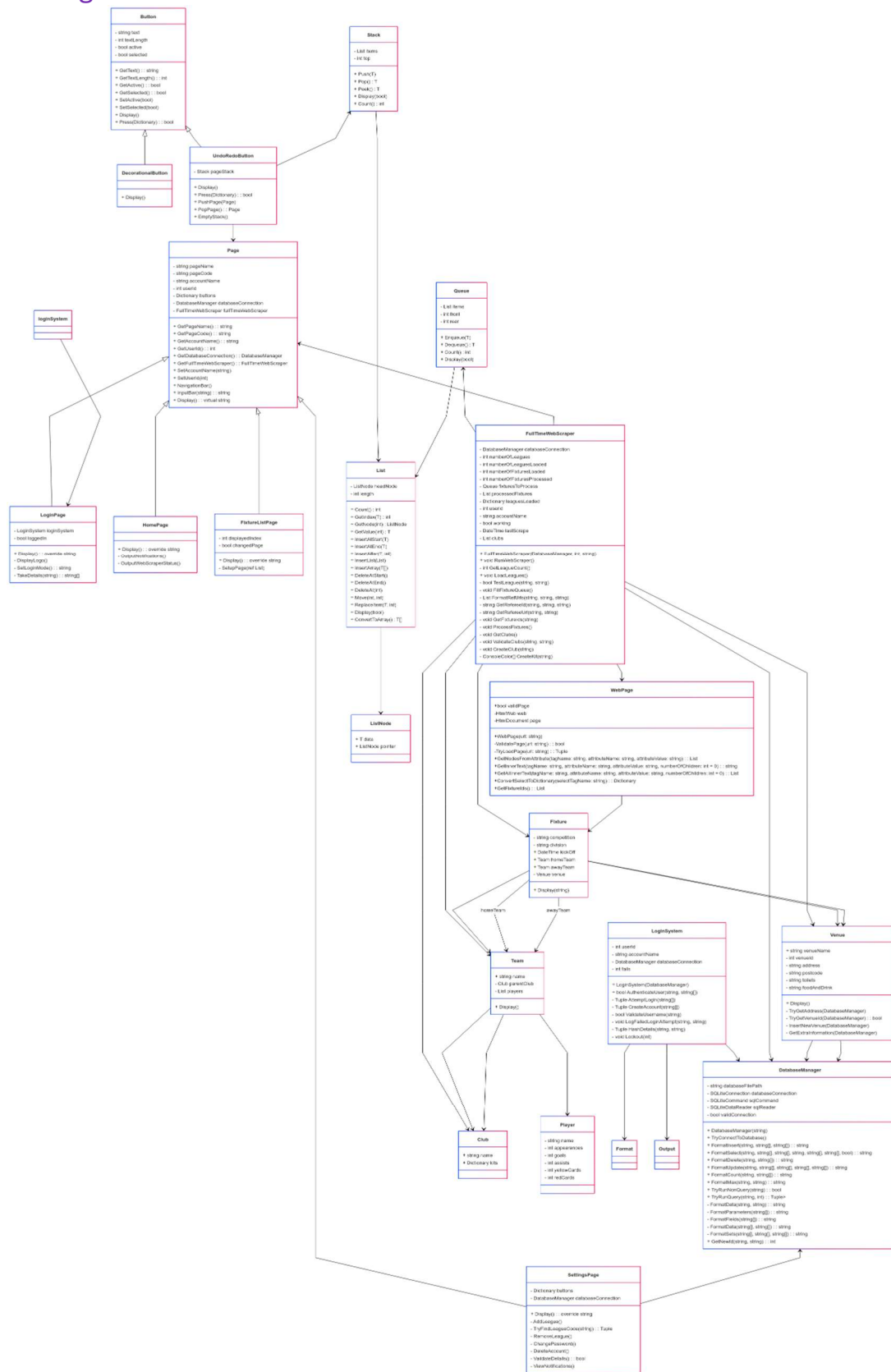


Figure 21

<https://www.mermaidchart.com/raw/d4e2de8e-df82-4f9d-885f-878de8976e81?theme=light&version=v0.1&format=svg>

Human-Computer Interaction:

User Interaction:

Page Inputs:

The app is entirely text-based, so I cannot take inputs through text boxes and button presses. As a result, I decided to implement an input bar at the bottom of every page, allowing users to provide input and navigate between pages.

Additionally, I must make the input options clear to the user. To do this, I will allocate a key for each “button” so the user can “press” it. This will enable the user to enter the full-page name or the assigned key into the input bar to select the input option.

Page Button:

This is an example of what a button looks like. The left is the button when not selected, and the right is the selected button, which is highlighted to clarify that it is active.

| [F] Fixture List | | [F] Fixture List |

Input Bar:

This is an example of the input bar. The text ‘Input’ changes based on the user’s input.

Input: ...

Page Navigation:

I will make a generic class each page inherits to create page navigation on the app. This will allow me to create default page behaviours while assigning specific functions for behaviours required for individual pages.

Changing Page:

The current page variable is pushed onto the undo button stack when a valid page name or initial is entered. The page relating to the input is assigned to the current page variable.

Undo Button:

When the undo button is ‘pressed’, the current page variable is pushed to the redo button stack, and the top value from the undo button stack is popped to the current page variable. If the undo button stack is empty, nothing happens – the button should be greyed out when this occurs.

Redo Button:

When the redo button is ‘pressed’, the current page variable is pushed to the undo button stack, and the top value from the redo button stack is popped to the current page variable. If the redo button stack is empty, nothing happens – the button should be greyed out when this occurs.

Page Layouts:

Each page on the app will be entirely text-based, allowing me to spend more time designing the features while limiting screen distractions so that I can focus on the information.

- Don't need to have multiple versions of the same screen.
- Label where inputs go to function-wise.

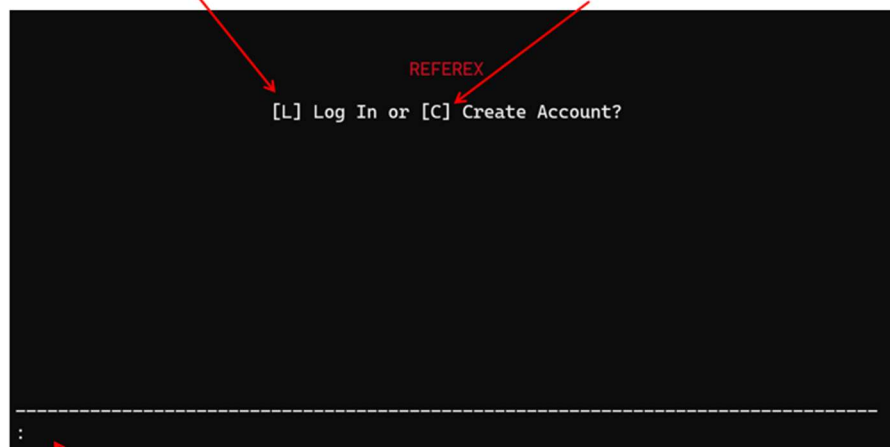
Login Page:

Start:

SCREEN 1

If 'l' is entered, the login page will move to screen 2.

If 'c' is entered, the login page will move to screen 3.

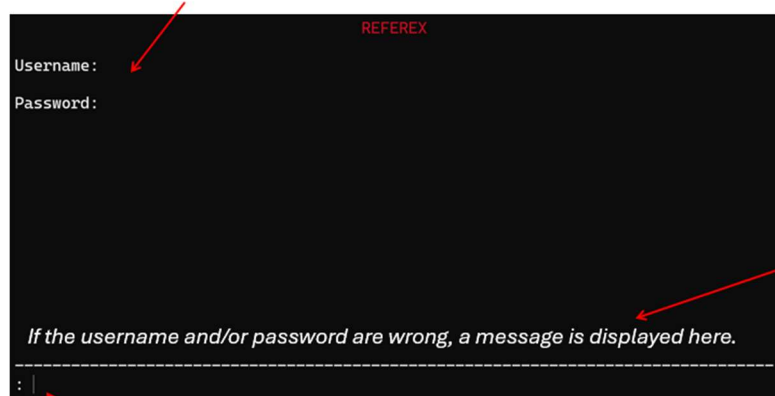


Input Bar (Page Function) : Takes the user input and returns it as a string

Logging In:

SCREEN 2

After the username is entered, it will appear here.



Input Bar (Page Function) : Takes the user input and returns it as a string

If the details are correct, the user will be passed onto the home page.

This will also log the failed attempt in the notification table.

Creating an Account:

SCREEN 3

After the username, password, and confirmation are entered, they will appear here.

Passed into a SELECT SQL to check for an existing username

```
REFEREX
Username:
Password:
Confirm Password:
Enter Name:

If the username is already taken, or the passwords don't match, a message
will appear here.

:
```

Input Bar (Page Function) : Takes the user input and returns it as a string

If the details are valid, the details will be inserted into the database, and the user will pass to the home page.

Home Page:

The currently selected page is always in red and will not do anything if selected.

If either is selected, the current page is set to the new page type and the current page is pushed to the undo stack.

If either of these is selected, the page is loaded off the relevant stack, and the current page is moved onto the other stack.

These are taken from the web scraper so that the user can monitor its status.

This allows the user to check that all the functions of the web scraper are working.

Changes yellow and shows the notification count if the user has any notifications

It takes input from the user, takes the first letter, and sets it in lower case for processing.

If 'l' is entered, the user ID is set to - 1 and the login system is set to logged out

```
[H] Home [F] Fixture List [S] Settings REFEREX [U] Undo [R] Redo
-----
4 / 4 Leagues Successfully Loaded (100 %)
0 / 0 Fixtures Successfully Processed (0 %)
Web Scraper: Success - 06/03/2025 09:01
You Have No Notifications
[L] Log Out
:
```

Settings Page:

If either is selected, the current page is set to the new page type and the current page is pushed to the undo stack.

The currently selected page is always in red and will not do anything if selected.

If either of these is selected, the page is loaded off the relevant stack, and the current page is moved onto the other stack.

Displays the list of leagues that the user has loaded.

Allows the user to edit the list of leagues they have loaded.

Allows the user to edit the details of their account.

It lets the user see all the notifications for their account.

It takes input from the user, takes the first letter, and sets it in lower case for processing.

```
[H] Home [F] Fixture List [S] Settings REFEREX [U] Undo [R] Redo
-----
Leagues Loaded:
- Blackwater and Dengie Youth Football League
- Essex & Suffolk Border League
- Essex County Women's Football League
- Essex Veterans League

- [A] Add League
- [R] Remove League
- [C] Change Password
- [D] Delete Account
- [V] View Notifications
-----
:
```

Edit League List:

If the 'u' is entered at any point, the operation will be aborted.

The user can enter the league's details here.

```
[U] Undo
Enter the name of the league to add: Test League

Enter the league ID as it appears in the URL on Full-Time: abc123

Enter the season ID as it appears in the URL on Full-Time: def456

Enter the league format: Men's Open-Age
```

Change Password:

The user is asked to log back in before making any changes to their account.

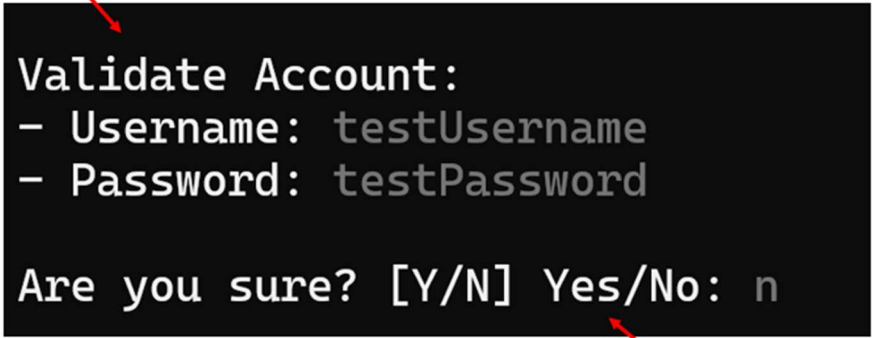
```
Validate Account:
- Username: testUsername
- Password: testPassword

New Password: testPassword
Confirm Password: testPassword
```

The user can enter their new password here.

Delete Account:

The user is asked to log back in before making any changes to their account.

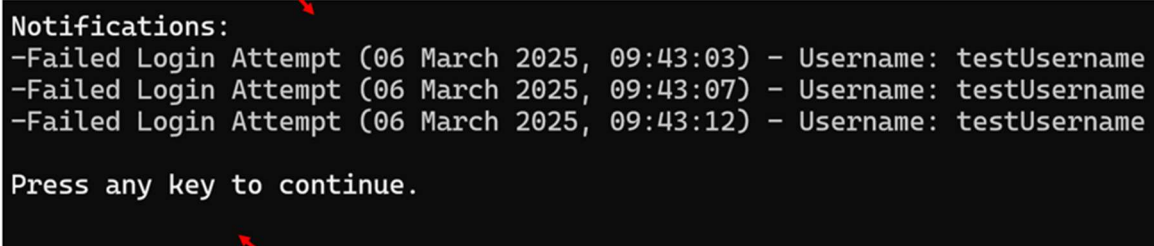


```
Validate Account:  
- Username: testUsername  
- Password: testPassword  
  
Are you sure? [Y/N] Yes/No: n
```

The user must enter a 'y' to delete their account – anything else will abort.

View Notifications:

Displays each of the user's notifications individually.



```
Notifications:  
-Failed Login Attempt (06 March 2025, 09:43:03) - Username: testUsername  
-Failed Login Attempt (06 March 2025, 09:43:07) - Username: testUsername  
-Failed Login Attempt (06 March 2025, 09:43:12) - Username: testUsername  
  
Press any key to continue.
```

Waits until any key is pressed then clears the notifications and returns to the settings page.

Fixture List Page:

If one of these options is selected, load that page, push the current page to the undo stack, and clear the redo button stack.

If the user enters 'u' or 'r', the buttons will pop a page from their stack and the new page will be loaded.

Each of the fixtures in the fixture list is displayed here.

Each of the teams is coloured based on their kits.

If the user enters a '+' or '-' then more fixtures will be loaded.

The input bar takes the user's input and returns the first character.

```
[H] Home [F] Fixture List [S] Settings REFEREX [U] Undo [R] Redo
-----
Essex County Women's Football League Division Three
30/03/2025 - 14:00
Woodham Radars Ladies FC vs. Runwell Sports Ladies FC
SOUTH WOODHAM PLAYING FIELD - CM3 5PL

Essex County Women's Football League Division Three
23/03/2025 - 14:00
Hatfield Peverel Women FC vs. Braintree Athletic Women FC
KEITH BIGDEN MEMORIAL GROUND - CM3 2JL

Essex County Women's Football League Division Four
16/03/2025 - 14:00
South Woodham Ferrers United Women FC vs. Harwich & Parkeston Women FC
WILLIAM DE FERRERS SCHOOL - CM3 5JU
-----
[-] Previous, [+] Next: |
```

Data Structures, Algorithms & Object-Oriented Programming:

Algorithms:

Merge Sort:

I have chosen to use a merge sort in my project when sorting fixtures by date or competition.

The algorithm works by:

1. Calculating a midpoint and using it to split the array in half.
2. Feeding that back into the algorithm to be split further until the array.
3. Repeating until the array is split into individual elements.
4. These parts are passed into the merge function to combine two arrays.
5. This process is repeated with the results until the array is fully recombined and sorted.

```
1 FUNCTION MergeSort(array)
2   IF LENGTH (array) <= 1
3     RETURN array
4   ENDIF
5
6   midpoint ← LENGTH (array) / 2
7
8   leftArray ← array(0, midpoint)
9   rightArray ← array(midpoint, LENGTH (array))
10
11  leftArray ← Merge(leftArray)
12  rightArray ← Merge(rightArray)
13
14  RETURN Merge(leftArray, rightArray)
15 ENDFUNCTION
16
17 FUNCTION Merge(leftArray, rightArray)
18   leftLength ← LENGTH (leftArray)
19   rightLength ← LENGTH (rightArray)
20
21   sortedArray ← NEW ARRAY (leftLength + rightLength)
22   leftIndex ← 0, rightIndex ← 0, mergeIndex ← 0;
23
24   WHILE leftIndex < leftLength AND rightIndex < rightLength
25     IF leftArray[leftIndex] > rightArray[rightIndex]
26       sortedArray[mergeIndex++] ← leftArray[leftIndex++]
27     ELSE
28       sortedArray[mergeIndex++] ← rightArray[rightIndex++]
29     ENDIF
30   ENDWHILE
31
32   WHILE leftIndex < leftLength
33     sortedArray [mergeIndex++] ← leftArray[leftIndex++]
34   ENDWHILE
35   WHILE rightIndex < rightLength
36     sortedArray [mergeIndex++] ← rightArray[rightIndex++]
37   ENDWHILE
38   RETURN sortedArray
39 ENDFUNCTION
```

Object-Oriented Programming:

Below, I have outlined the custom objects and classes I intend to use within my project, their applications, and the required functions and variables. I have highlighted any custom data type I have used by colouring it blue. I have also detailed how any complex functions work under the class they are a part of.

Back-End Systems:

Login System:

Variables:

Variable Name	Data Type	Private?	Initial Data
User ID	Integer	No	-1
Account Name	String	No	""
Database Manager	Database Manager	Yes	Taken as a parameter
Fails	Integer	Yes	0

Constructor:

Initialises the login system object. It takes the database connection as a parameter to allow for SQL interaction.

```
FUNCTION LoginSystem (DatabaseManager)
    DatabaseManager ← DatabaseManager
    userId ← -1
    accountName ← ""
    fails ← 0
ENDFUNCTION
```

Hashing:

I call the format utility class to run the Format.ToHash() function to hash the entered username and password before they are compared with the database.

```
FUNCTION ToHash (inputString)
    outputString ← ""

    USING SHA256 hash
        hashBytes ← COMPUTE(hash, inputString)
        FOREACH byte in hashBytes
            outputString += CONVERT(byte, STRING)
        ENDFOR
    ENDUSING

    RETURN outputString
ENDFUNCTION
```

SQL Queries / Commands:

This object uses the database manager to run SQL for the following tasks:

Attempting to get a user ID and account name for an entered username and password.

```
SELECT userId, accountName
FROM tblUserAccount
WHERE username = username, password = password;
```

Getting a new user ID when the user wants to create an account.

```
SELECT COUNT (*)
FROM tblUserAccount
```

Inserting a new account into the user details table.

```
INSERT INTO tblUserAccount  
VALUES (userId, username, password, accountName);
```

Checking to see if a username already exists in the user details table.

```
SELECT username  
FROM tblUserAccount  
WHERE username = username;
```

Getting the user ID from the user details table based on the username entered when the password is wrong.

```
SELECT userId  
FROM tblUserAccount  
WHERE username = username;
```

Getting a new notification ID when trying to log a new notification.

```
SELECT COUNT (*)  
FROM tblNotification;
```

Inserting a new notification into the notification table.

```
INSERT INTO tblNotification  
VALUES (notificationId, userId, message);
```

Database Manager:

Variables:

Variable Name	Data Type	Private?	Initial Data
Database File Path	String	Yes	Taken as a parameter
Database Connection	SQLiteConnection	Yes	Null
SQL Command	SQLiteCommand	Yes	Null
SQL Reader	SQLiteReader	Yes	Null
Valid Connection	Boolean	Yes	False

Constructor:

Initialises the database manager object. It takes the file path for the database as a parameter and then tests that it can connect.

```
FUNCTION DatabaseManager (filePath)  
    databaseFilePath ← filePath  
    TryConnectToDatabase()  
ENDFUNCTION
```

Try to Connect to the Database:

Attempts to load the connection between the code and the database file.

```
FUNCTION TryConnectToDatabase ()
    TRY
        databaseConnection ← CONNECT (databaseFilePath)
        sqlCommand ← NEW databaseConnection.command
        validConnection ← TRUE
    CATCH
        validConnection ← FALSE
    ENDTRYCATCH
ENDFUNCTION
```

Insert:

Attempts to format an INSERT SQL command. It adds the table name and formats each value based on the data type associated with it.

```
FUNCTION Insert (tableName, fields, dataTypes)
    IF data.Length != dataTypes.Length
        Output.Error("Invalid Parameters Entered.")
        RETURN ""
    ENDIF
    insertSql ← "INSERT INTO " + tableName + " VALUES ("
    insertSql + FormatData(data, dataTypes) + ");"
ENDFUNCTION
```

Delete:

Attempts to format a DELETE SQL command. It adds the table name and any parameters.

```
FUNCTION Delete (tableName, parameters)
    deleteSql ← "DELETE FROM " + tableName
    deleteSql += FormatParameters(parameters)
    RETURN deleteSql
ENDFUNCTION
```

Count:

Attempts to format a COUNT SQL query. It adds the table name and optionally adds any parameters specified by the user.

```
FUNCTION Count (tableName, parameters)
    countText ← "COUNT (*) FROM " + tableName

    IF parameters != NULL
        countSql += FormatParameters(parameters)
    ENDIF

    RETURN countSql
ENDFUNCTION
```

Select:

Attempts to format a SELECT SQL query. It adds the table name, the fields to be selected, any parameters specified by the user, and optionally adds code to perform a join operation with another table based on a set of fields.

```
FUNCTION Select (tableName, fields, parameters, joinTable, tableFields,
joinTableFields, left)
    selectSql ← "SELECT "
    selectSql += FormatFields(fields)
    selectSql += " FROM " + tableName

    IF joinTable != NULL
        IF tableFields.Length != joinTableFields.Length
            Output.Error("Invalid Parameters Entered.")
            RETURN ""
        ENDIF

        IF left
            selectSql += " LEFT JOIN " + joinTable + " ON "
        ELSE
            selectSql += " INNER JOIN " + joinTable + " ON "
        ENDIF

        selectSql += FormatSets(tableFields, joinTableFields, new
string[tableFields.Length])
    ENDIF

    IF parameters != NULL
        selectSql += FormatParameters(parameters);
    ENDIF

    RETURN selectSql + ";"
ENDFUNCTION
```

Update:

Attempts to format an UPDATE SQL command. It adds the table name, the fields to be updated and the data to replace the values in the fields, formatted using the data types specified. The user can also optionally add parameters to the command.

```
FUNCTION Update (tableName, fields, data, dataTypes, parameters)
    IF LENGTH(dataTypes) != LENGTH(data) != LENGTH(fields)
        RETURN ""
    ENDIF

    updateSql ← "UPDATE " + tableName + " SET "
    updateSql += FormatSets(fields, data, dataTypes)

    IF parameters != NULL
        updateSql += FormatParameters(parameters)
    ENDIF

    RETURN updateSql + ";"
ENDFUNCTION
```

Max:

Attempts to format a MAX SQL query with a table name and a field to get the max value of.

```
FUNCTION Max (tableName, fieldName)
    RETURN "SELECT MAX (" + fieldName + ") FROM " + tableName
ENDFUNCTION
```

Try Run Non-Query:

This takes the SQL for a non-query and attempting to run it on the database. It returns whether the operation succeeded as a boolean and on fails outputs the SQL statement and an exception message.

```
FUNCTION TryRunNonQuery(nonQuerySql)
    TryConnectToDatabase();
    IF validConnection == FALSE
        RETURN FALSE
    ENDIF

    TRY
        sqlCommand.CommandText ← nonQuerySql
        sqlCommand.ExecuteNonQuery()
        databaseConnection.Close();
        RETURN TRUE
    CATCH exception
        Output.Error($"Cannot execute Non-Query SQL. \n
        {nonQuerySql}", exception.Message)

        databaseConnection.Close()
        RETURN FALSE
    ENDTRYCATCH
ENDFUNCTION
```

Try Run Query:

This takes the SQL and attempts to run it through a reader. It then loops through each record the reader collects and all the values from each and adds them to an array which are then inserted into a list which is returned.

```
FUNCTION TryRunQuery(selectSql, fields)
    records ← NEW List<string>

    TryConnectToDatabase()
    IF validConnection == FALSE
        RETURN
    ENDIF

    TRY
        sqlCommand.CommandText ← selectSql
        sqlReader ← sqlCommand.ExecuteReader()

        WHILE sqlReader.Read()
            record ← NEW string[fields]

            FOR i ← 0, i < fields, i++
                record[i] ← sqlReader.GetValue(i)
            ENDFOR
            records.InsertAtEnd(record)
        ENDWHILE
    ENDTRY
```

```

        databaseConnection.Close();
        RETURN records
    CATCH exception
        Output.Error($"Cannot execute SELECT SQL. \n{selectSql}",
            exception.Message);

        databaseConnection.Close();
        RETURN
    ENDTRYCATCH
ENDFUNCTION

```

Pages:

I have created a base page class from which each page and sub-page can inherit. This will allow for more effortless page loading and stack operations. Each of the base variables is encapsulated and so is specified as a private variable and has getter/setter functions to allow for interaction.

Base Page:

Variables:

Variable Name	Data Type	Private?	Initial Data
Page Name	String	Yes	Taken as a parameter
Page Code	String	Yes	First character of page name
Account Name	String	Yes	Taken as a parameter
User ID	Integer	Yes	Taken as a parameter
Buttons	Dictionary<string, Button >	Yes	Taken as a parameter
Database Connection	DatabaseManager	Yes	Taken as a parameter
Full-Time Web Scraper	FullTimeWebScraper	Yes	Taken as a parameter

Constructor:

This initialises the object.

```

FUNCTION Page (accountName, userId, buttons, databaseConnection,
fullTimeWebScraper, pageName)
    pageName ← pageName
    pageCode ← pageName[0]
    accountName ← accountName
    userId ← userId
    buttons ← buttons
    databaseConnection ← databaseConnection
    fullTimeWebScraper ← fullTimeWebScraper
ENDFUNCTION

```

Login Page (Inherited from Page):

Variable Name	Data Type	Private?	Initial Data
Login System	LoginSystem	Yes	Initialised in constructor
Logged in	Boolean	Yes	False

This initialises the object and calls the base constructor.

```
FUNCTION LoginPage (accountName, userId, buttons, databaseConnection,
fullTimeWebScraper)
    BASE(accountName, userId, buttons, databaseConnection,
fullTimeWebScraper, "Login Page")
    loggedIn ← FALSE
    loginSystem ← NEW LoginSystem(databaseConnection)
ENDFUNCTION
```

Home Page (Inherited from Page):

Variable Name	Data Type	Private?	Initial Data
---------------	-----------	----------	--------------

This class has no unique variables

```
FUNCTION HomePage (accountName, userId, buttons, databaseConnection,
fullTimeWebScraper)
    BASE(accountName, userId, buttons, databaseConnection,
fullTimeWebScraper, "Home Page")
ENDFUNCTION
```

SQL Commands / Queries:

The page calls a COUNT SQL to get the number of notifications for the user.

```
SELECT COUNT (*)
FROM tblNotification
WHERE userId = userId;
```

Fixture List Page:

Variable Name	Data Type	Private?	Initial Data
Displayed Index	Integer	Yes	0
Changed Page	Boolean	Yes	True

This initialises the fixture list page object and calls the base page constructor.

```
FUNCTION FixtureListPage (accountName, userId, buttons,
databaseConnection, fullTimeWebScraper)
    BASE(accountName, userId, buttons, databaseConnection,
fullTimeWebScraper, "Fixture List Page")
    displayedIndex ← 0
    changedPage ← TRUE
ENDFUNCTION
```

This page calls the merge sort function to sort the fixture list by date when preparing to display the fixture list. This can be seen above in the algorithms section.

Settings Page:

Variable Name	Data Type	Private?	Initial Data
---------------	-----------	----------	--------------

This object has no unique variables.

This initialises the settings page object and calls the base page constructor.

```
FUNCTION SettingsPage (DatabaseManager)
    BASE(accountName, userId, buttons, databaseConnection,
        fullTimeWebscraper, "Settings Page")
ENDFUNCTION
```

SQL Commands / Queries:

This page utilises the base database connection to call SQL commands for the following tasks:

Getting the league ID from the league table based on the Full-Time league code.

```
SELECT leagueId
FROM tblLeague
WHERE fullTimeLeagueId = leagueCode;
```

Getting a new combination Id when trying to add a new combination to the combinations table.

```
SELECT COUNT (*)
FROM tblAccountLeagueCombination
```

Inserting a new combination into the combinations table.

```
INSERT INTO tblAccountLeagueCombination
VALUES (combinationId, userId, leagueId, "")
```

Trying to get the Full-Time league ID of a named league from the leagues table.

```
SELECT fullTimeLeagueId
FROM tblLeague
WHERE leagueName = leagueName;
```

Getting a new league ID from the leagues table when trying to insert a new league.

```
SELECT COUNT (*)
FROM tblLeague;
```

Inserting the new league into the leagues table.

```
INSERT INTO tblLeague
VALUES (leagueId, leagueName, fullTimeLeagueId, seasonId, leagueFormat);
```

Selecting the league ID of a league given its name.

```
SELECT leagueId
FROM tblLeague
WHERE leagueName = leagueName;
```

Deleting a combination from the combinations table based on a user and league ID.

```
DELETE FROM tblAccountLeagueCombination
WHERE userID = userID AND leagueId = leagueId;
```

Changing the password on an account based on the user ID.

```
UPDATE tblUserAccount
SET password = password
WHERE userID = userId;
```

Deleting a user account, their notifications and their combinations based on their user ID.

```
DELETE FROM tblUserAccount
WHERE userId = userId;
```

```
DELETE FROM tblNotification
WHERE userId = userId;
```

```
DELETE FROM tblAccountLeagueCombination
WHERE userId = userId;
```

Getting the notifications for a given user ID.

```
SELECT message
FROM tblNotification
WHERE userId = userId;
```

Deleting all the read notifications based on the user ID.

```
DELETE FROM tblNotification
WHERE userID = userId;
```

Web Scraper:

Variable Name	Data Type	Private?	Initial Data
Valid Page	Boolean	No	Set by a function
Web	HtmlWeb	Yes	Object initialised by constructor
Page	HtmlDocument	Yes	Set by a function

```
FUNCTION WebScraper (url)
    web ← NEW HtmlWeb()
    (validPage, page) ← TryLoadPage(url)
ENDFUNCTION
```

Full-Time Web Scraper:

Variable Name	Data Type	Private?	Initial Data
Database Connection	DatabaseManager	Yes	Taken as a parameter
Number of Leagues	Integer	No	0
Number of Leagues Loaded	Integer	No	0
Number of Fixtures Loaded	Integer	No	0
Number of Fixtures Processed	Integer	No	0
Fixtures to Process	Queue<string>	Yes	Initialised by the constructor
Processed Fixtures	List<Fixture>	No	Initialised by the constructor
Leagues Loaded	Dictionary<string, string[]>	No	Initialised by the constructor
User ID	Integer	No	Taken as a parameter
Account Name	String	No	Taken as a parameter
Working	Boolean	No	False
Last Scrape	DateTime	No	0:00
Clubs	List<Club>	Yes	Initialised by the constructor

This initialises the object.

```
FUNCTION FullTimeWebScraper (databaseConnection, userId, accountName)
    databaseConnection ← databaseConnection

    fixturesToProcess ← NEW Queue<string>()
    processedFixtures ← NEW List<Fixture>()
    leaguesLoaded ← NEW Dictionary<string, string[]>()

    userId ← userId
    accountName ← accountName

    numberOfLeagues ← 0
    numberOfLeaguesLoaded ← 0
    numberOfFixturesLoaded ← 0
    numberOfFixturesProcessed ← 0

    working ← FALSE
    lastScrape ← NEW DateTime(0)

    clubs ← NEW List<Club>()
    GetClubs()
ENDFUNCTION
```

SQL Commands / Queries:

Getting a count of the number of leagues loaded for a chosen user.

```
SELECT COUNT (*)
FROM tblAccountLeagueCombination
WHERE userId = userId;
```

Getting a club ID when creating a new club and adding it to the table.

```
SELECT COUNT (*)
FROM tblClub;
```

Running a cross table select statement getting the league name, combination ID, Full-Time league ID, season ID, referee ID and league format from the leagues table and combinations table joined together. This is based on the league IDs being the same and the user ID being the given one.

```
SELECT tblLeague.leagueName, tblAccountLeagueCombination.combinationId,
      tblLeague.fullTimeLeagueId, tblLeague.fullTimeSeasonId,
      tblAccountLeagueCombination.refereeId, tblLeague.leagueFormat
FROM tblLeague
INNER JOIN tblAccountLeagueCombination
ON tblLeague.leagueId = tblAccountLeagueCombination.leagueId
WHERE tblAccountLeagueCombination.userId = userId;
```

Updating a given combination with a referee ID.

```
UPDATE tblAccountLeagueCombination
SET refereeID = refereeID
WHERE combinationID = combinationID;
```

Getting all the clubs and their kits from the clubs' table.

```
SELECT clubName, homeKit, awayKit, thirdKit
FROM tblClub;
```

Inserting the new club into the clubs' table.

```
INSERT INTO tblClub
VALUES (clubId, clubName, homeKit, awayKit, thirdKit);
```

Custom Objects:

Button:

This is a base class that is used by most buttons. There are some child button classes from this with slightly different behaviours. All the variables below are encapsulated and so are private and have getters and setters for external interaction.

Variable Name	Data Type	Private?	Initial Data
Text	String	Yes	Taken as a parameter
Text Length	String	Yes	The length of the above text
Active	Boolean	Yes	True
Selected	Boolean	Yes	False

```
FUNCTION Button (Text)
  text ← Text
  TextLength ← LENGTH ButtonText
  Active ← TRUE
  Selected ← FALSE
ENDFUNCTION
```

Decorational Button (Inherited from Button):

This is the button from above but with the ability to be pressed removed.

Variable Name	Data Type	Private?	Initial Data
---------------	-----------	----------	--------------

This object does not have any unique variables

```
FUNCTION DecorationalButton (Text)
    BASE(Text)
ENDFUNCTION
```

Undo / Redo Button (Inherited from Button):

This is the button from above but with a stack of pages that it monitors itself.

Variable Name	Data Type	Private?	Initial Data
Page Stack	Stack<Page>	Yes	Initialised by the constructor

```
FUNCTION UndoRedoButton (Text)
    BASE(Text)
    pageStack ← NEW Stack<Page>()
ENDFUNCTION
```

```
FUNCTION PushPage (oldPage)
    pageStack.Push(oldPage)
    BASE.SetActive(true)
ENDFUNCTION
```

```
FUNCTION PopPage ()
    IF LENGTH(pageStack) == 1
        BASE.SetActive(false)
    ENDIF
    RETURN pageStack.Pop()
ENDFUNCTION
```

Fixture:

Variable Name	Data Type	Private?	Initial Data
Competition	String	No	Taken as a parameter
Division	String	No	Taken as a parameter
Kick-Off Date & Time	DateTime	No	Taken as a parameter
Home Team	Team	No	Taken as a parameter
Away Team	Team	No	Taken as a parameter
Venue	Venue	No	Taken as a parameter

```
FUNCTION Fixture (Competition, Division, KickOff, HomeTeam, AwayTeam, Venue)
    Competition ← Competition
    Division ← Division
    KickOff ← KickOff
    HomeTeam ← HomeTeam
    AwayTeam ← AwayTeam
    Venue ← Venue
ENDFUNCTION
```

Clubs:

Variable Name	Data Type	Private?	Initial Data
Club Name	String	No	Taken as a parameter
Kits	Dictionary<String, ConsoleColor[]>	No	Taken as a parameter

Initialises the object.

```
FUNCTION Club (clubName, kits)
    clubName ← clubName
    kits ← kits
ENDFUNCTION
```

Player:

Variable Name	Data Type	Private?	Initial Data
Player Name	String	No	Taken as a parameter
Appearances	Integer	No	Taken as a parameter
Goals	Integer	No	Taken as a parameter
Assists	Integer	No	Taken as a parameter
Yellow Cards	Integer	No	Taken as a parameter
Red Cards	Integer	No	Taken as a parameter

Initialises the player object.

```
FUNCTION Player (playerName, appearances, goals, assists, yellows, reds)
    playerName ← playerName
    appearances ← appearances
    goals ← goals
    assists ← assists
    yellowCards ← yellows
    redCards ← reds
ENDFUNCTION
```

Team:

Variable Name	Data Type	Private?	Initial Data
Name	String	No	Taken as a parameter
Club	Club	No	Taken from a parameter
Players	List<Player>	No	Null

This initialises the object.

```
FUNCTION Team (name, clubs)
    name ← name

    FOR i ← 0, i < LENGTH(clubs), i++
        IF name.contains(clubs[i].name)
            club ← clubs[i]
            RETURN
        ENDIF
    ENDFOR
ENDFUNCTION
```

Venue:

Variable Name	Data Type	Private?	Initial Data
Venue Name	String	No	Taken as a parameter
Venue ID	Integer	Yes	Set by a function
Address	String	Yes	Taken as a parameter
Postcode	String	Yes	Set by the constructor
Toilets	String	Yes	Set by a function
FoodAndDrink	String	Yes	Set by a function

Initialises the venue object.

```
FUNCTION Venue (venueName, address, databaseConnection)
    venueName ← venueName
    address ← address

    IF LENGTH(address) >= 7
        postcode ← SUBSTRING(address, LENGTH(address) - 7)
        address ← REMOVE(address, LENGTH(address) - 7)
    ELSE
        TryGetAddress(databaseConnection)
    ENDIF

    foundId ← TryGetVenueId(databaseConnection)
    IF foundId == FALSE
        InsertNewVenue(databaseConnection)
    ENDIF

    GetExtraInformation(databaseConnection)
ENDFUNCTION
```

SQL Commands / Queries:

Tries to select the venue address and postcode from the venue table whilst performing a left join with the alternate venue name table to check all options simultaneously. It is joined based on the venue IDs being the same and filtered by the entered name being in the venue name or the alternate name.

```
SELECT tblVenue.address, tblVenue.postcode
FROM tblVenue
LEFT JOIN tblVenueAlternateName
ON tblVenue.venueId = tblVenueAlternateName.venueId
WHERE tblVenue.venueName = venueName OR tblVenueAlternateName =
venueName;
```

Selecting the address based on an entered postcode from the venue table.

```
SELECT address
FROM tblVenue
WHERE postcode = postcode;
```

Selecting the venue ID based on the venue name.

```
SELECT venueId
FROM tblVenue
WHERE venueName = venueName;
```

Selecting the venue name and ID based on an alternative name whilst joining the tables based on the venue ID being the same.

```
SELECT tblVenue.venueId, tblVenue.venueName
FROM tblVenueAlternativeName
INNER JOIN tblVenue
ON tblVenue.venueId = tblVenueAlternativeName.venueId
WHERE alternativeName = venueName;
```

Selecting the venue ID and name from the venue table based on the entered postcode.

```
SELECT venueID, name
FROM tblVenue
WHERE postcode = postcode;
```

Getting a new venue ID for the venue table.

```
SELECT COUNT (*)
FROM tblVenue;
```

Inserting the new venue into the venue table.

```
INSERT INTO tblVenue;
VALUES (venueId, venueName, address, postcode, "", "");
```

Getting a new alternative ID for the alternate name table.

```
SELECT COUNT (*)
FROM tblVenueAlternativeName;
```

Inserting the new alternative venue name into the table.

```
INSERT INTO tblVenueAlternativeName
VALUES (alternativeId, venueId, alternativeName);
```

Selecting information about the food, drink, and toilet facilities based on a venue ID.

```
SELECT foodAndDrink, toilets
FROM tblVenue
WHERE venueId = venueId;
```

Updating the information about the facilities in the venue table based on the venue ID.

```
UPDATE tblVenue
SET foodAndDrink, toilets
WHERE venueID = venueId;
```

Data Structures:

Below, I have outlined the data structures I intend to use within my project, their applications, and the required functions and variables. I have highlighted any custom data type I have used by colouring it blue.

Stacks:

I will use stacks when creating the undo and redo buttons for the navigation system of my end-user application.

Variables:

Variable Name	Data Type	Private?	Initial Data
Top	Integer	Yes	-1
Items	LinkedList<T>	Yes	{}

Push:

Add an item to the top of the stack and increment the top pointer.

```
FUNCTION Push (item)
    Items ADD (item)
    Top += 1
ENDFUNCTION
```

Peek:

Return the top item from the stack.

```
FUNCTION Peek
    IF Top > -1
        RETURN Items[Top]
    ENDIF

    // Returns the default value for the given data type
    RETURN DEFAULT
ENDFUNCTION
```

Pop:

Return the top item from the stack and decrement the top pointer.

```
FUNCTION Pop
    IF Top > -1
        Item ← Peek()
        Top -= 1
    ENDIF

    // Returns the default value for the given data type
    RETURN DEFAULT
ENDFUNCTION
```

Print Stack:

Output the value of every item in the stack.

```
FUNCTION PrintStack
    FOR printIndex ← 0, printIndex <= top, printIndex += 1
        PRINT Items[printIndex]
    ENDFOR
ENDFUNCTION
```

Queues:

I will use queues during fixture processing.

Variables:

Variable Name	Data Type	Private?	Initial Data
Items	LinkedList<T>	Yes	{}
Front	Integer	Yes	0
Rear	Integer	Yes	-1

Enqueue:

Add an item to the rear of the queue and increment the rear pointer.

```
FUNCTION Enqueue (item)
    Items ADD (item)
    Rear += 1
ENDFUNCTION
```

Dequeue:

Return the first element from the queue and increment the front pointer.

```
FUNCTION Dequeue
    // Checking to see if the queue is empty
    IF Front > Rear
        // Returns the default value for the given data type
        RETURN DEFAULT
    ENDIF

    Item ← Items[Front]
    Front += 1
    RETURN Item
ENDFUNCTION
```

Size:

Return the number of elements in the queue.

```
FUNCTION Size
    RETURN Rear - Front + 1
ENDFUNCTION
```

Print Queue:

Output the value of every item in the queue.

```
FUNCTION PrintQueue
    FOR printIndex ← 0, printIndex <= top, printIndex += 1
        PRINT Items[printIndex]
    ENDFOR
ENDFUNCTION
```

Singly Linked Lists:

I will use linked lists throughout the project to replace the default list class.

Variables:

Variable Name	Data Type	Private?	Initial Data
Head Node	<code>ListNode<T></code>	Yes	Null
Length	Integer	Yes	0

List Node:

Variable Name	Data Type	Private?	Initial Data
Next	<code>ListNode<T></code>	No	Null
Data	T	No	0

Get Index:

Return the index of a given value or -1 if it's not in the list.

```
FUNCTION GetIndex (data)
    // Linear Search
    FOR searchIndex ← 0, searchIndex < length, searchIndex += 1
        IF items[searchIndex] == data
            RETURN searchIndex
        ENDIF
    ENDFOR

    RETURN -1
ENDFUNCTION
```

Get Node:

Return the node at a given index.

```
FUNCTION GetNode (index)
    IF index < 0 OR index >= length
        RETURN NEW ListNode<T>
    ENDIF

    FOR searchIndex ← 0, searchIndex < length, searchIndex += 1
        IF searchIndex == index
            RETURN items[searchIndex]
        ENDIF
    ENDFOR

    RETURN NEW ListNode<T>
ENDFUNCTION
```

Insert at the Beginning:

Insert a value at the start of the list and increment the length.

```
FUNCTION InsertAtStart (data)
    tempNode ← headNode
    headNode ← NEW ListNode<T>(data)
    headNode.next ← tempNode
    length += 1
ENDFUNCTION
```

Insert at the End:

Insert a value at the end of the list and increment the length.

```
FUNCTION InsertAtEnd (data)
    IF length == 0
        RETURN
    ENDIF

    endNode ← GetNode(length - 1)
    endNode.next ← NEW ListNode<T>(data)
    length += 1
ENDFUNCTION
```

Insert After Index:

Insert a value after the item at the given index and increment the length.

```
FUNCTION InsertAfterIndex (index, data)
    IF index <= 0
        InsertAtStart(data)
        RETURN
    ELSEIF index >= length - 1
        InsertAtEnd(data)
        RETURN
    ENDIF

    previousNode ← GetNode(index)
    nextNode ← GetNode(index + 1)

    newNode ← NEW ListNode<T>(data)
    newNode.next ← nextNode
    previousNode.next ← newNode
    length += 1
ENDFUNCTION
```

Delete At Start:

Remove the node from the start of the list and decrement the length.

```
FUNCTION DeleteAtStart
    IF length == 0
        RETURN
    ENDIF

    headNode ← headNode.next
    length -= 1
ENDFUNCTION
```

Delete At End:

Remove the node from the start of the end and decrement the length.

```
FUNCTION DeleteAtEnd
    IF length == 0
        RETURN
    ENDIF

    endNode ← GetNode(length - 2)
    endNode.next ← null
    length -= 1
ENDFUNCTION
```

Delete at Index:

Remove the node at the given index and decrement the length.

```
FUNCTION DeleteAtIndex (index)
    IF index <= 0
        DeleteAtStart()
        RETURN
    ELSEIF index >= length - 1
        DeleteAtEnd()
        RETURN
    ENDIF

    previousNode ← GetNode(index - 1)
    nextNode ← GetNode(index + 1)

    previousNode.next ← nextNode
    length -= 1
ENDFUNCTION
```

Move:

Move a node from one index to another.

```
FUNCTION Move (originalIndex, newIndex)
    IF originalIndex > -1 && originalIndex < length && newIndex > -1 &&
        newIndex < length
        RETURN
    ENDIF

    Node ← GetNode(originalIndex)
    RemoveAtIndex(originalIndex)
    InsertAtIndex(newIndex, Node)
ENDFUNCTION
```

Replace Item:

Replace the data value at an inputted index.

```
FUNCTION ReplaceItem (index, value)
    IF index > -1 && index < length
        RETURN
    ENDIF

    Node ← GetNode(originalIndex)
    Node.value ← value
ENDFUNCTION
```

Print List:

Output the value of every item in the list.

```
FUNCTION PrintList
    FOR printIndex ← 0, printIndex <= top, printIndex += 1
        PRINT Items[printIndex]
    ENDFOR
ENDFUNCTION
```

Database Structure:

My project will require an extensive database backend containing information for a login system, club, and venue data. After conducting research, I chose to create the relational database illustrated below.

Database Structure:

To create the most effective database layout, I have laid out the tables in a tabulated format with primary and foreign keys identified. Then, I converted this into the structure diagram seen below.

Tables:

tblUserAccount

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
userID	Integer	1	Yes	No
username	Text	D4f4ghd9qwxv404qCEG%\$g3dewvb4 (hashed)	No	No
password	Text	D4f4ghd9qwxv404qCEG%\$g3dewvb4 (hashed)	No	No
accountName	Text	Leo Weller	No	No

tblLeague

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
leagueID	Integer	1	Yes	No
leagueName	Text	Essex & Suffolk Border League	No	No
fullTimeID	Text	42352426	No	No
leagueFormat	Text	Open-Age Mens'	No	No

tblAccountLeagueCombination

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
combinationID	Integer	1	Yes	No
userID	Integer	1	No	Yes
leagueID	Integer	1	No	Yes
refereeID	Text	3754447	No	Yes

tblClub

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
clubID	Integer	1	Yes	No
clubName	Text	Maldon & Tiptree	No	No
homeKit	Text	Stripes: Blue, Red	No	No
awayKit	Text	Plain: Grey	No	No
thirdKit	Text	n/a	No	No

tblVenue

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
venueID	Integer	1	Yes	No
venueName	Text	Park Drive Stadium	No	No
address	Text	Maldon & Tiptree Football Club, Park Drive, Maldon	No	No
postcode	Text	CM9 5JG	No	No
foodAndDrink	Text	Not Reviewed	No	No
toilets	Text	Ok	No	No

tblVenueAlternateName

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
venueAlternateID	Integer	1	Yes	No
venueID	Integer	1	No	Yes
alternateName	Text	Park Drive	No	No

tblNotification

Field Name	Data Type	Exemplar Value	Primary Key	Foreign Key
notificationID	Integer	1	Yes	No
userID	Integer	1	No	Yes
message	Text	Failed Login Attempt: 08/02/2025 11:14, "wrongUsername", "wrongPassword"	No	No

DDL Scripts:

I am using DDL statements to create my database. This allows me to create my database tables easily according to my requirements. Below I have listed the DDL statements I used when creating my database.

tblUserAccount:

```
CREATE TABLE tblUserAccount (  
    userId          INTEGER          PRIMARY KEY,  
    username        VARCHAR(30),  
    password        VARCHAR(30),  
    accountName     VARCHAR(30)  
);
```

tblAccountLeagueCombination:

```
CREATE TABLE tblAccountLeagueCombination (  
    combinationId   INTEGER          PRIMARY KEY,  
    userId          INTEGER          FOREIGN KEY,  
    leagueId        INTEGER          FOREIGN KEY,  
    refereeId       VARCHAR(10)  
);
```

tblLeague:

```
CREATE TABLE tblLeague (  
    leagueId        INTEGER          PRIMARY KEY,  
    leagueName      VARCHAR(50),  
    fullTimeId      VARCHAR(10),  
    leagueFormat    VARCHAR(30)  
);
```

tblNotification:

```
CREATE TABLE tblNotification (  
    notificationId  INTEGER          PRIMARY KEY,  
    userId          INTEGER          FOREIGN KEY,  
    message         VARCHAR(50)  
);
```

tblClub:

```
CREATE TABLE tblClub (  
    clubId          INTEGER          PRIMARY KEY,  
    clubName        VARCHAR(30),  
    homeKit         VARCHAR(30),  
    awayKit         VARCHAR(30),  
    thirdKit        VARCHAR(30)  
);
```

tblVenue:

```
CREATE TABLE tblVenue (  
    venueId          INTEGER          PRIMARY KEY,  
    venueName        VARCHAR(30),  
    address           VARCHAR(50),  
    postcode          VARCHAR(8),  
    toilets           VARCHAR(10),  
    foodAndDrink      VARCHAR(10)  
);
```

tblVenueAlternateName:

```
CREATE TABLE tblVenueAlternateName (  
    venueAlternateId  INTEGER          PRIMARY KEY,  
    venueId           INTEGER          FOREIGN KEY,  
    alternateName     VARCHAR(30)  
);
```

Structure Diagram:

Below is the completed database structure diagram, with primary and foreign keys identified in bold and italics, respectively, and table relationships outlined with lines and arrows.

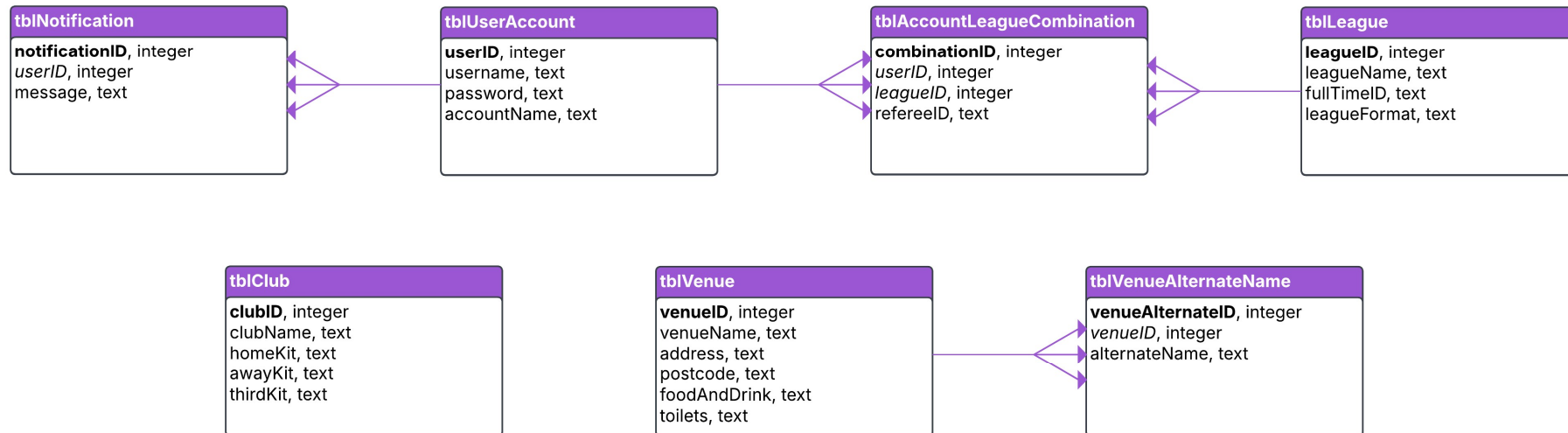


Figure 22

Technical Solution

Main File:

Band A Technical Skills:

N/a

Code:

```
1 namespace Referex___Documented
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             while (true)
8             {
9                 // Setting up the app
10                 int userId = -1;
11                 string accountName = "";
12
13                 DatabaseManager databaseConnection = new
14 DatabaseManager("database.db");
15                 FullTimeWebScraper fullTimeWebScraper = new
16 FullTimeWebScraper(databaseConnection, userId, accountName);
17
18                 Dictionary<string, Button> buttons =
19 InstantiateButtons();
20                 Button activeButton = buttons["h"];
21                 //
22
23
24                 // Logging in
25                 LoginPage loginPage = new LoginPage(accountName, ref
26 userId, buttons, databaseConnection, fullTimeWebScraper);
27                 while (loginPage.loggedIn == false)
28                 {
29                     loginPage.Display();
30                 }
31
32                 userId = loginPage.userId;
33                 accountName = loginPage.accountName;
34                 fullTimeWebScraper.userId = userId;
35                 fullTimeWebScraper.accountName = accountName;
36
37                 Output.Clear();
38                 //
39
40
41                 // Main app interface
42                 HomePage homePage = new HomePage(accountName, ref userId,
43 buttons, databaseConnection, fullTimeWebScraper);
44
45                 Page currentPage = homePage;
46                 buttons["h"].Press(buttons);
```

```

47         while (loginPage.loggedIn)
48         {
49             fullTimeWebScraper.RunWebScraper();
50             string userInput = currentPage.Display();
51             userId = currentPage.userId;
52             ProcessInput(ref loginPage.loggedIn, buttons, ref
53 currentPage, userInput, accountName, ref userId, databaseConnection,
54 fullTimeWebScraper);
55         }
56         //
57     }
58 }
59
60 }
61
62
63 static Dictionary<string, Button> InstantiateButtons()
64 {
65     Dictionary<string, Button> buttons = new Dictionary<string,
66 Button>();
67
68     buttons.Add("h", new Button("[H] Home"));
69     buttons.Add("f", new Button("[F] Fixture List"));
70     buttons.Add("s", new Button("[S] Settings"));
71     buttons.Add("N/A", new DecorationalButton("REFEREX"));
72     buttons.Add("u", new UndoRedoButton("[U] Undo"));
73     buttons.Add("r", new UndoRedoButton("[R] Redo"));
74
75     return buttons;
76 }
77
78
79 static void ProcessInput(ref bool loggedIn, Dictionary<string,
80 Button> buttons, ref Page currentPage, string userInput, string
81 accountName, ref int userId, DatabaseManager databaseConnection,
82 FullTimeWebScraper fullTimeWebScraper)
83 {
84     UndoRedoButton undoButton = (UndoRedoButton)buttons["u"];
85     UndoRedoButton redoButton = (UndoRedoButton)buttons["r"];
86
87     if (userId == -1)
88     {
89         loggedIn = false;
90         return;
91     }
92
93     if (!buttons.ContainsKey(userInput))
94     {
95         return;
96     }
97
98     if (buttons[userInput].Press(buttons))
99     {
100         switch (userInput)
101         {
102             case "h":
103                 undoButton.PushPage(currentPage);

```

```

104         redoButton.EmptyStack();
105         currentPage = new HomePage(accountName, ref
106         userId, buttons, databaseConnection, fullTimeWebScrapper);
107         return;
108
109         case "s":
110             undoButton.PushPage(currentPage);
111             redoButton.EmptyStack();
112             currentPage = new SettingsPage(accountName, ref
113         userId, buttons, databaseConnection, fullTimeWebScrapper);
114             return;
115
116         case "f":
117             undoButton.PushPage(currentPage);
118             redoButton.EmptyStack();
119             currentPage = new FixtureListPage(accountName,
120         ref userId, buttons, databaseConnection, fullTimeWebScrapper);
121             return;
122
123         case "u":
124             redoButton.PushPage(currentPage);
125             currentPage = undoButton.PopPage();
126             buttons[currentPage.pageCode].Press(buttons);
127             return;
128
129         case "r":
130             undoButton.PushPage(currentPage);
131             currentPage = redoButton.PopPage();
132             buttons[currentPage.pageCode].Press(buttons);
133             return;
134     }
135 }
136 }
137 }
138 }

```

Utility Classes:

Band A Technical Skills:

- Hashing: 222 – 239
- Recursive operations: 265 – 323
- Merge sort or similarly efficient sort: 265 – 323

Code:

```
1 using System.Security.Cryptography;
2 using System.Text;
3 using System;
4 using System.Xml;
5
6
7 public static class Input
8 {
9     public static string String(string inputMessage = "")
10    {
11        Console.Write(inputMessage + ": ");
12        return In();
13    }
14
15
16    public static Tuple<bool, int> Integer(string inputMessage = "")
17    {
18        int userInput = -1;
19        bool validInput = Int32.TryParse(String(inputMessage), out
20 userInput);
21        return Tuple.Create(validInput, userInput);
22    }
23
24
25    static string In()
26    {
27        Console.ForegroundColor = ConsoleColor.DarkGray;
28        string userInput = Console.ReadLine();
29        Console.ForegroundColor = ConsoleColor.White;
30        return userInput;
31    }
32 }
33
34
35 public static class Output
36 {
37     public static void InColour(string outputMessage, ConsoleColor
38 colour = ConsoleColor.White, bool newLine = false)
39     {
40         Console.ForegroundColor = colour;
41
42         if (newLine)
43         {
44             Console.WriteLine(outputMessage);
45         }
46         else
```

```

47         {
48             Console.Write(outputMessage);
49         }
50
51         Console.ForegroundColor = ConsoleColor.White;
52     }
53
54
55     public static void Error(string errorMessage, string exception =
56     "")
57     {
58         Output.InColour($"Error - {errorMessage}", ConsoleColor.Red,
59         true);
60
61         if (exception != "")
62         {
63             Output.InColour($"Exception - {exception}",
64             ConsoleColor.DarkGray, true);
65         }
66
67         Console.ReadKey();
68         Console.Clear();
69     }
70
71
72     public static void DuplicateStrings(string outputString, int
73     iterations, ConsoleColor colour = ConsoleColor.White)
74     {
75         for (int i = 0; i < iterations; i++)
76         {
77             Output.InColour(outputString, colour);
78         }
79     }
80
81
82     public static void Clear()
83     {
84         for (int i = 0; i < 1_000; i++)
85         {
86             Console.WriteLine("");
87         }
88     }
89
90
91     public static void Centre(string outputMessage, ConsoleColor colour
92     = ConsoleColor.White)
93     {
94         Output.DuplicateStrings(" ", (Console.WindowWidth -
95         outputMessage.Length) / 2);
96         Output.InColour(outputMessage, colour);
97     }
98
99
100    public static void Right(string outputMessage, ConsoleColor colour =
101    ConsoleColor.White)
102    {

```

```

103         Output.DuplicateStrings(" ", (Console.WindowWidth -
104         outputMessage.Length));
105         Output.InColour(outputMessage, colour);
106     }
107
108
109     public static void FillPage(int lines)
110     {
111         int verticalSpacing = Console.WindowHeight - lines;
112         for (int i = 0; i < verticalSpacing; i++)
113         {
114             Console.WriteLine("");
115         }
116     }
117
118
119     public static void Percentage(int variable1, int variable2, string
120     text)
121     {
122         int percentage;
123         if (variable2 != 0)
124         {
125             percentage = (variable1 * 100) / variable2;
126         }
127         else
128         {
129             percentage = 0;
130         }
131
132         ConsoleColor outputColour;
133         if (percentage <= 25)
134         {
135             outputColour = ConsoleColor.DarkRed;
136         }
137         else if (percentage > 75)
138         {
139             outputColour = ConsoleColor.Green;
140         }
141         else
142         {
143             outputColour = ConsoleColor.Yellow;
144         }
145
146         Output.InColour($"\\n{variable1}", outputColour);
147         Output.InColour($" / {variable2} {text} (");
148         Output.InColour($" {percentage}", outputColour);
149         Output.InColour($" %)", ConsoleColor.White, true);
150     }
151
152
153     public static void Colours()
154     {
155         string[] colourList = { "Red", "Orange", "Yellow", "Lime",
156         "Green", "Blue", "Navy", "Pink", "Purple", "White", "Black", "Grey",
157         "Sky", "Cyan" };
158         ConsoleColor[] colours = { ConsoleColor.DarkRed,
159         ConsoleColor.DarkYellow, ConsoleColor.Yellow, ConsoleColor.Green,

```

```

160 ConsoleColor.DarkGreen, ConsoleColor.Blue, ConsoleColor.DarkBlue,
161 ConsoleColor.Magenta, ConsoleColor.DarkMagenta, ConsoleColor.White,
162 ConsoleColor.DarkGray, ConsoleColor.Gray, ConsoleColor.DarkCyan,
163 ConsoleColor.Cyan};
164
165     for (int i = 0; i < colourList.Length - 1; i++)
166     {
167         Output.InColour(colourList[i] + ", ", colours[i]);
168     }
169     Output.InColour(colourList[colourList.Length - 1],
170 colours[colourList.Length - 1]);
171 }
172 }
173
174
175 public static class Format
176 {
177     public static string WebString(string text)
178     {
179         if (text == null)
180         {
181             Output.Error("Web scraping attempts blocked, please
182 enable a VPN.");
183         }
184         text = text.Trim().Replace("&", "&");
185
186         string newText = "";
187         int whitespaceCount = 0;
188         foreach (char c in text)
189         {
190             if (Char.IsLetterOrDigit(c) || c == '&' || c == ')' ||
191 c == '(')
192             {
193                 whitespaceCount = 0;
194             }
195             else
196             {
197                 whitespaceCount++;
198             }
199
200             if (whitespaceCount < 3)
201             {
202                 newText += c;
203             }
204             else
205             {
206                 newText = newText.Remove(newText.Length - 2);
207                 break;
208             }
209         }
210
211         if (newText.Contains("_"))
212         {
213             int index = newText.IndexOf('_');
214             newText = newText.Remove(index);
215         }
216

```

```

217         newText = newText.Replace(" 2024", "");
218         return newText;
219     }
220
221
222     public static string ToHash(string inputString)
223     {
224         StringBuilder stringBuilder = new StringBuilder();
225
226         using (SHA256 hash = SHA256.Create())
227         {
228             Encoding encoding = Encoding.UTF8;
229             byte[] result =
230 hash.ComputeHash(encoding.GetBytes(inputString));
231
232             foreach (byte b in result)
233             {
234                 stringBuilder.Append(b.ToString("x2"));
235             }
236         }
237
238         return stringBuilder.ToString();
239     }
240
241
242     public static DateTime StringToDateTime(string text)
243     {
244         string newText = "";
245
246         foreach (char c in text)
247         {
248             if (Char.IsDigit(c) || c == ':' || c == '/')
249             {
250                 newText += c;
251             }
252         }
253
254         int day = Convert.ToInt32(newText.Substring(0, 2));
255         int month = Convert.ToInt32(newText.Substring(3, 2));
256         int year = 2000 + Convert.ToInt32(newText.Substring(6, 2));
257
258         int hour = Convert.ToInt32(newText.Substring(8, 2));
259         int minute = Convert.ToInt32(newText.Substring(11, 2));
260
261         return new DateTime(year, month, day, hour, minute, 0);
262     }
263
264
265     public static Fixture[] MergeSortFixtureListByDate(Fixture[]
266 fixtures)
267     {
268         if (fixtures.Length <= 1)
269         {
270             return fixtures;
271         }
272
273         int midpoint = fixtures.Length / 2;

```

```

274         Fixture[] leftFixtures = Get.ArrayFromRange(fixtures, 0,
275 midpoint);
276         Fixture[] rightFixtures = Get.ArrayFromRange(fixtures,
277 midpoint, fixtures.Length);
278
279         leftFixtures = MergeSortFixtureListByDate(leftFixtures);
280         rightFixtures = MergeSortFixtureListByDate(rightFixtures);
281
282         return MergeFixtureListByDate(leftFixtures, rightFixtures);
283     }
284
285
286     public static Fixture[] MergeFixtureListByDate(Fixture[]
287 leftFixtures, Fixture[] rightFixtures)
288     {
289         int leftLength = leftFixtures.Length;
290         int rightLength = rightFixtures.Length;
291
292         Fixture[] sortedFixtures = new Fixture[leftLength +
293 rightLength];
294         int leftIndex = 0, rightIndex = 0, mergeIndex = 0;
295
296         while (leftIndex < leftLength && rightIndex < rightLength)
297         {
298             if (leftFixtures[leftIndex].kickOff >
299 rightFixtures[rightIndex].kickOff)
300             {
301                 sortedFixtures[mergeIndex++] = leftFixtures[leftIndex++];
302             }
303             else
304             {
305                 sortedFixtures[mergeIndex++] =
306 rightFixtures[rightIndex++];
307             }
308         }
309
310         while (leftIndex < leftLength)
311         {
312             sortedFixtures[mergeIndex++] =
313 leftFixtures[leftIndex++];
314         }
315         while (rightIndex < rightLength)
316         {
317             sortedFixtures[mergeIndex++] = rightFixtures[rightIndex++];
318         }
319
320         return sortedFixtures;
321     }
322 }
323
324
325 public static class Get
326 {
327     public static T[] ArrayFromRange<T>(T[] fullArray, int start, int
328 end)
329     {
330

```

```
331         T[] array = new T[end - start];
332         for (int i = start; i < end; i++)
333         {
334             array[i - start] = fullArray[i];
335         }
336         return array;
337     }
338 }
```

Database Manager:

Band A Technical Skills:

- Cross-table parameterised SQL: 59 – 93, 175 – 215
- Aggregate SQL functions: 127 – 144, 175 – 215
- User/CASE-generated DDL script: used outside the code (as detailed in design).

Code:

```
1 using System.Data.SQLite;
2 using System;
3
4
5 public class DatabaseManager
6 {
7     string databaseFilePath;
8     SQLiteConnection databaseConnection;
9     SQLiteCommand sqlCommand;
10    SQLiteDataReader sqlReader;
11    bool validConnection;
12
13
14    public DatabaseManager(string databaseFilePath)
15    {
16        this.databaseFilePath = databaseFilePath;
17
18        TryConnectToDatabase();
19        databaseConnection.Close();
20    }
21
22
23    public void TryConnectToDatabase()
24    {
25        try
26        {
27            databaseConnection = new SQLiteConnection($"Data
28 Source={databaseFilePath}; Version = 3; New = False; Compress = True;");
29            databaseConnection.Open();
30
31            sqlCommand = databaseConnection.CreateCommand();
32            validConnection = true;
33        }
34        catch (Exception exception)
35        {
36            Output.Error("Cannot connect to database.",
37 exception.Message);
38            validConnection = false;
39            databaseConnection.Close();
40        }
41    }
42
43
44    // Formatting SQL Statements
45    public string FormatInsert(string tableName, string[] data,
46 string[] dataTypes)
47    {
```

```

48         if (data.Length != dataTypes.Length)
49         {
50             Output.Error("Invalid Parameters Entered.");
51             return "";
52         }
53
54         string insertSql = $"INSERT INTO {tableName} VALUES (";
55         return insertSql + FormatData(data, dataTypes) + ");";
56     }
57
58
59     public string FormatSelect(string tableName, string[] fields,
60 string[] parameters = null, string joinTable = null, string[] tableFields
61 = null, string[] joinTableFields = null, bool left = false)
62     {
63         string selectSql = "SELECT ";
64         selectSql += FormatFields(fields);
65         selectSql += $" FROM {tableName}";
66
67         if (joinTable != null)
68         {
69             if (tableFields.Length != joinTableFields.Length)
70             {
71                 Output.Error("Invalid Parameters Entered.");
72                 return "";
73             }
74
75             if (left)
76             {
77                 selectSql += $" LEFT JOIN {joinTable} ON ";
78             }
79             else
80             {
81                 selectSql += $" INNER JOIN {joinTable} ON ";
82             }
83             selectSql += FormatSets(tableFields, joinTableFields, new
84 string[tableFields.Length]);
85         }
86
87         if (parameters != null)
88         {
89             selectSql += FormatParameters(parameters);
90         }
91
92         return selectSql + ";";
93     }
94
95
96     public string FormatDelete(string tableName, string[] parameters)
97     {
98         string deleteSql = $"DELETE FROM {tableName}";
99         deleteSql += FormatParameters(parameters);
100
101         return deleteSql + ";";
102     }
103
104

```

```

105     public string FormatUpdate(string tableName, string[] fields,
106 string[] data, string[] dataTypes, string[] parameters = null)
107     {
108         if (data.Length != dataTypes.Length || data.Length !=
109 fields.Length)
110         {
111             Output.Error("Invalid Parameters Entered.");
112             return "";
113         }
114
115         string updateSql = $"UPDATE {tableName} SET ";
116         updateSql += FormatSets(fields, data, dataTypes);
117
118         if (parameters != null)
119         {
120             updateSql += FormatParameters(parameters);
121         }
122
123         return updateSql += ";";
124     }
125
126
127     public string FormatCount(string tableName, string[] parameters =
128 null)
129     {
130         string countSql = $"SELECT COUNT (*) FROM {tableName}";
131
132         if (parameters != null)
133         {
134             countSql += FormatParameters(parameters);
135         }
136
137         return countSql + ";";
138     }
139
140
141     public string FormatMax(string tableName, string fieldName)
142     {
143         return $"SELECT MAX({fieldName}) FROM {tableName}";
144     }
145     //
146
147     // Running SQL
148     public bool TryRunNonQuery(string nonQuerySql) // Insert, Update,
149 Delete
150     {
151         TryConnectToDatabase();
152         if (validConnection == false)
153         {
154             return false;
155         }
156
157         try
158         {
159             sqlCommand.CommandText = nonQuerySql;
160             sqlCommand.ExecuteNonQuery();
161

```

```

162         databaseConnection.Close();
163         return true;
164     }
165     catch (Exception exception)
166     {
167         Output.Error($"Cannot execute Non-Query SQL.
168 \n{nonQuerySql}", exception.Message);
169         databaseConnection.Close();
170         return false;
171     }
172 }
173
174
175 public Tuple<bool, List<string[]>> TryRunQuery(string selectSql, int
176 fields) // Select, Count, Max
177 {
178     List<string[]> records = new List<string[]>();
179
180     TryConnectToDatabase();
181     if (validConnection == false)
182     {
183         return Tuple.Create(false, records);
184     }
185
186     try
187     {
188         sqlCommand.CommandText = selectSql;
189         sqlReader = sqlCommand.ExecuteReader();
190
191         while (sqlReader.Read())
192         {
193             string[] record = new string[fields];
194
195             for (int i = 0; i < fields; i++)
196             {
197                 record[i] = sqlReader.GetValue(i).ToString();
198                 //Output.InColour(record[i], ConsoleColor.Magenta);
199             }
200
201             records.InsertAtEnd(record);
202         }
203
204         databaseConnection.Close();
205         return Tuple.Create(true, records);
206     }
207     catch (Exception exception)
208     {
209         Output.Error($"Cannot execute SELECT SQL. \n{selectSql}",
210 exception.Message);
211
212         databaseConnection.Close();
213         return Tuple.Create(false, records);
214     }
215 }
216 //
217
218

```

```

219 // Formatting Functions
220 string FormatData(string data, string dataType)
221 {
222     if (dataType == "Text")
223     {
224         return $"{data.Replace("'", "'')}";
225     }
226     return data;
227 }
228
229
230 string FormatParameters(string[] parameters)
231 {
232     string parameterText = " WHERE ";
233
234     for (int i = 0; i < parameters.Length - 1; i++)
235     {
236         parameterText += parameters[i] + " AND ";
237     }
238     return parameterText + parameters[parameters.Length - 1];
239 }
240
241
242 string FormatFields(string[] fields)
243 {
244     string fieldText = "";
245
246     for (int i = 0; i < fields.Length - 1; i++)
247     {
248         fieldText += fields[i] + ", ";
249     }
250     return fieldText + fields[fields.Length - 1];
251 }
252
253
254 string FormatData(string[] data, string[] dataTypes)
255 {
256     string dataText = "";
257     for (int i = 0; i < data.Length - 1; i++)
258     {
259         dataText += FormatData(data[i], dataTypes[i]) + ", ";
260     }
261     return dataText + FormatData(data[data.Length - 1],
262 dataTypes[data.Length - 1]);
263 }
264
265
266 string FormatSets(string[] fields, string[] data, string[] dataTypes)
267 {
268     string fieldText = "";
269
270     for (int i = 0; i < fields.Length - 1; i++)
271     {
272         fieldText += fields[i] + " = " + FormatData(data[i],
273 dataTypes[i]) + ", ";
274     }

```

```

275         return fieldText + fields[fields.Length - 1] + " = " +
276 FormatData(data[fields.Length - 1], dataTypes[fields.Length - 1]);
277     }
278     //
279
280     public int GetNewId(string tableName, string columnName)
281     {
282         bool validId = false;
283         int newId = 1;
284
285         while (!validId)
286         {
287             string countSql = FormatCount(tableName, new string[] {
288 $"{columnName} = {newId}" });
289             (bool success, List<string[]> countRecords) =
290 TryRunQuery(countSql, 1);
291
292             if (!success)
293             {
294                 return -1;
295             }
296             if (countRecords[0][0] == "0")
297             {
298                 return newId;
299             }
300             newId++;
301         }
302         return -1;
303     }
304 }

```

Custom Data Structures:

Band A Technical Skills:

- List operations: 4 – 337
- Linked list maintenance: 4 – 337
- Stack / queue operations: 340 – 408

Code:

```
1 using System;
2
3
4 /// List Operations and Linked List Maintenance.
5 public class List<T>
6 {
7     ListNode<T> headNode;
8     int length;
9
10
11     public List()
12     {
13         headNode = null;
14         length = 0;
15     }
16
17
18     public int Count() { return length; }
19
20
21     // Direct Indexing
22     public T this[int index]
23     {
24         get => GetValue(index);
25         set => ReplaceItem(value, index);
26     }
27     //
28
29
30     // 'Get' Functions
31     public int GetIndex(T data)
32     {
33         ListNode<T> currentNode = headNode;
34
35         for (int i = 0; i < length; i++)
36         {
37             if (data.Equals(currentNode.data)) return i;
38             currentNode = currentNode.pointer;
39         }
40         return -1;
41     }
42
43
44     public ListNode<T> GetNode(int index)
45     {
46         if (index >= length)
47         {
```

```

48         Output.Error($"Cannot insert into list - invalid index
49 entered {index} (Range: 0 - {length - 1}).");
50         return null;
51     }
52
53     ListNode<T> currentNode = headNode;
54     for (int i = 0; i < index; i++)
55     {
56         currentNode = currentNode.pointer;
57     }
58     return currentNode;
59 }
60
61
62 public T GetValue(int index)
63 {
64     ListNode<T> node = GetNode(index);
65
66     if (node == null)
67     {
68         return default(T);
69     }
70
71     return node.data;
72 }
73 //
74
75
76 // Insert Functions
77 public void InsertAtStart(T data)
78 {
79     ListNode<T> newNode = new ListNode<T>(data);
80
81     newNode.pointer = headNode;
82     headNode = newNode;
83
84     length++;
85 }
86
87
88 public void InsertAtEnd(T data)
89 {
90     if (length == 0)
91     {
92         InsertAtStart(data);
93         return;
94     }
95
96     ListNode<T> newNode = new ListNode<T>(data);
97     GetNode(length - 1).pointer = newNode;
98
99     length++;
100 }
101
102
103 public void InsertAfter(T data, int index = 0)
104 {

```

```

105         if (index <= 0 || length == 0)
106         {
107             InsertAtStart(data);
108             return;
109         }
110         else if (index >= length - 1)
111         {
112             InsertAtEnd(data);
113             return;
114         }
115
116         ListNode<T> newNode = new ListNode<T>(data);
117         ListNode<T> previousNode = GetNode(index);
118
119         newNode.pointer = previousNode.pointer;
120         previousNode.pointer = newNode;
121
122         length++;
123     }
124
125
126     public void InsertList(List<T> items)
127     {
128         for (int i = 0; i < items.Count(); i++)
129         {
130             InsertAtEnd(items[i]);
131         }
132     }
133
134
135     public void InsertArray(T[] items)
136     {
137         for (int i = 0; i < items.Count(); i++)
138         {
139             InsertAtEnd(items[i]);
140         }
141     }
142     //
143
144
145     // Delete Functions
146     public void DeleteAtStart()
147     {
148         if (length == 0)
149         {
150             Output.Error("Cannot delete from an empty list.");
151             return;
152         }
153
154         headNode = GetNode(1);
155         length--;
156     }
157
158
159     public void DeleteAtEnd()
160     {
161         if (length == 0)

```

```

162     {
163         Output.Error("Cannot delete from an empty list.");
164         return;
165     }
166
167     GetNode(length - 1).pointer = null;
168     length--;
169 }
170
171
172 public void DeleteAt(int index)
173 {
174     if (index == 0)
175     {
176         DeleteAtStart();
177     }
178     else if (index == length - 1)
179     {
180         DeleteAtEnd();
181     }
182     if (length == 0)
183     {
184         Output.Error("Cannot delete from an empty list.");
185         return;
186     }
187
188     GetNode(index - 1).pointer = GetNode(index + 1);
189     length--;
190 }
191 //
192
193
194 // Miscellaneous Functions
195 public void Move(int oldIndex, int newIndex)
196 {
197     ListNode<T> node = GetNode(oldIndex);
198
199     DeleteAt(oldIndex);
200     InsertAfter(node.data, newIndex - 1);
201 }
202
203
204 public void ReplaceItem(T data, int index)
205 {
206     if (index > -1 && index < length)
207     {
208         ListNode<T> currentNode = headNode;
209         for (int i = 0; i < index; i++)
210         {
211             currentNode = currentNode.pointer;
212         }
213
214         currentNode.data = data;
215     }
216     else
217     {

```

```

218         Output.Error($"Cannot insert into list - invalid index
219 entered {index} (Range: 0 - {length - 1}).");
220     }
221 }
222
223
224 public void Display(bool newLine = false)
225 {
226     ListNode<T> currentNode = headNode;
227
228     for (int i = 0; i < length; i++)
229     {
230         if (newLine) Console.WriteLine(currentNode.data);
231         else Console.Write($"{currentNode.data} ");
232
233         currentNode = currentNode.pointer;
234     }
235     if (!newLine) Console.Write("\n");
236 }
237
238
239 public T[] ConvertToArray()
240 {
241     T[] array = new T[Count()];
242
243     for (int i = 0; i < array.Count(); i++)
244     {
245         array[i] = GetValue(i);
246     }
247     headNode = null;
248     length = 0;
249
250     return array;
251 }
252 //
253 }
254
255
256 public class ListNode<T>(T data)
257 {
258     public T data = data;
259     public ListNode<T> pointer = null;
260 }
261 ///
262
263
264 /// Stack Operations
265 public class Stack<T>
266 {
267     List<T> items;
268     int top;
269
270
271     public Stack()
272     {
273         items = new List<T>();
274         top = -1;

```

```

275     }
276
277
278     public void Push(T item)
279     {
280         if (items.Count() - 1 == top)
281         {
282             items.InsertAtEnd(item);
283             top++;
284
285             return;
286         }
287
288         items[++top] = item;
289     }
290
291     public T Pop()
292     {
293         if (top == -1)
294         {
295             Output.Error($"Cannot pop from an empty stack.");
296             return default(T);
297         }
298
299         return items[top--];
300     }
301
302
303     public T Peek()
304     {
305         if (top == -1)
306         {
307             Output.Error($"Cannot peek from an empty stack.");
308             return default(T);
309         }
310
311         return items[top];
312     }
313
314
315     public void Display(bool newLine = false)
316     {
317         for (int i = 0; i <= top; i++)
318         {
319             if (newLine)
320             {
321                 Console.WriteLine(items[i]);
322             }
323             else
324             {
325                 Console.Write(items[i] + " ");
326             }
327         }
328     }
329 }
330
331

```

```

332     public int Count()
333     {
334         return top + 1;
335     }
336 }
337 ///
338
339
340 /// Queue Operations
341 public class Queue<T>
342 {
343     List<T> items;
344     int front;
345     int rear;
346
347
348     public Queue()
349     {
350         items = new List<T>();
351         front = -1;
352         rear = -1;
353     }
354
355
356     public void Enqueue(T item)
357     {
358         items.InsertAtEnd(item);
359         rear++;
360
361         if (front == -1)
362         {
363             front = 0;
364         }
365     }
366
367
368     public T Dequeue()
369     {
370         if (front == -1 || front > rear)
371         {
372             Output.Error("Cannot dequeue from an empty queue.");
373             return default(T);
374         }
375
376         T item = items[front];
377         front++;
378
379         return item;
380     }
381
382
383     public int Count()
384     {
385         if (front == -1)
386         {
387             return 0;
388         }

```

```
389         return rear - front + 1;
390     }
391
392
393     public void Display(bool newLine = false)
394     {
395         for (int i = front; i <= rear; i++)
396         {
397             if (newLine)
398             {
399                 Console.WriteLine(items[i]);
400             }
401             else
402             {
403                 Console.Write(items[i] + " ");
404             }
405         }
406     }
407 }
408 ///
```

Web Scraper:

Band A Technical Skills:

n/a

Code:

```
1 using HtmlAgilityPack;
2 using System;
3
4
5 public class WebPage
6 {
7     public readonly bool validPage;
8
9     HtmlWeb web;
10    HtmlDocument page;
11
12
13    public WebPage(string url)
14    {
15        web = new HtmlWeb();
16        (validPage, page) = TryLoadPage(url);
17    }
18
19
20    bool ValidatePage(string url)
21    {
22        try
23        {
24            HtmlDocument document = web.Load(url);
25
26            if (document.Text.Length > 0)
27            {
28                return true;
29            }
30            else
31            {
32                Output.Error($"Page not found. {url}");
33                return false;
34            }
35        }
36        catch (Exception exception)
37        {
38            Output.Error($"Cannot load URL. {url}",
39exception.Message);
40            return false;
41        }
42    }
43
44
45    Tuple<bool, HtmlDocument> TryLoadPage(string url)
46    {
47        if (!ValidatePage(url))
48        {
49            return Tuple.Create<bool, HtmlDocument>(false, null);
50        }
51    }
```

```

51         return Tuple.Create(true, web.Load(url));
52     }
53
54
55     public List<HtmlNode> GetNodesFromAttribute(string tagName, string
56 attributeName, string attributeValue)
57     {
58         List<HtmlNode> collectedNodes = new List<HtmlNode>();
59         HtmlNodeCollection nodes =
60 page.DocumentNode.SelectNodes($"//{tagName}");
61
62         foreach (HtmlNode node in nodes)
63         {
64             string comparisonValue =
65 node.GetAttributeValue(attributeName, "Not Found");
66
67             if (attributeValue.Length == 0 && comparisonValue !=
68 "Not Found")
69             {
70                 collectedNodes.InsertAtEnd(node);
71             }
72             if (comparisonValue == attributeValue)
73             {
74                 collectedNodes.InsertAtEnd(node);
75             }
76         }
77     }
78     return collectedNodes;
79 }
80
81
82     public string GetInnerText(string tagName, string attributeName,
83 string attributeValue, int numberOfChildren = 0)
84     {
85         return GetAllInnerText(tagName, attributeName,
86 attributeValue, numberOfChildren)[0];
87     }
88
89
90     public List<string> GetAllInnerText(string tagName, string
91 attributeName, string attributeValue, int numberOfChildren = 0)
92     {
93         List<HtmlNode> nodes = GetNodesFromAttribute(tagName,
94 attributeName, attributeValue);
95         List<string> innerText = new List<string>();
96
97         if (nodes.Count() == 0)
98         {
99             return innerText;
100         }
101
102         for (int i = 0; i < nodes.Count(); i++)
103         {
104             HtmlNode node = nodes[i];
105             for (int c = 0; c < numberOfChildren; c++)
106             {

```

```

108         if (node.FirstChild != null)
109         {
110             node = node.FirstChild;
111         }
112     }
113     innerText.InsertAtEnd(node.InnerText);
114 }
115     return innerText;
116 }
117
118
119     public Dictionary<string, string> ConvertSelectToDictionary(string
120 selectTagName)
121     {
122         Dictionary<string, string> selectOptions = new
123 Dictionary<string, string>();
124         HtmlNode selectNode = GetNodesFromAttribute("select", "name",
125 selectTagName)[0];
126
127         if (selectNode == null)
128         {
129             return selectOptions;
130         }
131
132         foreach (HtmlNode child in selectNode.ChildNodes)
133         {
134             string key = child.InnerText;
135             string value = child.GetAttributeValue("value", "NOT FOUND");
136
137             if (value != "NOT FOUND" &&
138 !selectOptions.ContainsKey(key))
139             {
140                 selectOptions.Add(key, value);
141             }
142         }
143
144         return selectOptions;
145     }
146
147
148     public List<string> GetFixtureIds()
149     {
150         List<string> values = new List<string>();
151         List<HtmlNode> nodes = GetNodesFromAttribute("td", "class",
152 "left");
153
154         bool add = true;
155         for (int i = 0; i < nodes.Count(); i++)
156         {
157             foreach (HtmlNode child in nodes[i].ChildNodes)
158             {
159                 string link = child.GetAttributeValue("href", "NOT
160 FOUND");
161
162                 if (link != "NOT FOUND" && add)
163                 {
164                     values.InsertAtEnd(link.Substring(24));

```

```
165         }  
166         add = !add;  
167     }  
168 }  
169  
170     return values;  
171 }
```

Custom Objects:

Band A Technical Skills:

- Dynamic generation of objects based on complex user-defined use of OOP model: 4 – 136

Code:

```
1 using System;
2
3
4 public class Button
5 {
6     private readonly string text;
7     private readonly int textLength;
8     private bool active;
9     private bool selected;
10
11
12     // Getters / Setters
13     public string GetText() { return text; }
14     public int GetTextLength() { return textLength; }
15     public bool GetActive() { return active; }
16     public bool GetSelected() { return selected; }
17     public void SetActive(bool active) { this.active = active; }
18     public void SetSelected(bool selected) { this.selected = selected; }
19 }
20
21 //
22
23 public Button(string text, bool selected = false)
24 {
25     this.text = text;
26     textLength = text.Length;
27
28     active = true;
29     this.selected = selected;
30 }
31
32
33 public virtual void Display()
34 {
35     ConsoleColor colour = ConsoleColor.White;
36
37     if (selected)
38     {
39         colour = ConsoleColor.DarkRed;
40     }
41     if (active == false)
42     {
43         colour = ConsoleColor.DarkGray;
44     }
45
46     Output.InColour(text, colour);
47 }
48
49
50 public void Activate()
```

```

51     {
52         active = true;
53     }
54
55
56     public void Deactivate()
57     {
58         active = false;
59     }
60
61
62     public void Select()
63     {
64         selected = true;
65     }
66
67
68     public void Deselect()
69     {
70         selected = false;
71     }
72
73
74     public virtual bool Press(Dictionary<string, Button> buttons)
75     {
76         if (active && !selected)
77         {
78             foreach (Button button in buttons.Values)
79             {
80                 button.Deselect();
81             }
82             Select();
83             return true;
84         }
85         return false;
86     }
87 }
88
89
90 public class DecorationalButton : Button
91 {
92     public DecorationalButton(string text) : base(text, true) { }
93
94
95     public override void Display()
96     {
97         Output.InColour(text, ConsoleColor.DarkRed);
98     }
99 }
100
101
102 public class UndoRedoButton : Button
103 {
104     Stack<Page> pageStack;
105
106     public UndoRedoButton(string text) : base(text)
107     {

```

```

108         pageStack = new Stack<Page>();
109         base.Deactivate();
110     }
111
112
113     public override void Display()
114     {
115         base.Display();
116     }
117
118
119     public override bool Press(Dictionary<string, Button> buttons)
120     {
121         return active;
122     }
123
124
125     public void PushPage(Page oldPage)
126     {
127         pageStack.Push(oldPage);
128         active = true;
129     }
130
131
132     public Page PopPage()
133     {
134         if (pageStack.Count() == 1)
135         {
136             active = false;
137         }
138         return pageStack.Pop();
139     }
140
141
142     public void EmptyStack()
143     {
144         active = false;
145         pageStack = new Stack<Page>();
146     }
147 }
148
149
150 public class Fixture
151 {
152     string competition;
153     string division;
154     public DateTime kickOff;
155
156     public Team homeTeam;
157     public Team awayTeam;
158     Venue venue;
159
160
161     public Fixture(string competition, string division, DateTime
162 kickOff, Team homeTeam, Team awayTeam, Venue venue)
163     {
164         this.competition = competition;

```

```

165         this.division = division;
166         this.kickOff = kickOff;
167         this.homeTeam = homeTeam;
168         this.awayTeam = awayTeam;
169         this.venue = venue;
170     }
171
172
173     public void Display(string mode)
174     {
175         if (mode == "Simple")
176         {
177             SimpleDisplay();
178         }
179     }
180
181
182     void SimpleDisplay()
183     {
184         Console.WriteLine($"{competition} {division}
185 \n{kickOff.ToShortDateString()} - {kickOff.ToShortTimeString()}");
186
187         homeTeam.Display();
188         Console.Write(" vs. ");
189         awayTeam.Display();
190
191         Console.Write("\n");
192
193         venue.Display();
194     }
195 }
196
197
198 public class Team
199 {
200     public string name;
201     Club parentClub;
202     List<Player> players;
203
204
205     public Team(string name, List<Club> clubs)
206     {
207         this.name = name;
208
209         for (int i = 0; i < clubs.Count(); i++)
210         {
211             if (name.Contains(clubs[i].name))
212             {
213                 parentClub = clubs[i];
214                 return;
215             }
216         }
217     }
218
219
220     public void Display()
221     {

```

```

222         ConsoleColor[] kitColours = parentClub.kits["Home"];
223
224         for (int i = 0; i < name.Length; i++)
225         {
226             Output.InColour(name[i].ToString(), kitColours[i %
227 kitColours.Length]);
228         }
229     }
230 }
231
232
233 public class Venue
234 {
235     public string venueName;
236     int venueId;
237
238     string address;
239     string postcode;
240     string toilets;
241     string foodAndDrink;
242
243
244     public Venue(string venueName, string address, DatabaseManager
245 databaseConnection)
246     {
247         this.venueName = venueName;
248         this.address = address;
249
250         if (address.Length >= 7)
251         {
252             postcode = address.Substring(address.Length - 7);
253             this.address = address.Remove(address.Length - 7);
254         }
255         else
256         {
257             TryGetAddress(databaseConnection);
258         }
259
260         bool foundId = TryGetVenueId(databaseConnection);
261         if (!foundId)
262         {
263             InsertNewVenue(databaseConnection);
264         }
265
266         GetExtraInformation(databaseConnection);
267     }
268
269
270     public void Display()
271     {
272         Console.WriteLine($"{venueName} - {postcode}");
273     }
274
275
276     void TryGetAddress(DatabaseManager databaseConnection)
277     {

```

```

278         string selectSql =
279 databaseConnection.FormatSelect("tblVenue", new string[] {
280 "tblVenue.address", "tblVenue.postcode" }, new string[] {
281 $"tblVenue.venueName = '{venueName}' OR
282 tblVenueAlternateName.alternateName = '{venueName}'" },
283 "tblVenueAlternateName", new string[] { "tblVenue.venueId" }, new
284 string[] { "tblVenueAlternateName.venueId" }, true);
285         (bool success, List<string[]> records) =
286 databaseConnection.TryRunQuery(selectSql, 2);
287
288         if (success && records.Count() > 0)
289         {
290             address = records[0][0];
291             postcode = records[0][1];
292             return;
293         }
294
295         Console.WriteLine($"Missing information for {venueName}:");
296         postcode = Input.String("- Postcode").ToUpper();
297
298         selectSql = databaseConnection.FormatSelect("tblVenue", new
299 string[] { "address" }, new string[] { $"postcode = '{postcode}'" });
300         (success, records) = databaseConnection.TryRunQuery(selectSql,
301 1);
302
303         if (success && records.Count() > 0)
304         {
305             address = records[0][0];
306             return;
307         }
308
309         address = Input.String("- Address").ToUpper();
310     }
311
312     bool TryGetVenueId(DatabaseManager databaseConnection)
313     {
314         // Getting the normal ID
315         string selectSql = databaseConnection.FormatSelect("tblVenue",
316 new string[] { "venueId" }, new string[] { $"venueName = '{venueName}'"
317 });
318         (bool success, List<string[]> records) =
319 databaseConnection.TryRunQuery(selectSql, 1);
320
321         if (success && records.Count() > 0)
322         {
323             venueId = Convert.ToInt32(records[0][0]);
324             return true;
325         }
326         //
327
328         // Getting the alternate ID
329         selectSql =
330 databaseConnection.FormatSelect("tblVenueAlternateName", new string[] {
331 "tblVenue.venueId", "tblVenue.venueName" }, new string[] {

```

```

334 $"alternateName = '{venueName}'" }, "tblVenue", new string[] {
335 "tblVenue.venueId" }, new string[] { "tblVenueAlternateName.venueId" });
336 (success, records) = databaseConnection.TryRunQuery(selectSql,
337 2);
338
339     if (success && records.Count() > 0)
340     {
341         venueId = Convert.ToInt32(records[0][0]);
342         venueName = records[0][1];
343         return true;
344     }
345     //
346     return false;
347 }
348
349
350
351 void InsertNewVenue(DatabaseManager databaseConnection)
352 {
353     string selectSql =
354 databaseConnection.FormatSelect("tblVenue", new string[] { "venueId",
355 "venueName" }, new string[] { $"postcode = '{postcode}'" });
356 (bool success, List<string[]> records) =
357 databaseConnection.TryRunQuery(selectSql, 2);
358
359     string insertSql;
360     int venueId;
361     if (!success || records.Count() == 0)
362     {
363         venueId = databaseConnection.GetNewId("tblVenue",
364 "venueId");
365         insertSql = databaseConnection.FormatInsert("tblVenue",
366 new string[] { venueId.ToString(), venueName, address, postcode, "", ""
367 }, new string[] { "Integer", "Text", "Text", "Text", "Text", "Text" });
368         databaseConnection.TryRunNonQuery(insertSql);
369         return;
370     }
371
372     venueId = Convert.ToInt32(records[0][0]);
373     int alternateVenueId =
374 databaseConnection.GetNewId("tblVenueAlternateName", "venueAlternateId");
375     insertSql =
376 databaseConnection.FormatInsert("tblVenueAlternateName", new string[] {
377 alternateVenueId.ToString(), venueId.ToString(), venueName }, new
378 string[] { "Integer", "Integer", "Text" });
379     databaseConnection.TryRunNonQuery(insertSql);
380
381     venueName = records[0][1];
382 }
383
384
385 void GetExtraInformation(DatabaseManager databaseConnection)
386 {
387     string selectSql =
388 databaseConnection.FormatSelect("tblVenue", new string[] {
389 "foodAndDrink", "toilets" }, new string[] { $"venueId = {venueId}" });

```

```

390         (bool success, List<string[]> records) =
391         databaseConnection.TryRunQuery(selectSql, 2);
392
393         if (success && records.Count() > 0)
394         {
395             if (records[0][0] != "")
396             {
397                 foodAndDrink = records[0][0];
398                 toilets = records[0][1];
399                 return;
400             }
401         }
402
403         Console.WriteLine($"Missing information for {venueName}:");
404         foodAndDrink = Input.String("- Rate the food and drink");
405         toilets = Input.String("- Rate the toilet facilities");
406
407         string updateSql =
408         databaseConnection.FormatUpdate("tblVenue", new string[] {
409         "foodAndDrink", "toilets" }, new string[] { foodAndDrink, toilets }, new
410         string[] { "Text", "Text" }, new string[] { $"venueId = {venueId}" });
411         databaseConnection.TryRunNonQuery(updateSql);
412     }
413 }
414
415 public class Club
416 {
417     public string name;
418     public Dictionary<string, ConsoleColor[]> kits;
419
420     public Club(string name, Dictionary<string, ConsoleColor[]> kits)
421     {
422         this.name = name;
423         this.kits = kits;
424     }
425 }
426
427

```

Full-Time Web Scraper:

Band A Technical Skills:

n/a

Code:

```
1 using System;
2 using System.Xml.Linq;
3
4
5 public class FullTimeWebScraper
6 {
7     DatabaseManager databaseConnection;
8
9     public int numberOfLeagues;
10    public int numberOfLeaguesLoaded;
11
12    public int numberOfFixturesLoaded;
13    public int numberOfFixturesProcessed;
14
15    Queue<string> fixturesToProcess;
16    public List<Fixture> processedFixtures;
17    public Dictionary<string, string[]> leaguesLoaded;
18
19    public int userId;
20    public string accountName;
21
22    public bool working;
23    public DateTime lastScrape;
24
25    List<Club> clubs;
26
27
28    public FullTimeWebScraper(DatabaseManager databaseConnection, int
29    userId, string accountName)
30    {
31        this.databaseConnection = databaseConnection;
32
33        fixturesToProcess = new Queue<string>();
34        processedFixtures = new List<Fixture>();
35        leaguesLoaded = new Dictionary<string, string[]>();
36
37        this.userId = userId;
38        this.accountName = accountName;
39
40        numberOfLeagues = 0;
41        numberOfLeaguesLoaded = 0;
42        numberOfFixturesLoaded = 0;
43        numberOfFixturesProcessed = 0;
44
45        working = false;
46        lastScrape = new DateTime(0);
47
48        clubs = new List<Club>();
49        GetClubs();
50    }
```

```

51
52
53     public void RunWebScrapper()
54     {
55         if (lastScrape.Ticks + (30_000_000_000) > DateTime.Now.Ticks)
56         {
57             return;
58         }
59         try
60         {
61             processedFixtures = new List<Fixture>();
62             numberOfLeagues = GetLeagueCount();
63
64             Output.InColour("Loading Leagues...",
65 ConsoleColor.DarkGray);
66             LoadLeagues();
67             numberOfLeaguesLoaded = leaguesLoaded.Count();
68             Console.Clear();
69
70             Output.InColour("Getting Fixtures...",
71 ConsoleColor.DarkGray);
72             FillFixtureQueue();
73             numberOfFixturesLoaded = processedFixtures.Count() +
74 fixturesToProcess.Count();
75             Console.Clear();
76
77             Output.InColour("Processing Fixtures...",
78 ConsoleColor.DarkGray);
79             ProcessFixtures();
80             numberOfFixturesProcessed = processedFixtures.Count();
81             Console.Clear();
82
83             working = true;
84         }
85         catch (Exception exception)
86         {
87             Output.Error("Web Scrapper Failed", exception.Message);
88             working = false;
89         }
90
91         lastScrape = DateTime.Now;
92     }
93
94     int GetLeagueCount()
95     {
96         string countSql =
97 databaseConnection.FormatCount("tblAccountLeagueCombination", new
98 string[] { $"userId = {userId}" });
99
100
101         (bool success, List<string[]> records) =
102 databaseConnection.TryRunQuery(countSql, 1);
103
104         if (success)
105         {
106             return Convert.ToInt32(records[0][0]);
107         }

```

```

108         return -1;
109     }
110
111     public void LoadLeagues()
112     {
113         string selectSql =
114 databaseConnection.FormatSelect("tblLeague", new string[] {
115 "tblLeague.leagueName", "tblAccountLeagueCombination.combinationId",
116 "tblLeague.fullTimeLeagueId", "tblLeague.fullTimeSeasonId",
117 "tblAccountLeagueCombination.refereeId", "tblLeague.leagueFormat" }, new
118 string[] { $"tblAccountLeagueCombination.userId = {userId}" },
119 "tblAccountLeagueCombination", new string[] { "tblLeague.leagueId" }, new
120 string[] { "tblAccountLeagueCombination.leagueId" });
121         (bool success, List<string[]> records) =
122 databaseConnection.TryRunQuery(selectSql, 6);
123
124         if (success)
125         {
126             for (int i = 0; i < records.Count(); i++)
127             {
128                 string leagueName = records[i][0];
129                 string leagueId = records[i][2];
130
131                 if (leaguesLoaded.ContainsKey(leagueName) ==
132 false && TestLeague(leagueId, leagueName))
133                 {
134                     leaguesLoaded.Add(records[i][0], new
135 string[] { records[i][1], records[i][2], records[i][3], records[i][4],
136 records[i][5] });
137                 }
138             }
139         }
140
141     }
142
143     public bool TestLeague(string leagueId, string leagueName)
144     {
145         string url =
146 $"https://fulltime.thefa.com/index.html?league={leagueId}";
147
148         WebPage leaguePage = new WebPage(url);
149         string pageName =
150 Format.WebString(leaguePage.GetInnerText("div", "class", "league-name"));
151
152         bool valid = pageName == leagueName;
153
154         if (!valid)
155         {
156             Output.Error($"League ({leagueName}, {leagueId}) cannot
157 be found on Full-Time.");
158         }
159         return valid;
160     }
161
162     void FillFixtureQueue()
163
164

```

```

165     {
166         List<string> urls = new List<string>();
167
168         foreach (string leagueName in leaguesLoaded.Keys)
169         {
170             string combinationId = leaguesLoaded[leagueName][0];
171             string seasonId = leaguesLoaded[leagueName][2];
172             string refereeId = leaguesLoaded[leagueName][3];
173             string leagueFormat = leaguesLoaded[leagueName][4];
174
175             if (refereeId == "")
176             {
177                 refereeId = GetRefereeId(seasonId, leagueName,
178 combinationId);
179             }
180
181             urls.InsertList(FormatRefUrls(seasonId, refereeId,
182 leagueFormat));
183         }
184
185         for (int i = 0; i < urls.Count(); i++)
186         {
187             GetFixtureIds(urls[i]);
188         }
189     }
190
191
192     List<string> FormatRefUrls(string seasonId, string refereeId,
193 string leagueFormat)
194     {
195         List<string> urls = new List<string>();
196
197         WebPage page = new
198 WebPage($"https://fulltime.thefa.com/referees.html?selectedSeason={season
199 Id}&selectedFixtureGroupAgeGroup=&selectedFixtureGroupKey=&selectedRelate
200 dFixtureOption=&selectedReferee={refereeId}&selectedDateCode=");
201         Dictionary<string, string> dateCodes =
202 page.ConvertSelectToDictionary("selectedDateCode");
203
204         string url = GetRefereeUrl(seasonId, refereeId);
205         urls.InsertAtEnd(url);
206
207         return urls;
208     }
209
210
211     string GetRefereeId(string seasonId, string leagueName, string
212 combinationId)
213     {
214         WebPage page = new
215 WebPage($"https://fulltime.thefa.com/referees.html?selectedSeason={season
216 Id}&selectedFixtureGroupAgeGroup=&selectedFixtureGroupKey=&selectedRelate
217 dFixtureOption=&selectedReferee=&selectedDateCode=");
218         Dictionary<string, string> refereeIds =
219 page.ConvertSelectToDictionary("selectedReferee");
220
221

```

```

222         if (!refereeIds.ContainsKey(accountName))
223         {
224             Output.Error($"Account name ({accountName}) cannot be
225 found on the {leagueName} page.");
226             return "";
227         }
228
229         string updateSql =
230 databaseConnection.FormatUpdate("tblAccountLeagueCombination", new
231 string[] { "refereeId" }, new string[] { refereeIds[accountName] }, new
232 string[] { "Text" }, new string[] { $"combinationId = {combinationId}"
233 });
234         bool success = databaseConnection.TryRunNonQuery(updateSql);
235
236         return refereeIds[accountName];
237     }
238
239     string GetRefereeUrl(string seasonId, string refereeId)
240     {
241         return
242 $"https://fulltime.thefa.com/referees.html?selectedSeason={seasonId}&sele
243 ctedFixtureGroupAgeGroup=0&selectedFixtureGroupKey=&selectedRelatedFixtur
244 eOption=2&selectedReferee={refereeId}&selectedDateCode=";
245     }
246
247
248     void GetFixtureIds(string url)
249     {
250         WebPage fixturePage = new WebPage(url);
251         List<string> fixtureIds = fixturePage.GetFixtureIds();
252
253         for (int i = 0; i < fixtureIds.Count(); i++)
254         {
255             fixturesToProcess.Enqueue(fixtureIds[i]);
256         }
257     }
258
259     void ProcessFixtures()
260     {
261         while (fixturesToProcess.Count() > 0)
262         {
263             string fixtureUrl =
264 "https://fulltime.thefa.com/displayFixture.html?id=" +
265 fixturesToProcess.Dequeue();
266             WebPage fixturePage = new WebPage(fixtureUrl);
267
268             string competition =
269 Format.WebString(fixturePage.GetInnerText("div", "class", "league-
270 name"));
271             string division =
272 Format.WebString(fixturePage.GetInnerText("div", "class", "fa-county-
273 details"));
274
275
276

```

```

277         DateTime kickOff =
278         Format.StringToDateTime(fixturePage.GetInnerText("div", "class",
279         "fixture-date-time flex center middle"));
280
281         string homeTeamName =
282         Format.WebString(fixturePage.GetInnerText("div", "class", "home-team flex
283         middle right"));
284         string awayTeamName =
285         Format.WebString(fixturePage.GetInnerText("div", "class", "road-team flex
286         middle left"));
287
288         ValidateClubs(homeTeamName, awayTeamName);
289
290         Team homeTeam = new Team(homeTeamName, clubs);
291         Team awayTeam = new Team(awayTeamName, clubs);
292
293         string unprocessedVenueAddress =
294         fixturePage.GetInnerText("section", "id", "fixture-venue").Trim();
295         string venueName =
296         Format.WebString(unprocessedVenueAddress).Replace("Venue: ",
297         "").ToUpper().Replace("'", "");
298         string venueAddress =
299         Format.WebString(unprocessedVenueAddress.Substring(venueName.Length +
300         7)).ToUpper().Replace("'", "");
301
302         if (venueAddress.Length == 0)
303         {
304             venueAddress = "";
305         }
306
307         Venue venue = new Venue(venueName, venueAddress,
308         databaseConnection);
309
310         processedFixtures.InsertAtEnd(new Fixture(competition,
311         division, kickOff, homeTeam, awayTeam, venue));
312     }
313 }
314
315 void GetClubs()
316 {
317     string selectSql = databaseConnection.FormatSelect("tblClub",
318     new string[] { "clubName", "homeKit", "awayKit", "thirdKit" });
319     (bool success, List<string[]> records) =
320     databaseConnection.TryRunQuery(selectSql, 4);
321
322     if (!success || records.Count() == 0)
323     {
324         return;
325     }
326
327     for (int clubIndex = 0; clubIndex < records.Count();
328     clubIndex++)
329     {
330         string clubName = records[clubIndex][0];
331         ConsoleColor[] homeKit =
332         CreateKit(records[clubIndex][1]);

```

```

334         ConsoleColor[] awayKit =
335         CreateKit(records[clubIndex][2]);
336         ConsoleColor[] thirdKit =
337         CreateKit(records[clubIndex][3]);
338
339         clubs.InsertAtEnd(new Club(clubName, new
340 Dictionary<string, ConsoleColor[]> { { "Home", homeKit }, { "Away",
341 awayKit }, { "Third", thirdKit } }));
342     }
343 }
344
345
346 void ValidateClubs(string homeTeamName, string awayTeamName)
347 {
348     bool homeTeamFound = false, awayTeamFound = false;
349
350     for (int i = 0; i < clubs.Count(); i++)
351     {
352         if (homeTeamName.Contains(clubs[i].name))
353         {
354             homeTeamFound = true;
355         }
356         if (awayTeamName.Contains(clubs[i].name))
357         {
358             awayTeamFound = true;
359         }
360     }
361
362     if (homeTeamFound == false)
363     {
364         CreateClub(homeTeamName);
365     }
366     if (awayTeamFound == false)
367     {
368         CreateClub(awayTeamName);
369     }
370 }
371
372
373 void CreateClub(string teamName)
374 {
375     Console.WriteLine($"{teamName}\nClub information missing for
376 {teamName}:");
377     string clubName = Input.String("- Club Name");
378
379     Console.WriteLine("\nKits must be their constituent colours
380 separated by ;s\nYou can use: ");
381     Output.Colours();
382
383     string homeKit = Input.String("\n- Home Kit");
384     ConsoleColor[] homeKitPattern = CreateKit(homeKit);
385
386     string awayKit = Input.String("- Away Kit");
387     ConsoleColor[] awayKitPattern = CreateKit(awayKit);
388
389     string thirdKit = Input.String("- Third Kit");
390     ConsoleColor[] thirdKitPattern = CreateKit(thirdKit);

```

```

391         clubs.InsertAtEnd(new Club(clubName, new Dictionary<string,
392 ConsoleColor[]> { { "Home", homeKitPattern }, { "Away", awayKitPattern },
393 { "Third", thirdKitPattern } }));
394
395
396         int clubId = databaseConnection.GetNewId("tblClub",
397 "clubId");
398         string insertSql = databaseConnection.FormatInsert("tblClub",
399 new string[] { clubId.ToString(), clubName, homeKit, awayKit, thirdKit },
400 new string[] { "Integer", "Text", "Text", "Text", "Text" });
401         databaseConnection.TryRunNonQuery(insertSql);
402     }
403
404
405     ConsoleColor[] CreateKit(string kitString)
406     {
407         string[] colours = kitString.ToLower().Split(';');
408         ConsoleColor[] kit = new ConsoleColor[colours.Length];
409
410         if (colours[0] == "")
411         {
412             return new ConsoleColor[] { ConsoleColor.White };
413         }
414
415         for (int i = 0; i < colours.Length; i++)
416         {
417             switch (colours[i])
418             {
419                 case "red":
420                     kit[i] = ConsoleColor.DarkRed;
421                     break;
422                 case "orange":
423                     kit[i] = ConsoleColor.DarkYellow;
424                     break;
425                 case "yellow":
426                     kit[i] = ConsoleColor.Yellow;
427                     break;
428                 case "blue":
429                     kit[i] = ConsoleColor.Blue;
430                     break;
431                 case "navy":
432                     kit[i] = ConsoleColor.DarkBlue;
433                     break;
434                 case "lime":
435                     kit[i] = ConsoleColor.Green;
436                     break;
437                 case "green":
438                     kit[i] = ConsoleColor.DarkGreen;
439                     break;
440                 case "pink":
441                     kit[i] = ConsoleColor.Magenta;
442                     break;
443                 case "purple":
444                     kit[i] = ConsoleColor.DarkMagenta;
445                     break;
446                 case "white":
447                     kit[i] = ConsoleColor.White;

```

```
448         break;
449     case "black":
450         kit[i] = ConsoleColor.DarkGray;
451         break;
452     case "grey":
453         kit[i] = ConsoleColor.Gray;
454         break;
455     case "sky":
456         kit[i] = ConsoleColor.DarkCyan;
457         break;
458     case "cyan":
459         kit[i] = ConsoleColor.Cyan;
460         break;
461     }
462 }
463
464     return kit;
465 }
466 }
```

Login System:

Band A Technical Skills:

n/a

Code:

```
1 using System;
2
3
4 public class LoginSystem
5 {
6     public int userId;
7     public string accountName;
8     DatabaseManager databaseConnection;
9     int fails;
10
11
12     public LoginSystem(DatabaseManager databaseConnection)
13     {
14         this.databaseConnection = databaseConnection;
15         userId = -1;
16         accountName = "";
17         fails = 0;
18     }
19
20
21     public bool AuthenticateUser(string loginMode, string[]
22 userDetails)
23     {
24         if (loginMode == "l")
25         {
26             (userId, accountName) = AttemptLogin(userDetails);
27         }
28         else
29         {
30             (userId, accountName) = CreateAccount(userDetails);
31         }
32
33         return userId != -1;
34     }
35
36
37     public Tuple<int, string> AttemptLogin(string[] userDetails)
38     {
39         string plaintextUsername = userDetails[0];
40         string plaintextPassword = userDetails[1];
41
42         (string hashedUsername, string hashedPassword) =
43 HashDetails(plaintextUsername, plaintextPassword);
44
45         string selectSql =
46 databaseConnection.FormatSelect("tblUserAccount", new string[] {
47 "userId", "accountName" }, new string[] { $"username =
48 '{hashedUsername}', "password = '{hashedPassword}'" });
49         (bool success, List<string[]> records) =
50 databaseConnection.TryRunQuery(selectSql, 2);
```

```

51         if (success == false || records.Count() == 0)
52         {
53             LogFailedLoginAttempt(plaintextUsername,
54 hashedUsername);
55             return Tuple.Create(-1, "Invalid Username or Password.");
56         }
57
58         fails = 0;
59         return Tuple.Create(Convert.ToInt32(records[0][0]),
60 records[0][1]);
61     }
62
63
64
65     Tuple<int, string> CreateAccount(string[] userDetails)
66     {
67         if (userDetails[1] != userDetails[2] || userDetails[1].Length
68 < 8) // The passwords don't match or the password isn't long enough.
69         {
70             return Tuple.Create(-1, "The passwords don't match or the
71 password isn't long enough.");
72         }
73
74         string plaintextUsername = userDetails[0];
75         string plaintextPassword = userDetails[1];
76         string newAccountName = userDetails[3];
77
78         (string hashedUsername, string hashedPassword) =
79 HashDetails(plaintextUsername, plaintextPassword);
80
81         if (ValidateUsername(hashedUsername) == false) // Attempting to
82 use an existing username.
83         {
84             return Tuple.Create(-1, "Cannot use an existing username.");
85         }
86
87         int newUserId = databaseConnection.GetNewId("tblUserAccount",
88 "userId");
89
90         string insertSql =
91 databaseConnection.FormatInsert("tblUserAccount", new string[] {
92 newUserId.ToString(), hashedUsername, hashedPassword, newAccountName },
93 new string[] { "Integer", "Text", "Text", "Text" });
94         bool success = databaseConnection.TryRunNonQuery(insertSql);
95
96         if (success)
97         {
98             return Tuple.Create(newUserId, newAccountName);
99         }
100         return Tuple.Create(-1, "");
101     }
102
103
104     bool ValidateUsername(string username)
105     {

```

```

106         string selectSql =
107 databaseConnection.FormatSelect("tblUserAccount", new string[] {
108 "username" }, new string[] { $"username = '{username}'" });
109         (bool success, List<string[]> records) =
110 databaseConnection.TryRunQuery(selectSql, 1);
111
112         if (success)
113         {
114             return records.Count() == 0;
115         }
116         return false;
117     }
118
119
120     void LogFailedLoginAttempt(string username, string hashedUsername)
121     {
122         if (++fails >= 3)
123         {
124             Lockout(30 + (fails - 3) * 30);
125         }
126
127         string userId = "0";
128
129         string selectSql =
130 databaseConnection.FormatSelect("tblUserAccount", new string[] { "userId"
131 }, new string[] { $"username = '{hashedUsername}'" });
132         (bool success, List<string[]> records) =
133 databaseConnection.TryRunQuery(selectSql, 1);
134
135         DateTime = DateTime.Now;
136
137         string message = $"Failed Login Attempt
138 ({.ToLongDateString()}, {.ToLongTimeString()}) - Username: {username}";
139         if (success && records.Count() > 0)
140         {
141             userId = records[0][0];
142         }
143
144         string maxSql =
145 databaseConnection.FormatMax("tblNotification", "notificationId");
146         (success, records) = databaseConnection.TryRunQuery(selectSql,
147 1);
148
149         if (!success)
150         {
151             return;
152         }
153
154         int notificationId =
155 databaseConnection.GetNewId("tblNotification", "notificationId");
156
157         string insertSql =
158 databaseConnection.FormatInsert("tblNotification", new string[] {
159 notificationId.ToString(), userId, message }, new string[] { "Integer",
160 "Integer", "Text" });
161         databaseConnection.TryRunNonQuery(insertSql);
162     }

```

```
163
164
165     Tuple<string, string> HashDetails(string username, string password)
166     {
167         return Tuple.Create(Format.ToHash(username),
168 Format.ToHash(password));
169     }
170
171
172     void Lockout(int time)
173     {
174         Output.InColour($"You are now locked out for {time}
175 seconds");
176         DateTime releaseTime = DateTime.Now.AddSeconds(time);
177         while (DateTime.Now < releaseTime) continue;
178     }
179 }
```

Page Classes:

Band A Technical Skills:

- Dynamic generation of objects based on complex user-defined use of OOP model: 1 – 690

Code:

```
1 using System;
2
3
4 public class Page
5 {
6     private string pageName;
7     private string pageCode;
8
9     private string accountName;
10    private int userId;
11
12    Dictionary<string, Button> buttons;
13
14    private DatabaseManager databaseConnection;
15    private FullTimeWebScraper fullTimeWebScraper;
16
17    // Getter / Setter Functions
18    public string GetPageName() { return pageName; }
19    public string GetPageCode() { return pageCode; }
20    public string GetAccountName() { return accountName; }
21    public int GetUserId() { return userId; }
22    public DatabaseManager GetDatabaseConnection() { return
23 databaseConnection; }
24    public FullTimeWebScraper GetFullTimeWebScraper() { return
25 fullTimeWebScraper; }
26    public void SetAccountName(string newName) { accountName = newName; }
27    public void SetUserId (int newId) { userId = newId; }
28    //
29
30
31
32    public Page(string accountName, ref int userId, Dictionary<string,
33 Button> buttons, DatabaseManager databaseConnection, FullTimeWebScraper
34 fullTimeWebScraper, string pageName = "Blank Page")
35    {
36        this.pageName = pageName;
37        this.pageCode = pageName[0].ToString().ToLower();
38
39        this.accountName = accountName;
40        this.userId = userId;
41
42        this.buttons = buttons;
43
44        this.databaseConnection = databaseConnection;
45        this.fullTimeWebScraper = fullTimeWebScraper;
46    }
47
48
49    public void NavigationBar()
50    {
```

```

51         int buttonTextLength = 0;
52         foreach (Button button in buttons.Values)
53         {
54             buttonTextLength += button.textLength;
55         }
56
57         if (buttonTextLength > Console.WindowWidth)
58         {
59             Output.Error($"Button text is too long
60 ({buttonTextLength} > {Console.WindowWidth}).");
61         }
62
63         int spacing = (Console.WindowWidth - buttonTextLength) /
64 (buttons.Count() + 1);
65
66
67         foreach (Button button in buttons.Values)
68         {
69             Output.DuplicateStrings(" ", spacing);
70             button.Display();
71         }
72         Console.WriteLine("\n");
73         Output.DuplicateStrings("-", Console.WindowWidth);
74     }
75
76
77     public string InputBar(string extraText = "")
78     {
79         Output.DuplicateStrings("-", Console.WindowWidth);
80         return Input.String(extraText);
81     }
82
83
84     public virtual string Display()
85     {
86         Output.Clear();
87         NavigationBar();
88
89         Output.FillPage(3);
90
91         return InputBar();
92     }
93 }
94
95
96 public class LoginPage : Page
97 {
98     LoginSystem loginSystem;
99     public bool loggedIn;
100
101
102     public LoginPage(string accountName, ref int userId,
103 Dictionary<string, Button> buttons, DatabaseManager databaseConnection,
104 FullTimeWebScrapper fullTimeWebScrapper) : base(accountName, ref userId,
105 buttons, databaseConnection, fullTimeWebScrapper, "Login Page")
106     {
107         loggedIn = false;

```

```

108         loginSystem = new LoginSystem(databaseConnection);
109     }
110
111
112     public override string Display()
113     {
114         string loginMode = SetLoginMode();
115         string[] userDetails = TakeDetails(loginMode);
116
117         loggedIn = loginSystem.AuthenticateUser(loginMode,
118 userDetails);
119         userId = loginSystem.userId;
120         accountName = loginSystem.accountName;
121
122         if (loggedIn == false)
123         {
124             Output.Clear();
125             Output.InColour(accountName, ConsoleColor.Red);
126             Console.ReadKey();
127             Output.Clear();
128         }
129
130         return "";
131     }
132
133
134     void DisplayLogo()
135     {
136         int spacing = (Console.WindowWidth - 7) / 2;
137         Output.DuplicateStrings(" ", spacing);
138         Output.InColour("REFEREX\n", ConsoleColor.DarkRed);
139     }
140
141
142     string SetLoginMode()
143     {
144         string mode = "";
145
146         while (mode != "l" && mode != "c")
147         {
148             Output.Clear();
149             DisplayLogo();
150
151             Console.Write("\n");
152             Output.Centre("[L] Log In or [C] Create Account?");
153
154             Output.FillPage(6);
155
156             try
157             {
158                 mode = InputBar()[0].ToString().ToLower();
159             }
160             catch
161             {
162                 mode = "";
163             }
164         }

```

```

165         return mode;
166     }
167
168     string[] TakeDetails(string mode)
169     {
170         string[] userDetails = new string[4] { "", "", "", "" };
171         int detailsCount = 4;
172
173         if (mode == "l")
174         {
175             userDetails = new string[2] { "", "" };
176             detailsCount = 2;
177         }
178
179         for (int i = 0; i < detailsCount; i++)
180         {
181             Output.Clear();
182             DisplayLogo();
183
184             Output.InColour("\nUsername: ");
185             Output.InColour(userDetails[0], ConsoleColor.DarkGray);
186
187             Output.InColour("\n\nPassword: ");
188             Output.DuplicateStrings("*",
189 Convert.ToInt32(userDetails[1].Length != 0) * 10, ConsoleColor.DarkGray);
190
191             int verticalSpacing = 6;
192             if (detailsCount == 4)
193             {
194                 Output.InColour("\n\nConfirm Password: ");
195                 Output.DuplicateStrings("*",
196 Convert.ToInt32(userDetails[2].Length != 0) * 10, ConsoleColor.DarkGray);
197
198                 Output.InColour("\n\nEnter Name: ");
199                 verticalSpacing = 10;
200             }
201
202             Output.FillPage(verticalSpacing);
203
204             userDetails[i] = InputBar();
205         }
206
207         return userDetails;
208     }
209 }
210
211
212
213 public class HomePage : Page
214 {
215     public HomePage(string accountName, ref int userId,
216 Dictionary<string, Button> buttons, DatabaseManager databaseConnection,
217 FullTimeWebScraper fullTimeWebScraper, string pageName = "Home Page") :
218 base(accountName, ref userId, buttons, databaseConnection,
219 fullTimeWebScraper, pageName)
220 {
221

```

```

222
223
224     public override string Display()
225     {
226         Output.Clear();
227         NavigationBar();
228
229         Console.WriteLine("\n");
230         Output.Percentage(fullTimeWebScraper.numberOfLeaguesLoaded,
231 fullTimeWebScraper.numberOfLeagues, "Leagues Successfully Loaded");
232         Console.WriteLine("");
233         Output.Percentage(fullTimeWebScraper.numberOfFixturesProcessed,
234 fullTimeWebScraper.numberOfFixturesLoaded, "Fixtures Successfully
235 Processed");
236         Console.WriteLine("\n");
237         OutputWebScraperStatus();
238
239         Output.FillPage(15);
240
241         OutputNotifications();
242         Output.Right("[L] Log Out");
243
244         string userInput;
245         try
246         {
247             userInput = InputBar()[0].ToString().ToLower();
248         }
249         catch
250         {
251             userInput = "";
252         }
253
254         if (userInput == "l")
255         {
256             userId = -1;
257         }
258         return userInput;
259     }
260
261     void OutputNotifications()
262     {
263         string countSql =
264 databaseConnection.FormatCount("tblNotification", new string[] { $"userId
265 = {userId}" });
266         (bool success, List<string[]> records) =
267 databaseConnection.TryRunQuery(countSql, 1);
268
269         if (!success)
270         {
271             Output.Error("Cannot Display Notifications");
272             return;
273         }
274
275         int notifications = Convert.ToInt32(records[0][0]);
276
277         if (notifications == 0)
278

```

```

279         {
280             Output.Right($"You Have No Notifications",
281 ConsoleColor.DarkGray);
282         }
283         else if (notifications == 1)
284         {
285             Output.Right($"{notifications} Notification",
286 ConsoleColor.DarkYellow);
287         }
288         else
289         {
290             Output.Right($"{notifications} Notifications",
291 ConsoleColor.DarkYellow);
292         }
293     }
294
295
296     void OutputWebScrapperStatus()
297     {
298         Console.Write("Web Scraper: ");
299
300         if (fullTimeWebScrapper.working)
301         {
302             Output.InColour("Success", ConsoleColor.Green);
303         }
304         else
305         {
306             Output.InColour("Error", ConsoleColor.DarkRed);
307         }
308
309         Console.WriteLine($" -
310 {fullTimeWebScrapper.lastScrape.ToShortDateString()}
311 {fullTimeWebScrapper.lastScrape.ToShortTimeString()}");
312     }
313 }
314
315
316 public class SettingsPage : Page
317 {
318     Dictionary<string, Button> buttons;
319     DatabaseManager databaseConnection;
320
321
322     public SettingsPage(string accountName, ref int userId,
323 Dictionary<string, Button> buttons, DatabaseManager databaseConnection,
324 FullTimeWebScrapper fullTimeWebScrapper, string pageName = "Settings Page")
325 : base(accountName, ref userId, buttons, databaseConnection,
326 fullTimeWebScrapper, pageName)
327     {
328         this.buttons = buttons;
329         this.databaseConnection = databaseConnection;
330     }
331
332     public override string Display()
333     {
334         Output.Clear();
335         NavigationBar();

```

```

336         Console.WriteLine("\n");
337         Console.WriteLine("Leagues Loaded: ");
338         foreach (string leagueName in
339 fullTimeWebScraper.leaguesLoaded.Keys)
340         {
341             Output.InColour($"- {leagueName}",
342 ConsoleColor.DarkGray, true);
343         }
344
345         Output.FillPage(14 + (fullTimeWebScraper.numberOfLeagues % 6));
346
347         Console.WriteLine("\n\n- [A] Add League \n- [R] Remove League \n-
348 [C] Change Password \n- [D] Delete Account\n- [V] View Notifications
349 \n");
350
351         string userInput;
352         try
353         {
354             userInput = InputBar()[0].ToString().ToLower();
355         }
356         catch
357         {
358             userInput = "";
359         }
360
361         if (userInput == "a")
362         {
363             AddLeague();
364         }
365         else if (userInput == "r")
366         {
367             RemoveLeague();
368         }
369         else if (userInput == "c")
370         {
371             ChangePassword();
372         }
373         else if (userInput == "d")
374         {
375             DeleteAccount();
376         }
377         else if (userInput == "v")
378         {
379             ViewNotifications();
380         }
381         return userInput;
382     }
383
384
385     void AddLeague()
386     {
387         Output.Clear();
388         buttons["u"].Display();
389
390         string leagueName = Input.String("\nEnter the name of the
391 league to add");

```

```

393         if (leagueName == "u")
394         {
395             return;
396         }
397
398         (bool found, string leagueCode) =
399         TryFindLeagueCode(leagueName);
400
401         if (found == false)
402         {
403             return;
404         }
405
406         string selectSql =
407         databaseConnection.FormatSelect("tblLeague", new string[] { "leagueId" },
408         new string[] { $"fullTimeLeagueId = '{leagueCode}'" });
409         (bool success, List<string[]> records) =
410         databaseConnection.TryRunQuery(selectSql, 1);
411
412         int combinationId =
413         databaseConnection.GetNewId("tblAccountLeagueCombination",
414         "combinationId");
415
416         string insertSql =
417         databaseConnection.FormatInsert("tblAccountLeagueCombination", new
418         string[] { combinationId.ToString(), userId.ToString(), records[0][0], ""
419         }, new string[] { "Integer", "Integer", "Integer", "Text" });
420         databaseConnection.TryRunNonQuery(insertSql);
421         fullTimeWebScraper.lastScrape = new DateTime(0);
422     }
423
424
425     Tuple<bool, string> TryFindLeagueCode(string leagueName)
426     {
427         string selectSql =
428         databaseConnection.FormatSelect("tblLeague", new string[] {
429         "fullTimeLeagueId" }, new string[] { $"leagueName = '{leagueName}'" });
430         (bool success, List<string[]> records) =
431         databaseConnection.TryRunQuery(selectSql, 1);
432
433         if (success && records.Count() > 0)
434         {
435             return Tuple.Create(true, records[0][0]);
436         }
437
438         int id = databaseConnection.GetNewId("tblLeague",
439         "leagueId");
440         string leagueId = Input.String("\nEnter the league ID as it
441         appears in the URL on Full-Time");
442         string seasonId = Input.String("\nEnter the season ID as it
443         appears in the URL on Full-Time");
444         string leagueFormat = Input.String("\nEnter the league format");
445         bool validId = fullTimeWebScraper.TestLeague(leagueId,
446         leagueName);
447
448         if (!validId)

```

```

450         {
451             return Tuple.Create(false, "");
452         }
453
454         string insertSql = databaseConnection.FormatInsert("tblLeague",
455 new string[] { id.ToString(), leagueName, leagueId, seasonId,
456 leagueFormat }, new string[] { "Integer", "Text", "Text", "Text", "Text"
457 });
458         success = databaseConnection.TryRunNonQuery(insertSql);
459
460         if (success)
461         {
462             return Tuple.Create(true, leagueId);
463         }
464
465         return Tuple.Create(false, "");
466     }
467
468     void RemoveLeague()
469     {
470         Output.Clear();
471         buttons["u"].Display();
472
473         string leagueName = Input.String("\nEnter the name of the league
474 to remove");
475
476         if (leagueName == "u")
477         {
478             return;
479         }
480
481         if
482 (!fullTimeWebScraper.leaguesLoaded.ContainsKey(leagueName))
483         {
484             Output.Error("League cannot be found.");
485             return;
486         }
487
488         string selectSql =
489 databaseConnection.FormatSelect("tblLeague", new string[] { "leagueId" },
490 new string[] { $"leagueName = '{leagueName}'" });
491         (bool success, List<string[]> records) =
492 databaseConnection.TryRunQuery(selectSql, 1);
493
494         if (success == false || records.Count() == 0)
495         {
496             return;
497         }
498
499         string deleteSql =
500 databaseConnection.FormatDelete("tblAccountLeagueCombination", new
501 string[] { $"userId = {userId}", $"leagueId = '{records[0][0]}'" });
502         success = databaseConnection.TryRunNonQuery(deleteSql);
503
504         if (success)
505         {

```

```

507         fullTimeWebScraper.leaguesLoaded.Remove(leagueName);
508         fullTimeWebScraper.lastScrape = new DateTime(0);
509     }
510 }
511
512 void ChangePassword()
513 {
514     bool valid = ValidateDetails();
515
516     if (!valid)
517     {
518         return;
519     }
520
521     string newPassword = Input.String("\nNew Password");
522     string confirmPassword = Input.String("Confirm Password");
523
524     if (newPassword != confirmPassword || newPassword.Length < 8)
525     {
526         return;
527     }
528
529     string updateSql =
530 databaseConnection.FormatUpdate("tblUserAccount", new string[] {
531 "password" }, new string[] { Format.ToHash(newPassword) }, new string[] {
532 "Text" }, new string[] { $"userId = {userId}" });
533     bool success = databaseConnection.TryRunNonQuery(updateSql);
534 }
535
536 void DeleteAccount()
537 {
538     bool valid = ValidateDetails();
539
540     if (!valid)
541     {
542         return;
543     }
544
545     string confirm = Input.String("\nAre you sure? [Y/N
546 Yes/No").ToLower();
547
548     if (confirm != "y")
549     {
550         return;
551     }
552
553     string[] tables = new string[] { "tblUserAccount",
554 "tblAccountLeagueCombination", "tblNotification" };
555
556     foreach (string table in tables)
557     {
558         string deleteSql =
559 databaseConnection.FormatDelete(table, new string[] { $"userID =
560 {userId}" });
561         databaseConnection.TryRunNonQuery(deleteSql);
562     }
563 }

```

```

564     }
565
566         userId = -1;
567     }
568
569
570     bool ValidateDetails()
571     {
572         Output.Clear();
573
574         Console.WriteLine("Validate Account:");
575         string username = Input.String("- Username");
576         string password = Input.String("- Password");
577
578         LoginSystem loginSystem = new
579 LoginSystem(databaseConnection);
580
581         (int checkId, string accountName) =
582 loginSystem.AttemptLogin(new string[] { username, password });
583
584         return userId == checkId;
585     }
586
587
588     void ViewNotifications()
589     {
590         Output.Clear();
591
592         string selectSql =
593 databaseConnection.FormatSelect("tblNotification", new string[] {
594 "message" }, new string[] { $"userId = {userId}" });
595         (bool success, List<string[]> records) =
596 databaseConnection.TryRunQuery(selectSql, 1);
597
598         Console.WriteLine("Notifications:");
599         for (int i = 0; i < records.Count(); i++)
600         {
601             Output.InColour($"-{records[i][0]}", ConsoleColor.Gray,
602 true);
603         }
604
605         Console.WriteLine("\nPress any key to continue.");
606         Console.ReadKey();
607
608         string deleteSql =
609 databaseConnection.FormatDelete("tblNotification", new string[] {
610 $"userId = {userId}" });
611         databaseConnection.TryRunNonQuery(deleteSql);
612     }
613 }
614
615
616 public class FixtureListPage : Page
617 {
618     int displayedIndex;
619     bool changedPage;
620

```

```

621     public FixtureListPage(string accountName, ref int userId,
622 Dictionary<string, Button> buttons, DatabaseManager databaseConnection,
623 FullTimeWebScrapper fullTimeWebScrapper, string pageName = "Fixture List
624 Page") : base(accountName, ref userId, buttons, databaseConnection,
625 fullTimeWebScrapper, pageName)
626     {
627         displayedIndex = 0;
628         changedPage = true;
629     }
630
631
632
633     public override string Display()
634     {
635         if (changedPage)
636         {
637             SetupPage(ref fullTimeWebScrapper.processedFixtures);
638         }
639
640         Output.Clear();
641         NavigationBar();
642
643         List<Fixture> fixtureList =
644 fullTimeWebScrapper.processedFixtures;
645
646         int spacePerFixture = 4;
647         int fixtureLimit = (Console.WindowHeight - 5) / spacePerFixture;
648
649         for (int i = displayedIndex; i < fixtureLimit + displayedIndex;
650 i++)
651         {
652             Console.WriteLine("");
653
654             if (i < fixtureList.Count() && i >= 0)
655             {
656                 fixtureList[i].Display("Simple");
657             }
658             else
659             {
660                 Output.DuplicateStrings("\n", spacePerFixture);
661             }
662         }
663
664         Output.FillPage(3 + (fixtureLimit + 1) * spacePerFixture);
665
666         string userInput;
667         try
668         {
669             userInput = InputBar("[-] Previous, [+]
670 Next")[0].ToString().ToLower();
671         }
672         catch
673         {
674             userInput = "";
675         }
676
677         changedPage = true;

```

```

678         if (userInput == "-")
679         {
680             displayedIndex -= 3;
681             changedPage = false;
682         }
683         else if (userInput == "+")
684         {
685             displayedIndex += 3;
686             changedPage = false;
687         }
688         return userInput;
689     }
690
691     void SetupPage(ref List<Fixture> fixtureList)
692     {
693         Output.Clear();
694         Console.WriteLine("Sort by [D] Date or [C] Competition?");
695
696         if (Input.String().ToLower() == "d")
697         {
698             Fixture[] fixtures =
700 Format.MergeSortFixtureListByDate(fixtureList.ConvertToArray());
701             fixtureList.InsertArray(fixtures);
702         }
703     }

```

Testing

Testing Plan:

To test my project, I aim to take each of my aims and objectives and use normal, boundary, and/or erroneous data to prove that the aims and objectives have been fully met. I have chosen to separate the objective tests into groups based on the aims they are a part of. Where possible, the tests have been carried out in the normal running of the app, but some tests are carried out on specific functions as it becomes easier to pass in the relevant data for the tests. Each test will also include a time stamp relating to a place in the testing video.

Tests:

Login System:

Aim: I aim to implement a login system to ensure user account security.

Objective: The system should hash any login details stored in the database to ensure they cannot be read directly from the database or decrypted and reentered into the system.

Test: Use the login system to create a new account, enter some details, and check that the database values are hashed. Then, go back to the login system and attempt to log in using the new details.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
1	Normal	An account is created, and when correct details are entered, the user is granted access to the account.	Account was created Login was possible Details were hashed	Pass
2	Erroneous	Once the account has been created, if incorrect details are entered, the user is not granted access to the account.	Not given access to the account	Pass

Objective: To prevent duplication, the system should validate new usernames against existing users during account creation.

Test: Using the login system to create a new account, create an account using a new username, and then create a second one using the same username.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
3	Normal	When a new username is entered, the user should be able to create a new account.	Account was created	Pass
4	Erroneous	When an existing username is used, account creation should be halted, and the user should be prompted to start again.	Account creation failed	Pass

Objective: Passwords should be checked to comply with NIST guidelines for increased account security.

Test: Using the login system to create a new account, enter a password that is too short, then retry with a correct length password.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
5	Normal	When a password longer than eight characters long is entered, it should be accepted.	Account was created	Pass
6	Boundary	When a password that is precisely eight characters long is entered, it should be accepted.	Account was created	Pass
7	Erroneous	When a password less than eight characters long is entered, it should be rejected.	Account creation failed	Pass

Objective: Users should only be given a limited number of attempts to log in to the system before they are locked out, with the lock-out time increasing for each failed attempt to prevent brute-force attacks.

Test: Using the login system to attempt to log in to an existing account, enter the wrong details three times in a row to be locked out initially. Once the timeout finishes, enter the wrong information again.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
8	Normal	After three failed login attempts, the user should be locked out for thirty seconds.	Could not log in Locked out for 30 secs	Pass
9	Normal	After the fourth failed attempt, the user should be locked out for a minute.	Could not log in Locked out for 60 secs	Pass

Objective: Failed login attempts should be logged in a table, and the related user should be informed of the failed logins when they next log in.

Test: Using the login system to attempt to log in to an existing account, enter the wrong username and password, then enter the correct username and the wrong password.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
10	Normal	When the correct username but the wrong password is entered, a notification with the relevant user ID is logged.	Could not log in Attempt logged	Pass
11	Boundary	When the username and password are wrong, a notification should be logged with no user ID attached.	Could not log in Attempt logged	Pass

Web-Scraper:

Aim: I aim to create a web scraper capable of retrieving requested information from the Full-Time website.

Sub-Aim: I aim to create a set of functions to format URLs that allow users to access any page within the Full-Time website.

Objective: The web scraper should have functions to format URLs for each of the site's sub-pages, considering inputted parameters such as the referee's name, date ranges, division, or team names.

Test 1: Using the web scraper, generate a URL for the 'Home' page of a league, passing the league ID as a parameter.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
12	Normal	A URL that loads the relevant 'Home' page should be generated when a valid league ID is entered.	Each URL was correctly generated and displayed.	Pass
13	Erroneous	The program should display an error message when an invalid league ID is entered.	An error message was displayed.	Pass

Test 2: Using the web scraper, generate a URL for the 'Refs' page of a league, giving the referee's name and season ID as parameters.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
14	Normal	When the season ID is valid, and the referee's name relates to a referee on the panel for that league, a URL should be generated for the relevant 'Refs' page.	The URLs were generated and displayed.	Pass
15	Boundary	When the season ID is valid, but the referee is not on the league panel, the system should return an error message explaining that the referee could not be found.	The error message was displayed.	Pass
16	Erroneous	An error message explaining that the league season could not be loaded should be displayed when the season ID is invalid.	The error message was displayed.	Pass

Test 3: Using the web scraper, generate a URL for a fixture when given the fixture ID as a parameter.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
17	Normal	When the fixture ID is valid, the program should return a valid URL for the fixture page.	The URL was created and displayed.	Pass
18	Erroneous	When the fixture ID is invalid, an error message should display explaining that the fixture could not be found.	An error message was displayed.	Pass

Objective: The URL formatting functions should be able to deal with the differing URL formats of youth and open-age football leagues.

Test: Using the web scraper, attempt to load the league pages of both the Blackwater & Dengie Youth Football League and the Essex & Suffolk Border League.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
19	Normal	The Blackwater League should successfully load.	The league successfully loaded.	Pass
20	Normal	The Border League should successfully load.	The league successfully loaded.	Pass

Objective: Any URLs created using the formatting functions should be checked to ensure the page loads correctly before it's passed over to the web scraper to search through.

Test: Using the web scraper, run a URL generated by the system and a broken URL to check that the invalid URL is identified.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
21	Normal	When the valid URL is entered, the function should return true.	The function returned true.	Pass
22	Erroneous	When the invalid URL is entered, the function should return false.	The function returned false.	Pass

Sub-Aim: I aim to create a set of functions to retrieve information from the HTML returned when the page URLs are loaded.

Objective: The system should have a function that takes an HTML tag and attribute as parameters and returns the relevant element from the HTML page for further processing.

Test: Using the web scraper, attempt to grab the 'select' tag containing the id 'form1_selectedReferee' from the 'Refs' page.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
23	Normal	On the 'Refs' page, the 'select' tag should be found and the HTML element returned.	The tag was found.	Pass

Objective: A function should translate a 'select' tag into a dictionary so that the drop-down boxes can filter the website pages based on text parameters.

Test: Using the web scraper, attempt to grab the 'select' tag containing the id 'form1_selectedReferee' from both the 'Refs' page. Then, try to scrape the 'div' containing the class 'league-name.'

Number	Data Type	Expected Result	Actual Result	Pass/Fail
24	Normal	When attempting to scrape the 'select' tag, a dictionary should be returned with the names of referees as the keys and their relevant IDs as the values.	The keys and values were found and displayed.	Pass

Sub-Aim: I aim to use a user's list of leagues to collect a list of fixtures.

Objective: The system should run a cross-table parameterised SQL SELECT query on the user and league combination table to gather a list of leagues for the web scraper to search. If the league has been loaded previously for the user, this will return the league ID and the related referee ID.

Test: Using the database manager and the user ID returned by the login system, attempt to gather the league names and IDs relating to the user. Then, compare this with the data present in the database.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
25	Normal	If the user ID is correct and combinations are present in the database, they should all be returned.	The number of leagues returned was correct.	Pass
26	Boundary	If the user ID is correct, but there are no combinations in the database, a message explaining that no existing combinations should be returned.	The system displayed no leagues.	Pass
27	Erroneous	If the user ID is wrong, an error message explaining that the user does not exist should be displayed.	This is self moderated through the login system.	Pass

Objective: If the referee ID isn't present, the drop-down translator should be used with the drop-down box of referees on the 'Refs' page to get it. The ID can then be placed in an SQL UPDATE command for future use.

Test: Using the login system, attempt to log in to an account with referee IDs present for each league combination, then log in to an account with some league combinations missing.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
28	Normal	The complete list should be returned without further processing if each league combination has a correct referee ID.	The leagues were correctly loaded.	Pass
29	Boundary	If some combinations have referee IDs and others are missing, the existing ones should be outputted, and the others should be collected and stored before outputting.	The IDs were replaced and the leagues were loaded.	Pass

Objective: Once the referee ID has been identified, the system should collect all the fixture IDs and add them to a queue for processing.

Test: Using the Full-Time web scraper, attempt to get the fixture list for a user ID and their given leagues.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
30	Normal	If an account has multiple leagues, each league should be individually processed, and the league IDs collected should relate to the ones on the Full-Time website.	The fixture list was generated.	Pass
31	Boundary	If an account has no leagues, the queue returned should be empty.	This step was skipped.	Pass

Sub-Aim: I aim to process each fixture to collect the information on the Full-Time website.

Objective: Each fixture should be taken from the queue, and its league ID should be used to generate the URL for its sub-page.

Test: Using the web scraper, take each fixture ID and add them to the default URL before checking they work.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
32	Normal	Each fixture ID should generate a working page URL.	The URLs were generated.	Pass
33	Boundary	If the queue is empty, the program should do nothing.	This step was skipped.	Pass

Objective: This sub-page should be searched to retrieve the fixture's division/cup name, kick-off date and time, venue address, team names, and match officials' names.

Test: Using the Full-Time web scraper, input a fixture ID and return the valid information for the fixture. This should be done for fixtures of multiple league formats.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
34	Normal	When inputting a youth league fixture ID, the correct information will be returned.	The correct information was returned.	Pass
35	Normal	The correct information will be returned when an open-age league fixture is inputted.	The correct information was returned.	Pass

End-User Application:

Aim: I aim to create an end-user application that allows referees to view their upcoming fixtures for multiple leagues simultaneously faster.

Sub-Aim: I aim to implement a simple navigation system so the user can move between pages quickly and easily traverse back to previously visited pages.

Objective: The navigation bar should display all pages that can be visited and the information that must be entered to get there.

Test: When on the app, look at the navigation bar.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
41	Normal	The bar should have the 'Home', 'Settings' and 'Fixtures' pages with 'H', 'S', and 'F' next to them.	The bar was present with the correct buttons.	Pass

Objective: The input bar should accept the page's name or initial while rejecting invalid inputs.

Test: On the app, attempt to move between the pages using their full name or initials.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
42	Normal	The home page should be loaded when it's selected.	The home page loaded.	Pass
43	Normal	The settings page should be loaded when it's selected.	The settings page loaded.	Pass
44	Normal	The fixture list page should be loaded when it's selected.	The fixture list page loaded.	Pass
45	Erroneous	When invalid inputs are detected, they should be ignored.	The inputs were ignored.	Pass

Objective: The navigation bar should highlight the current page and grey out the undo and redo buttons when they cannot be used.

Test: On the app, move between pages using the page names and undo and redo buttons.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
46	Normal	The page currently loaded should have its name coloured differently on the navigation bar.	The button was coloured.	Pass
47	Boundary	When the undo and redo buttons are pressed, they should become greyed out if their stacks go to zero.	The buttons changed colours.	Pass

Objective: Once a valid input has been entered, the relevant page should be loaded, and the current page should be added to the undo button's stack.

Test: On the app, move to a new page. Then, press the undo button to return to the original page.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
48	Normal	When a valid input is entered, the correct page should be loaded, and then the undo button should take you back to the original page.	The correct page was loaded.	Pass

Objective: When the undo button is pressed, the current page should be added to the redo button's stack, and a page should be popped from the undo button's stack and loaded. The opposite should also be valid for the redo button.

Test: On the app, attempt to navigate using both the undo and redo buttons.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
49	Normal	When the undo stack has items, move back to the previous pages.	The pages were loaded.	Pass
50	Normal	Move to the 'future' pages when the redo stack has items.	The pages were loaded.	Pass
51	Boundary	If the undo stack is empty, do nothing when pressed.	Nothing happened.	Pass
52	Boundary	If the redo stack is empty, do nothing when pressed.	Nothing happened.	Pass

Sub-Aim: I aim to create a home page that the user can use to monitor the status of the Full-Time web scraper.

Objective: The home page should display the number of leagues the user has chosen to load. This should be implemented using an SQL COUNT query on the database.

Test: When on the home page, look at the number of leagues that should be loaded and compare that to the information in the database.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
53	Normal	If the user has leagues to load, the value displayed should relate to the number of records in the database.	The correct number of leagues were loaded.	Pass
54	Boundary	The home page should display zero if the user has no leagues selected in the database.	No leagues were loaded.	Pass

Objective: The page should display the current number of fixtures located and processed.

Test: On the home page, compare the number of located and processed fixtures to the number on Full-Time.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
55	Normal	If the user has leagues to search, the fixture count should match the number on the Full-Time website.	The fixture count was correct.	Pass
56	Boundary	The fixture count should equal zero if the user has no leagues to search.	No fixtures were loaded.	Pass

Objective: The page should display the for the last time the web-scraper attempted to run and whether it succeeded.

Test: When initially loading into the app, view the and compare it to the current time.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
57	Normal	They should roughly match the current time as it is run for the first time when the app is loaded.	The time was correct.	Pass

Objective: The home page should have an additional option for logging out of the system. When this is used, the user should be returned to the login screen.

Test: When on the home page, attempt to log out.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
58	Normal	The user should be returned to the login screen.	The login screen was loaded.	Pass

Sub-Aim: I aim to create a settings page for users to edit their list of leagues, change their passwords, and delete their accounts.

Objective: The settings page should display the complete list of loaded leagues and offer to edit the list.

Test: When on the settings page, look at the list of loaded leagues.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
59	Normal	Every league name should be displayed when the list contains leagues, and there should be an option to add or remove leagues.	The list of leagues was displayed.	Pass
60	Boundary	If the list of leagues is empty, there should only be the option to add leagues.	No leagues were displayed.	Pass

Objective: If the user chooses to edit the list, they should be able to choose between removing or adding a league.

Test: When on the settings page, attempt to edit the list of leagues.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
61	Normal	When the list contains leagues, there should be an option to add or remove leagues.	The option was present.	Pass
62	Boundary	If the list of leagues is empty, there should only be the option to add leagues.	The remove option wasn't present.	Pass

Objective: If the user chooses to remove a league, they should be prompted to enter the league's name from their list. An SQL DELETE command should remove the combination from the database table if a valid name is entered. If the entered name is invalid, the user should be returned to the list editing screen.

Test: Choose to remove a league from the list, type in one of the league names, and compare the database table before and after the operation.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
63	Normal	If the league name entered is already in the list, it should be removed from both the list and the database.	The league was removed.	Pass
64	Erroneous	If the league name is not on the list, nothing should happen.	The request was ignored.	Pass

Objective: If the user chooses to add a league, they should be prompted to enter the league's name. An SQL SELECT query should be used to check for the league ID in the database. The combination can be inserted into the table if the query finds one. Otherwise, the user will be directed to the Full-Time website to get the league ID. The ID entered will then be tested, and if it's valid, the league will be added to the list.

Test: Choose to add a league to the list, type in a league name and an ID, and compare the database table before and after the operation.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
65	Normal	If the league name is valid and in the database, the connection should be created, and the list should be updated.	The league was added.	Pass
66	Boundary	If the league name is valid but not in the database and the ID is valid, the connection should be created, and the list should be updated.	The league was added.	Pass
67	Boundary	If the league ID is valid but invalid, the correct name should be displayed, and the user should be prompted to accept or refuse the league. If they accept, the connection should be created, and the list should be updated.	The league was added.	Pass
68	Erroneous	Nothing should be updated if the league ID is invalid and the league name isn't in the database.	The request was ignored.	Pass

Objective: If the user chooses to edit or delete their account, they will be prompted to enter their current password. If the login system validates it, they will be allowed to proceed. Otherwise, they will be returned to the settings page.

Test: Attempt to edit the password for an account and then attempt to delete an account. Do this in coordination with the tests below.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
69	Normal	If the password is correct when trying to change the password, then proceed.	The user was able to change their password.	Pass
70	Normal	If the password is correct when trying to delete the account – proceed.	The user was able to delete their account.	Pass
71	Erroneous	Return to the settings page if the password is wrong when changing it.	The request was ignored.	Pass
72	Erroneous	Return to the settings page if the password is wrong when deleting the account.	The request was ignored.	Pass

Objective: If the user is trying to change their password, they will be prompted to enter a new password and to confirm it. If the password is valid, an SQL UPDATE command will update the login details on the system.

Test: Login, change the account password, log out, and attempt to use the old and new details.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
73	Normal	When using the correct username and password, the home page should load.	The home page was loaded.	Pass
74	Boundary	When using the old password, the details should be rejected.	The details were rejected.	Pass

Objective: If the user wants to delete their account, multiple SQL DELETE commands will remove their login details and any other records with their user ID present.

Test: Login, delete the account, and then try to log in to the account. Then, check the combination table for the old account ID.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
75	Normal	When attempting to log in, the details should be rejected, and the old user ID should not be in the database.	The details were rejected.	Pass

Sub-Aim: I aim to create a fixture list page where users can view the list of assigned fixtures based on their leagues.

Objective: When users enter the fixture list page, they should be presented with the option of or sorting their fixture list by date or competition.

Test: Enter the fixture list page and attempt to use each option.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
76	Normal	If sorting by date, the items should be placed in ascending order.	The fixtures were in the right order.	Pass
77	Normal	If sorting by competition, they should be sorted by competition.	The fixtures were in the right order.	Pass

Objective: A recursive merge sort algorithm should sort the fixture list.

Test: Choose to sort the fixture list and check that it is correct.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
78	Normal	If sorting by descending date, the fixtures should be in order with the latest fixture first.	The fixtures were in the right order.	Pass

Objective: If the user selects the simple format, each fixture should display the team names, competition and division names, the venue address, and the kick-off date and time.

Test: Load the fixture page and view the fixtures.

Number	Data Type	Expected Result	Actual Result	Pass/Fail
79	Normal	The fixture list should display the information given above.	Each fixture displayed the correct information.	Pass

Testing Video & Results:

Testing Video:

I have recorded a video containing all the above tests being carried out. It can be found here:

<https://youtu.be/mvE-rD3sLx0>

Results:

Test Category	Passes	Fails	Tests
Login System	11	0	11
Web Scraper	39	0	39
End-User Application	39	0	39
Total	79	0	79

Evaluation

Aims & Objectives:

Completeness:

During the testing phase of this project, I proved that each of the individual aims and objectives had been met. To reiterate that, I have listed my aims below with the test numbers corresponding to each.

Aim	Test Numbers
I aim to implement a login system to ensure user account security.	1-11
I aim to create a web scraper capable of retrieving requested information from the Full-Time website.	12-40
<i>I aim to create a set of functions to format URLs that allow users to access any page within the Full-Time website.</i>	12-22
<i>I aim to create a set of functions to retrieve information from the HTML returned when the page URLs are loaded.</i>	23-24
<i>I aim to use a user's list of leagues to collect a list of fixtures.</i>	25-31
<i>I aim to process each fixture to collect the information on the Full-Time website.</i>	32-35
<i>I aim to supplement the information on the Full-Time website with additional information stored in the database tables.</i>	46-40
I aim to create an end-user application that allows referees to view their upcoming fixtures for multiple leagues simultaneously faster.	41-79
<i>I aim to implement a simple navigation system so the user can move between pages quickly and easily traverse back to previously visited pages.</i>	41-52
<i>I aim to create a home page that the user can use to monitor the status of the Full-Time web scraper.</i>	53-58
<i>I aim to create a settings page for users to edit their list of leagues, change their passwords, and delete their accounts.</i>	59-75
<i>I aim to create a fixture list page where users can view the list of assigned fixtures based on their leagues and filtering preferences.</i>	76-79

As you can see, each aim has been extensively tested and proven to have been completed.

How it was Achieved:

The solution was created to complete the aims and objectives through an extensive process beginning with the analysis of the user interview, explored through research, and thoroughly planned in the design stage.

This planning was done iteratively alongside the technical solution to create the most efficient code that could adequately cover the aims and objectives. The final design of the program can be seen in the design section above.

Exit Interview:

After completing the testing stage of my project, I presented the final solution to my users to gauge opinions on the code and to get ideas for future improvements.

To do this, I have taken the original interview questions and rephrased them to discuss the solution so that I can get an accurate comparison between what the user originally wanted, and what I have created. I have removed a couple of questions that were not relevant.

- Old Questions
- New Questions
- Responses

Leo Weller Interview:

Q3 – What do you primarily use the Full-Time website for?

Q1 – What do you plan to use the Referex system for?

“I am planning on using Referex to get the information for my games because its so much easier to use than Full-Time.”

The end user prefers the system to the previously existing solution and will use it for their future refereeing.

Q4 – You mentioned the ‘bare minimum’ information. What does that include?

Q5 – Do you feel like taking that information makes you fully prepared for your games?

Q2 – What information can you now get, and is it sufficient for your games?

“Having now used Referex for a weekend of games, its so much better than Full-Time, I can get the information I need such as venue, division, and team colours without having to look in three different places.”

Leo can get all the information he wants about his games all in one place which he had detailed as an important objective at the start of the project.

Q6 – So, you say the Full-Time website is hard to navigate. Can you elaborate?

Q3 – Do you find the Referex app simpler to navigate around than Full-Time website?

“Yes. The page options and the inclusion of undo and redo buttons makes moving around the app really easy.”

They find moving around the page easy and appreciate the inclusion of the undo and redo buttons.

Q7 – What do you find challenging about the filtering options available on the Full-Time website?

Q4 – Do you find the sorting options available easy to use?

“The options seem to work well, and the sorting process completes really quickly.”

The user appreciates that the sorting options work and they agree that the merge sort algorithm implemented sorts the fixture list efficiently.

Q9 – Would you consider the app secure if it had a login system and encrypted important information?

Q5 – What are your thoughts on the login system?

“The login system is sleek to use, and the lockouts really slow down any brute force attacks. Also, the notifications option is really helpful.”

The user has noted that the login system works and that the additional features add an extra layer of security to the system.

Q6 – Do you agree that the app is secure?

“Having seen the backend database, the table of usernames and passwords is completely unreadable, so I would agree that the system is secure.”

Leo agrees that the database is secure and so is happy to have an account on the system.

Q10 – What features would you want the app to have to replace the Full-Time website?

Q7 – In the initial interview, you asked for the app to have a collated fixture list across multiple leagues that you could sort by date – do you agree the app has that?

“Yes. Referex clearly has those features.”

The user agrees that the aims and objectives that they set out for me have clearly been met.

Q8 – Are there any improvements you would like to have been implemented?

“I would have liked for there to have been extra information such as table positions and team statistic leaders, but this would need to be carefully managed to prevent too much information being on the screen at once. Additionally, a graphical interface could have worked well, but the text based one is still good.”

Leo has identified additional information from the Full-Time website that could have been transferred across to the app but accepts there is limited space when trying to prioritise readability.

He has also indicated that a graphical UI might have been nice but accepts that the text-based one allowed for features to be prioritised over looks.

REFSIX Interview:

Interview:

Q13 – Have you attempted other methods to gather the information?

Q8 – What are your thoughts on the web scraping techniques I have used to scrape the website?

“The web scraping you have done seems to be working effectively, and all the information collected is accurate.”

The client likes the way we have implemented the web scraping techniques that they had initially suggested and agrees that it is accurate to Full-Time.

Q14 – What information does your app require for referees to add games?

Q9 – Would Referex provide your users with enough information to create fixtures on your app?

“Yes, the information provided covers all the basis needed to add a fixture to REFSIX.”

The client agrees that the information provided on the fixture list page could be used to create a game on the REFSIX app.

Q15 – And is all this information available on the FT site?

Q16 – How would you like the extra information entered when creating a fixture?

Q10 – Is the information that I added to a fixture useful and is there any other bits of data you would have liked to have seen added?

“The team colours is a nice addition, but I would have liked to have seen the match format added to games with the names of all the match officials.”

Luke has identified that whilst there is some information added to games, there is still more information that could optionally be added to REFSIX games that could be in the solution.

Q17 – You mentioned that this extra information would be stored separately; how would this information be collected, and what would happen if some information wasn't available?

Q11 – Does the interaction with the database work as you expected it to, and have you encountered any issues with it?

“Yes. The database management works really smoothly.”

The client agrees that the database handling is carried out effectively and works as they would have expected it to.

Q12 – Are there any potential improvements you might want for the app?

“Not really. Apart from what I have already said, it's a really good app.”

Luke couldn't suggest any further features for the app and is very happy with the final outcome.

Improving the Solution:

Client & User Feedback:

After interviewing both Leo and Luke (REFSIX), I have collated a list of potential improvements that could be implemented into the solution in future. These are:

- Information such as table positions, goal difference, points, etc. could have been taken from Full-Time and added to each fixture in an extended match format view to give extra information to referees.
- A graphical interface could have been implemented to give a more polished feel to the app, but this would have to be carefully considered to prevent information overload on the screen.
- Match format information could have been added for referees using REFSIX. This would consist of half length, intervals, the number of players per team, substitutes, etc.
- The names of all the match officials could have been added when there are multiple assigned to a single game. This would also require the role for each to be displayed to the user.

My Opinion:

Personally, I broadly agree with the interviewees. My main point of improvement would have been to have implemented a graphical interface; however, I recognise that this would have come at the expense of other features and the overall sleekness of the final product and so this is a point for future developments.

In relation to the missing information from Full-Time, this would have been a difficult feature to balance as a major point of the initial interviews was to not fill the screen with too much information at once whilst limiting the app down to a small number of pages to simplify data collection and general navigation, so this would have been complicated to implement.

References

Glossary:

Blackwater & Dengie Youth Football League:

Affiliated to the Essex FA, the Blackwater and Dengie Youth Football League has over fifty clubs and four-hundred-and-fifty teams ([Blackwater & Dengie Youth Football League, 2025](#)).

Essex & Suffolk Border League:

A football league situated with its headquarters on the edge of both Essex & Suffolk county borders, providing adult Association Football to local clubs both located within a forty-mile radius of League Headquarters and up to Step Seven of the National Football League System ([Essex & Suffolk Border League, 2025](#)).

Essex County Women's League:

Affiliated to Essex FA, with teams split across five divisions up to Level Seven of the Women's Football Pyramid (the league doesn't have a website).

Essex Veterans League:

The Essex Veterans League is the largest Saturday league in Essex. The league has a minimum playing age of thirty-five, and eighty-three teams play across eight divisions ([Essex Veterans League, 2022](#)).

National Football League System:

The English football league system, or the football pyramid, is a series of interconnected leagues for England's men's association football clubs. The system has a hierarchical format with promotion and relegation between leagues at different levels, allowing even the most minor club the theoretical possibility of ultimately rising to the very top of the system ([Wikipedia Contributors, 2019](#)).

REFSIX:

The REFSIX app replaces a referee's notebook and allows them to record match incidents, such as goals, cautions, and dismissals, while timing the match and stoppage time ([REFSIX, 2022](#)).

Bibliography:

BDYFL Home Page:

Blackwater & Dengie Youth Football League (2025). Blackwater and Dengie Youth Football League | U7 to U18 Junior Grassroots Football League in Chelmsford, Blackwater and Dengie, Essex BDYFL. [online] bdyfl.org.

Available at: <https://bdyfl.org/index.php> [Accessed 28 Jan. 2025].

ESBL Home Page:

Essex & Suffolk Border League (2025). The Essex and Suffolk Border League Website. [online] Essex & Suffolk Border Football League.

Available at: <https://esblfootball.com/> [Accessed 28 Jan. 2025].

EVL Home Page:

Essex Veterans League (2022). Essex Veterans Football League :: Home Page. [online] essex-veterans-league.co.uk.

Available at: <https://essex-veterans-league.co.uk/> [Accessed 28 Jan. 2025].

The English Football League System:

Wikipedia Contributors (2019). English Football League System. [online] Wikipedia.

Available at: https://en.wikipedia.org/wiki/English_football_league_system [Accessed 28 Jan. 2025].

The REFSIX Team:

REFSIX (2022). Team. [online] REFSIX.

Available at: <https://refsix.com/team> [Accessed 28 Jan. 2025].

REFSIX Home Page:

REFSIX (2022). REFSIX - The App for Football/Soccer Referees. [online] REFSIX.

Available at: <https://refsix.com/> [Accessed 28 Jan. 2025].

SQLite Home Page:

SQLite (2024). SQLite Home Page. [online] www.sqlite.org.

Available at: <https://www.sqlite.org/> [Accessed 29 Jan. 2025].

SQLite Library Footprint:

SQLite (2021). SQLite Library Footprint. [online] www.sqlite.org.

Available at: <https://www.sqlite.org/footprint.html> [Accessed 25 Jun. 2024].

Self-Contained SQLite:

SQLite (2021). SQLite is a Self-Contained System. [online] www.sqlite.org.

Available at: <https://www.sqlite.org/selfcontained.html> [Accessed 25 Jun. 2024].

Benchmark Tests SQLite vs. The File System:

Davies, C. (2022). Basic Benchmarks for SQLite vs. File System. [online] GitHub

Available at: <https://github.com/chrisdavies/dbench/blob/main/readme.md> [Accessed 25 Jun. 2024].

Types of Databases:

siddhant_baroth (2020). Types of Databases. [online] GeeksForGeeks

Available at: <https://geeksforgeeks.org/types-of-databases/> [Accessed 25 Jun. 2024].

SQL Tutorial:

W3Schools (2019). SQL Tutorial. [online] W3schools.com.

Available at: <https://www.w3schools.com/sql/default.asp> [Accessed 29 Jan. 2025].

Web Scraping:

GeeksForGeeks (2020). What Is Web Scraping and How to Use It? [online] GeeksforGeeks.

Available at: <https://www.geeksforgeeks.org/what-is-web-scraping-and-how-to-use-it/> [Accessed 29 Jan. 2025].

HtmlAgilityPack Home Page:

ZZZ Projects (n.d.). Html Agility Pack | Html Agility Pack. [online] html-agility-pack.net.

Available at: <https://html-agility-pack.net/> [Accessed 29 Jan. 2025].

Encryption Algorithms & C# Libraries:

codezone (2023). 10 Common Encryption Algorithms and Libraries in C#. [online] Medium.

Available at: <https://blog.stackademic.com/10-common-encryption-algorithms-and-libraries-in-c-f95598e16b1b> [Accessed 29 Jan. 2025].

NIST Password Guidelines:

Gracy, M. (2024). NIST Password Guidelines 2024: 11 Rules to Follow. [online] Sprinto.

Available at: <https://sprinto.com/blog/nist-password-guidelines/> [Accessed 29 Jan. 2025].